

INGEGNERIA DEL SOFTWARE

MODULO 1

INTRODUZIONE

INGEGNERIA DEL SOFTWARE

L'ingegneria del software è una disciplina che si occupa della progettazione, sviluppo e gestione del software in modo strutturato e metodico. Si basa sui principi dell'ingegneria tradizionale e mira a trattare la produzione del software come un processo industriale, simile all'ingegneria meccanica o elettrica. Questo significa applicare approcci di progettazione, validazione, gestione del progetto e controllo di qualità per creare software affidabile ed efficiente.

Se l'approccio di ingegneria del software non viene seguito, possono verificarsi problemi come **scarsa qualità** del software, **manca di competitività** nel mercato e **conseguenze finanziarie** come costi e tempi di sviluppo superiori a quanto previsto. Questi problemi sono spesso chiamati "**cost overrun**" (superamento dei costi previsti) e "**time overrun**" (ritardi nello sviluppo).

In passato, la **produzione di software** non veniva considerata una **disciplina ingegneristica**, ma più come un'attività tecnica o di programmazione. L'abilità di scrivere codice e l'uso delle ultime tecnologie erano considerati sufficienti per creare software. Tuttavia, negli anni è emersa la necessità di trattare la produzione del software come una vera e propria disciplina ingegneristica.

Quello tra la teoria della programmazione e i principi dell'ingegneria è stato definito "**matrimonio non consumato**", questo si riferisce alla necessità di unire la teoria della programmazione con i principi dell'ingegneria. Questo significa che gli ingegneri del software dovrebbero avere una solida comprensione dei fondamenti teorici della programmazione, mentre i teorici dell'informatica dovrebbero comprendere i principi di progettazione e validazione dell'ingegneria.

Ingegneria del software è un termine coniato nel **1968** durante la conferenza **NATO**, in Germania, per testimoniare l'esigenza che il software fosse inquadrato all'interno di una disciplina ingegneristica.

ASPETTI TIPICI DELL'INGEGNERIA DEL SOFTWARE – PROBLEMI ACCIDENTALI

- **Accidentali del prodotto software:** Questi sono problemi o difetti nel software che possono essere superati con il progresso della tecnologia. Sono errori non legati alla natura intrinseca del software, ma piuttosto a situazioni temporanee o fattori contingenti.
- **Accidentali di attitudine:** Si riferisce ai problemi causati dalle competenze o dall'esperienza limitata del team di sviluppo. Questi possono essere superati con la formazione e l'acquisizione di competenze migliori.
- **Accidentali di manutenzione:** Questi riguardano le sfide legate alla manutenzione del software dopo il suo rilascio. La manutenzione è spesso una parte significativa del ciclo di vita del software e richiede risorse a lungo termine.
- **Accidentali di specifica e progetto:** Rappresentano le difficoltà nell'ottenere requisiti chiari e ben definiti e nel tradurli in un progetto adeguato. Una cattiva definizione dei requisiti può portare a risultati insoddisfacenti.
- **Accidentali di teaming:** Questi problemi riguardano la gestione delle dinamiche di squadra, la comunicazione interna, le divergenze di opinioni e altri aspetti legati alla collaborazione del team di sviluppo.

CICLO DI VITA DEL SW

Il ciclo di vita del software è diviso in **tre stadi** e **sei fasi**:

- **Stadio di Sviluppo:**
 1. **Requisiti:** Identificare e definire i requisiti del software.
 2. **Specifiche:** Analizzare e definire in dettaglio i requisiti raccolti.
 3. **Pianificazione:** Pianificare il processo di sviluppo, definendo tempi e risorse.
 4. **Progetto:** Creare progetti preliminari e dettagliati per l'architettura e il design del software.
 5. **Codifica:** Scrivere il codice sorgente del software.
 6. **Integrazione:** Integrare i moduli o componenti del software in un'unica soluzione.
- **Stadio di Manutenzione:** Gestire e correggere eventuali problemi nel software dopo il rilascio. Questa fase rappresenta una parte significativa dei costi del ciclo di vita del software.
- **Stadio di Dismissione:** Questo stadio coinvolge la rimozione del software obsoleto o non più supportato.

Le **6 fasi** sono molto importanti perché l'**effetto delle modifiche** varia secondo la fase in cui vengono introdotte. In fasi avanzate, una modifica può comportare rivolgimenti che richiedono nuove risorse o correzioni importanti al progetto, cioè costi supplementari. Se si svolge una modifica durante la fase dei requisiti, la correzione costerà 1; se invece avviene tra la fase di pianificazione e progettazione, costerà tra 1.5 a 6 volte; se avviene dopo il rilascio, costerà tra le 60 e 100 volte.

Il **testing** è un'attività cruciale nel ciclo di sviluppo del software, ma non è considerato una fase separata.

Viene svolto lungo l'intero ciclo in due modi:

- **Verifica:** Alla fine di ogni fase, si verifica se quella fase è stata eseguita correttamente e se i risultati sono in linea con i requisiti.
- **Validazione:** Alla fine dello sviluppo, il prodotto finale viene validato per assicurarsi che soddisfi i requisiti del cliente.

Il **Defect Removal Efficiency (DRE)** è una misura della percentuale di difetti che vengono trovati e risolti prima del rilascio del software. Un alto valore di DRE indica che molti difetti sono stati identificati e corretti durante lo sviluppo. Questa misura riflette l'efficacia del processo di controllo della qualità.

ASPETTI TIPICI DELL'INGEGNERIA DEL SOFTWARE – PROBLEMI ESSENZIALI

I problemi essenziali del prodotto sw sono quelli non superabili col progresso dei mezzi e conoscenze.

- **Complessità:** Rappresenta il livello di intricazione e interconnessione dei componenti del software. La gestione della complessità è fondamentale per sviluppare software facilmente comprensibili, manutenibili ed estendibili.
- **Conformità:** Si riferisce alla capacità del software di rispettare standard, linee guida e specifiche. La conformità è importante per garantire che il software sia compatibile con altri sistemi e che soddisfi requisiti di sicurezza e qualità.
- **Cambiabilità:** Riguarda la capacità di apportare modifiche al software senza dover riscrivere l'intero sistema. Un software facilmente adattabile alle nuove esigenze o ai cambiamenti del contesto è cruciale per il suo successo a lungo termine.
- **Invisibilità:** Si riferisce al fatto che gli utenti finali interagiscono principalmente con l'interfaccia utente del software e spesso non sono consapevoli della complessità interna. L'obiettivo è fornire un'esperienza utente intuitiva ed efficiente.

ASPETTI TIPICI DELL'INGEGNERIA DEL SOFTWARE – PROBLEMI DI COSTO

- **Costo verso Dimensione (Size):** Il costo di sviluppo del software è in parte proporzionale alla sua dimensione. Tuttavia, il costo aumenta in modo non lineare con la dimensione del software. Aumentando la dimensione, il costo potrebbe crescere in modo quadratico, come descritto dalla formula $C=aS^2$, dove C è il costo, a è una costante e S è la dimensione.
- **Costo verso Repliche:** Creare copie aggiuntive di un software già sviluppato comporta costi minimi, poiché gran parte dell'investimento iniziale è già stato effettuato.
- **Costo verso Ampiezza di Mercato:** Aumentare l'ampiezza di mercato, cioè cercare di vendere il software a un pubblico più vasto, può comportare costi significativi. Un prodotto di dimensioni doppie richiederebbe un prezzo quattro volte superiore e un mercato quattro volte più grande per garantire la stessa redditività.

DEFINIZIONI E CONCETTI CHIAVE

- **Prodotto Sw (Software):** Si riferisce al software stesso, che comprende sia il codice sorgente che la documentazione associata.
- **Artefatto:** È un prodotto software intermedio che può essere un documento di requisiti, un documento di specifica o un documento di progetto.
- **Codice:** Rappresenta il prodotto software finale, ovvero il programma informatico scritto in un linguaggio di programmazione.
- **Sistema Sw (Software System):** È l'insieme organizzato di prodotti software che lavorano insieme per raggiungere uno scopo specifico.
- **Cliente:** È il soggetto che richiede e ordina il prodotto software. In alcuni casi, il cliente può essere lo stesso sviluppatore.
- **Sviluppatore:** È il soggetto o l'organizzazione che produce il prodotto software in risposta alle richieste del cliente.
- **Utente:** È il soggetto o l'organizzazione che utilizza il prodotto software dopo che è stato sviluppato e consegnato.
- **Sw Interno:** Si riferisce a situazioni in cui il cliente e lo sviluppatore sono la stessa persona o organizzazione.
- **Sw a Contratto:** Si verifica quando il cliente e lo sviluppatore sono soggetti distinti e il prodotto software è sviluppato su base contrattuale.
- **Difetto (Defect):** È un'anomalia o un errore presente nel prodotto software.
- **Guasto (Failure):** Si verifica quando il software presenta un comportamento anomalo dovuto alla presenza di un difetto, risultando in un funzionamento non corretto o imprevisto.
- **Errore:** Si riferisce all'azione sbagliata di chi, per ignoranza, distrazione o altri motivi, introduce un difetto nel software.

AFFIDABILITA' - RELIABILITY

Informalmente, l'affidabilità si riferisce alla **credibilità** del prodotto software nel suo funzionamento. In altre parole, quanto è affidabile secondo la percezione comune. Mentre, formalmente, è una misura quantitativa della **probabilità** che il prodotto software **funzioni correttamente** entro un certo intervallo temporale.

L'affidabilità del software è legata alla quantità di **difetti** presenti. Un software con molti difetti è considerato meno affidabile. In generale, l'affidabilità aumenta man mano che il numero di difetti diminuisce. Un **software affidabile** è meno incline a guasti e problemi imprevisti e rappresenta un elemento cruciale per il successo e l'efficacia di un'applicazione o di un sistema software.

Relazione Complessa tra Affidabilità e Difetti: L'affidabilità del software presenta una relazione non semplice tra l'affidabilità osservata e il numero di difetti latenti (difetti presenti ma non ancora scoperti). Non è sempre possibile stabilire un rapporto diretto tra la presenza di difetti latenti e l'affidabilità osservata.

Regola 10-90: La regola 10-90 afferma che in programmi di notevoli dimensioni, il 90% del tempo di esecuzione totale è dedicato all'esecuzione del solo 10% delle istruzioni. Questa porzione del programma è chiamata "core" o "nucleo" del programma. Questo concetto sottolinea che alcune parti del software sono più critiche e usate più frequentemente rispetto ad altre.

Localizzazione del Difetto e Affidabilità: Il miglioramento dell'affidabilità derivante dall'eliminazione di un difetto dipende dalla posizione di quel difetto all'interno del software. In particolare, se un difetto appartiene al nucleo del programma, il suo impatto sull'affidabilità sarà maggiore rispetto a un difetto in una parte meno critica del software.

Affidabilità e Profilo Operativo: L'affidabilità osservata del software dipende da come viene utilizzato, ovvero dal suo profilo operativo. Il profilo operativo rappresenta come il software è effettivamente utilizzato dagli utenti in termini di frequenza e modalità di utilizzo.

Dipendenza dall'Utente: Poiché gli utenti utilizzano il software in modi diversi in base ai loro profili operativi, i difetti che si manifestano per un utente potrebbero non manifestarsi per un altro. Pertanto, l'affidabilità di un prodotto software dipende anche dall'utente e da come lo utilizza.

CONFRONTO TRA AFFIDABILITA' HW E SW

- **Guasti Sw (Software):** Questi si verificano a causa di difetti nei programmi. Ad esempio, un bug o un errore nel codice può causare il malfunzionamento del software. Inoltre, il software non si "consuma" fisicamente, ma può subire problemi legati alla sua esecuzione.
- **Guasti Hw (Hardware):** Questi sono spesso causati dal consumo o deterioramento dei componenti hardware, dalla deviazione rispetto alle specifiche o dalla rottura di componenti. A differenza del software, l'hardware fisico è soggetto a usura nel tempo.

I **difetti software** sono spesso "**latenti**", il che significa che possono rimanere nascosti fino a quando non si manifestano. Il software potrebbe continuare a guastarsi fino a quando i difetti non vengono corretti tramite aggiornamenti o correzioni.

A causa della differenza degli effetti dei difetti hardware e software, le **metriche** utilizzate per valutare l'affidabilità dell'hardware non possono essere facilmente estese al software.

Dopo la **riparazione dell'hardware**, la sua affidabilità torna generalmente allo stato precedente.

Dopo la **riparazione del software**, l'affidabilità può variare: potrebbe aumentare o diminuire a seconda delle correzioni effettuate.

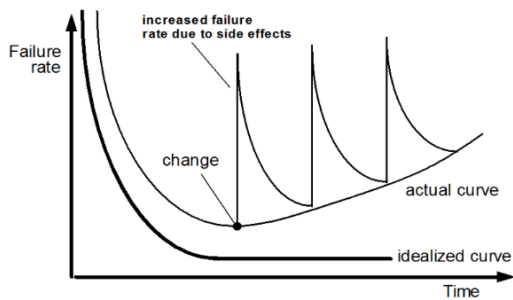
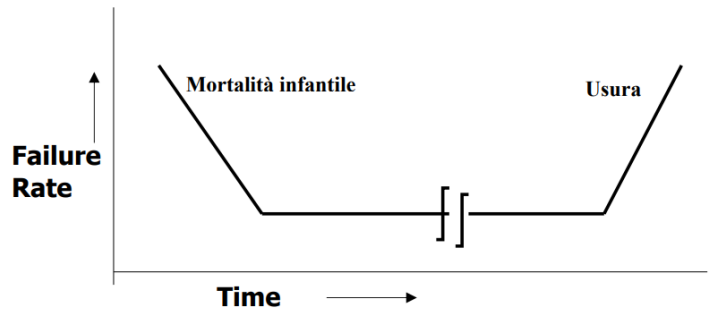
L'obiettivo dell'affidabilità dell'**hardware** è la "**stabilità**", cioè mantenere costante la frequenza di guasti nel tempo.

L'obiettivo dell'affidabilità del **software** è la "**crescita di affidabilità**", cioè ridurre la frequenza di guasti nel tempo attraverso aggiornamenti e migliorie.

ANDAMENTO FREQUENZA DI GUASTO HW

Sulle ascisse c'è il tempo, l'istante 0 corrisponde al momento in cui il prodotto viene rilasciato al cliente o sul mercato. Sulle ordinate c'è la frequenza di guasto.

Quando viene rilasciato un prodotto è molto probabile che questo manifesti dei malfunzionamenti (mortalità infantile – corrisponde ai 10 giorni per il cambio). Dopo, se si segue un corretto piano di manutenzione, l'affidabilità rimane stabile. Passato un certo numero di anni entra in gioco l'usura che incrementa la frequenza di guasto.



ANDAMENTO FREQUENZA DI GUASTO SW

Il comportamento ideale è una curva che parte con una frequenza di guasto molto alta in corrispondenza del lancio, poi dopo le modifiche del sw la frequenza di guasto diminuisce tendendo a zero. Il comportamento reale, invece, ha la mortalità infantile ma, in corrispondenza di ogni modifica del sw, la frequenza di guasto ritorna a come se fosse un prodotto appena rilasciato.

DISPONIBILITA' - AVAILABILITY

La disponibilità del software si riferisce alla **percentuale di tempo** in cui il software è utilizzabile durante la sua vita. Questo dipende dal **numero di guasti** che si verificano e dal tempo necessario per **ripararli**.

L'**affidabilità** e la **disponibilità** sono fondamentali per quei prodotti sw detti **critici**, ovvero per sistemi in cui il malfunzionamento del servizio può causare perdita di efficienza e sicurezza (perdite economiche e sociali), come ad esempio sistemi di trasporto, di governo del traffico aereo, di governo del volo, di produzione e distribuzione di energia, di comunicazione, etc.

- **THERAC 25** - è stata una macchina per la radioterapia che venne installata in 11 esemplari in alcuni ospedali degli Stati Uniti e del Canada; è stata affetta da una serie di bug nel programma informatico responsabile del suo funzionamento, che provocarono sei incidenti durante i quali ai pazienti venne somministrata una dose di radiazioni 100 volte superiore a quella necessaria (in alcuni casi l'apparecchio si attivò anche senza aver ricevuto l'apposito comando), causando un avvelenamento da radiazioni e, direttamente, il decesso di tre dei sei pazienti.
- **ARIANE ESA** - il razzo si autodistrusse dopo 40 secondi dal lancio per via di un malfunzionamento del software di controllo; un dato a 64 bit in virgola mobile venne convertito in un intero a 16 bit con segno, questa operazione causò una trap del processore (operazione errata). Motivi di efficienza avevano spinto i progettisti a disabilitare il controllo software sulle trap, anche se altre conversioni simili nel codice erano corrette.

PROCESSO SOFTWARE

Il processo software è una sequenza di attività che sono necessarie per sviluppare un **prodotto software** con le caratteristiche di qualità desiderate, rispettando i tempi e i costi previsti. Nel processo, vengono applicati metodi, tecniche e strumenti, si producono prodotti intermedi e finali, si controlla il progetto e si gestiscono le modifiche.

Il processo software si sviluppa attraverso un **ciclo di vita** che comprende **tre stadi: sviluppo, manutenzione e dismissione**.

Nello **stadio di sviluppo**, ci sono due tipi di fasi:

- Le **fasi di definizione** si concentrano su cosa il software deve fare, includendo la definizione dei requisiti e la produzione delle specifiche.
- Le **fasi di produzione** si concentrano su come realizzare ciò che è stato definito nelle fasi precedenti. Qui si progetta, si codifica, si integra e si consegna il software al cliente.

Lo **stadio di manutenzione** è dedicato al supporto del software realizzato e coinvolge fasi sia di definizione che di produzione. Durante ogni fase si procede ad effettuare il **testing** di quanto prodotto, mediante opportune tecniche di **verifica e validazione (V&V)** applicate sia ai prodotti intermedi che al prodotto finale.

TIPI DI MANUTENZIONE

- **Manutenzione Correttiva:** Ha l'obiettivo di eliminare i difetti nel software che causano guasti o malfunzionamenti.
- **Manutenzione Adattativa:** Si concentra sull'adattamento del software a cambiamenti nell'ambiente operativo in cui il software è in uso.
- **Manutenzione Perfettiva:** Si occupa di estendere il software per aggiungere nuove funzionalità o miglioramenti.
- **Manutenzione Preventiva (o Software Reengineering):** Coinvolge modifiche al software per semplificare le future correzioni, adattamenti e miglioramenti.

MODELLI DI CICLO DI VITA

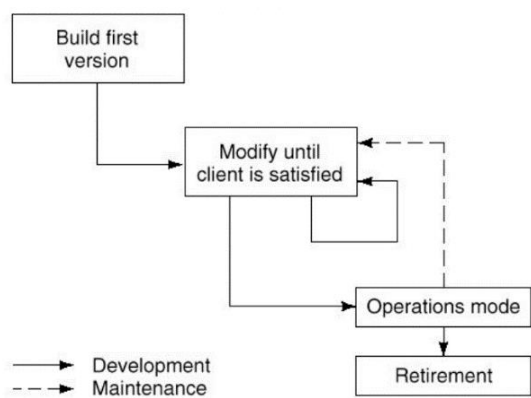
Secondo la definizione dello **standard IEEE Std 610-12**, il **ciclo di vita del software** è l'intervallo di tempo che va dall'origine dell'esigenza di costruire un prodotto software fino al momento in cui il prodotto stesso viene dismesso. Questo intervallo include una serie di fasi attraverso le quali il software progredisce, tra cui definizione dei requisiti, specifica, pianificazione, progettazione preliminare, progettazione dettagliata, codifica, integrazione, testing, uso, manutenzione e dismissione. È importante notare che queste fasi **possono sovrapporsi** o essere eseguite in modo **iterativo**, a seconda del modello di sviluppo adottato.

Un **modello di ciclo di vita** del software definisce la sequenza di fasi che il prodotto software attraversa, dall'inizio alla fine del suo sviluppo, inclusi gli ordini di esecuzione di tali fasi. La **scelta** di un modello di ciclo di vita dipende da vari fattori, come la natura dell'applicazione, la maturità dell'organizzazione, i metodi e le tecnologie utilizzate e i vincoli imposti dal cliente.

RETTANGOLI = attività/fasi di sviluppo

FRECCE CONTINUE = attività condotte in fase sviluppo

FRECCE TRATTEGGIATE = attività condotte in fase di manutenzione



BUILD & FIX

L'**assenza** di un modello del ciclo di vita corrisponde ad una modalità di sviluppo detta "**Build & Fix**" (o "fix-it-later"), in cui il prodotto software viene sviluppato e successivamente rilavorato fino a soddisfare le necessità del cliente. Questa è una modalità di sviluppo molto costosa e poco utilizzata.

BUILD FIRST VERSION = codifica prodotto

MODIFY UNTIL CLIENT IS SATISFIED = prodotto rilasciato al cliente e rilavorato fino a quando il cliente non è soddisfatto

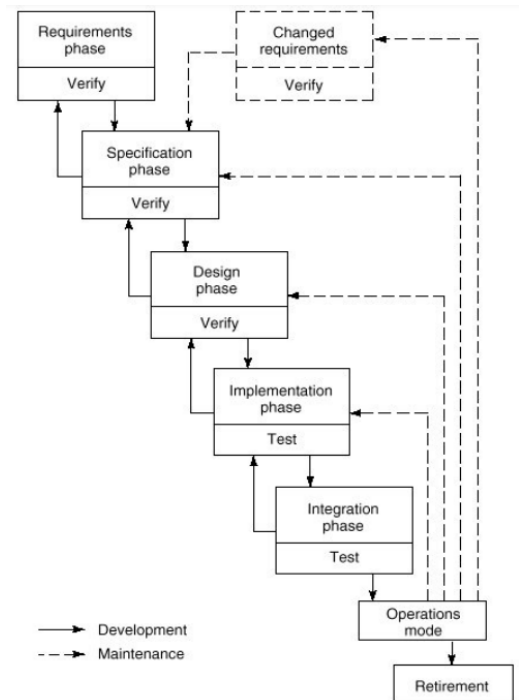
OPERATION MODE = prodotto entra in modalità operativa quando è accettato dal cliente

RETIREMENT = prodotto viene dismesso

MODELLO WATERFALL

Il modello Waterfall è uno dei primi e più noti modelli di ciclo di vita del software. Esso rappresenta un approccio sequenziale allo sviluppo del software, dove le attività vengono eseguite in modo **lineare e sequenziale**, come una cascata, passando da una fase all'altra in modo graduale e unidirezionale. Ecco una spiegazione dettagliata di ciascuna fase:

- **Definizione dei Requisiti:**
 - Inizia con la raccolta dei requisiti del sistema, ovvero i bisogni e le specifiche dell'applicazione che si sta sviluppando.
 - I requisiti vengono documentati in dettaglio, in modo da creare una base chiara per lo sviluppo successivo.
- **Progettazione:**
 - Basandosi sui requisiti, si progetta l'architettura del sistema e si definiscono i dettagli tecnici.
 - Questa fase si concentra sul "come" il sistema soddisferà i requisiti stabiliti nella fase precedente.
- **Sviluppo:**
 - Qui, il codice sorgente viene scritto sulla base del progetto stabilito nella fase di progettazione.
 - I programmatori traducono il design in codice eseguibile.
- **Testing:**
 - Il software sviluppato viene sottoposto a rigorosi test per identificare difetti e verificare che il software funzioni come previsto.
 - Si effettuano test di unità, integrazione e sistema.
- **Validazione:**
 - In questa fase, il software viene validato rispetto ai requisiti iniziali.
 - Ciò implica verificare che il software soddisfi le esigenze degli utenti finali e funzioni correttamente nell'ambiente di utilizzo.
- **Consegna e Manutenzione:**
 - Una volta che il software ha superato con successo il testing e la validazione, viene rilasciato ai clienti o utenti finali.
 - Inizia la fase di manutenzione, che comprende la correzione di eventuali bug e l'implementazione di modifiche richieste o nuove funzionalità.

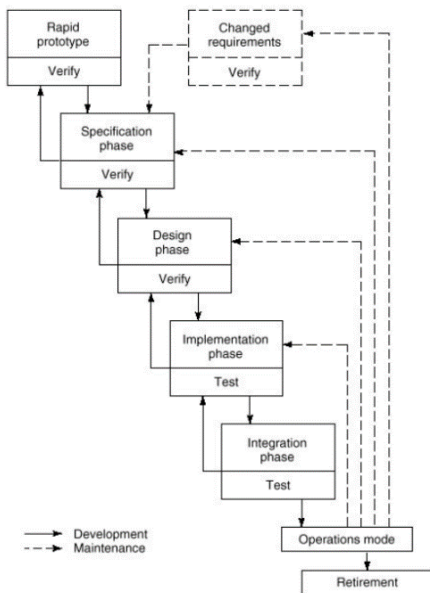


Il modello Waterfall è caratterizzato dalla natura sequenziale delle fasi, dove ciascuna fase è completata prima di passare alla successiva. Questo modello è particolarmente adatto per progetti con requisiti chiari e stabili fin dall'inizio. Tuttavia, può essere **inflexibile** in caso di cambiamenti dei requisiti, poiché tornare indietro in una fase precedente può richiedere notevole tempo e sforzo. Questo modello è spesso utilizzato in progetti di piccole dimensioni o in situazioni in cui i requisiti sono ben definiti e non soggetti a cambiamenti significativi.

Nel modello Waterfall, il processo di **V&V** avviene alla fine di ciascuna **fase**, prima di passare alla successiva. Lo scopo di questo processo è quello di individuare eventuali errori o problemi prima che diventino problemi più significativi nelle fasi successive dello sviluppo.

- La **verifica** è il processo di conferma che il software è stato sviluppato in conformità con i requisiti specificati e che è stato eseguito correttamente. Coinvolge il controllo e la revisione dei prodotti software per identificare errori, difetti o discrepanze rispetto ai requisiti stabiliti.
- La **validazione** è il processo di conferma che il software soddisfi effettivamente le esigenze dell'utente e sia in grado di funzionare in modo efficace nell'ambiente operativo previsto. Si concentra sulla comprensione delle esigenze dell'utente e sulla verifica che il software risponda a tali esigenze.

RAPID PROTOTYPING MODEL



Il modello Rapid Prototyping è un approccio di sviluppo del software che si concentra sulla creazione di **prototipi rapidi e iterativi** del software per ottenere un feedback tempestivo dal cliente o dagli utenti finali. Questo approccio è particolarmente utile quando i requisiti non sono ben definiti o possono cambiare durante lo sviluppo.

- **Definizione dei Requisiti Iniziali:**

- Inizia con la raccolta dei requisiti iniziali, ma questi possono essere meno dettagliati rispetto ad altri modelli poiché il focus è sulla creazione rapida di prototipi.

- **Creazione del Prototipo:**

- In questa fase, viene creato un prototipo rapido del software che include solo le funzionalità chiave o i requisiti principali.
- Il prototipo è una versione semplificata del software che può essere sviluppata rapidamente per ottenere un'idea iniziale del prodotto.

- **Valutazione e Feedback:**

- Il prototipo viene presentato al cliente o agli utenti finali per ottenere feedback. Questo è un passo critico in cui gli stakeholder* possono esprimere opinioni sulle funzionalità, l'usabilità e altre caratteristiche.

- **Raffinamento e Iterazione:**

- In base al feedback ricevuto, il prototipo viene raffinato e ulteriormente sviluppato. Nuove funzionalità possono essere aggiunte o modificate in base alle esigenze emerse.

- **Cicli di Prototipazione:**

- Questo processo di creazione, valutazione e raffinamento può essere iterato più volte fino a quando il prototipo si avvicina alle aspettative degli utenti.

- **Sviluppo Completo:**

- Dopo diverse iterazioni di prototipazione e feedback, il prototipo viene progressivamente sviluppato in un prodotto completo e pronto per il rilascio.

* Gli stakeholder sono individui, gruppi o entità che hanno un interesse o un coinvolgimento nel processo di sviluppo del software*

Il modello Rapid Prototyping è adatto quando i **requisiti** sono **incerti**, non ben definiti o possono cambiare durante lo sviluppo. Il processo è altamente **iterativo**, con cicli rapidi di creazione, valutazione e raffinamento del prototipo.

I prototipi non sono versioni finali del software, ma versioni **semplificate** e funzionali per ottenere un feedback tempestivo.

L'approccio enfatizza la **comunicazione** con il cliente o gli utenti finali e l'adattamento continuo alle loro esigenze e preferenze.

Vantaggi:

- **Rilevare Malintesi:** I prototipi aiutano a rilevare eventuali malintesi tra gli utenti e gli sviluppatori, in quanto mostrano in modo tangibile come il software risponderà alle esigenze degli utenti.
- **Individuare Servizi Mancanti o Confusi:** L'interazione con il prototipo può rivelare aree in cui i servizi offerti dal sistema non sono chiari o mancanti.
- **Sistema Funzionante Precoce:** I prototipi forniscono una versione funzionante del sistema in una fase precoce dello sviluppo, consentendo agli utenti di vedere e interagire con il software in anticipo.
- **Base per la Specifica Software:** Il prototipo può servire come base per derivare una specifica software più dettagliata, poiché offre un'idea concreta di come il software dovrebbe funzionare.
- **Supporto alla Formazione e ai Test:** Il prototipo può essere utilizzato per formare gli utenti sul funzionamento del software e per condurre test preliminari.

THROW-AWAY PROTOTYPING

La prototipizzazione "throw-away" è un approccio in cui viene creato un prototipo pratico del prodotto con l'obiettivo principale di **identificare problemi** nei requisiti. Alcuni punti importanti su questo approccio:

- **Scarto del Prototipo:** Una volta identificati i problemi e ottenute le informazioni necessarie, il prototipo viene scartato. Non viene utilizzato come base per lo sviluppo del prodotto finale.
- **Minimizzare i Rischi:** Questo approccio è utilizzato per ridurre i rischi associati ai requisiti errati o poco chiari. Il prototipo aiuta a individuare questi problemi in modo tempestivo.
- **Processo di Sviluppo Diverso:** Dopo aver creato il prototipo e compreso meglio i requisiti, il processo di sviluppo del prodotto finale può utilizzare un approccio diverso.

- **Focus sui Requisiti Iniziali:** Il prototipo "throw-away" si basa sui requisiti iniziali, il che significa che potrebbero mancare alcune caratteristiche o aspetti più complessi del sistema.
- **Limitazioni del Prototipo:** Non è da considerare un prodotto finale, poiché potrebbe mancare la piena funzionalità e non è stato progettato per la manutenzione a lungo termine.

Consegna del Prototipo "Throw-away":

Quando si parla di consegnare un prototipo "throw-away" come **prodotto finale**, si tratta di un'idea che può presentare delle sfide e rischi significativi. Ecco perché questa pratica non è raccomandata:

- **Impossibilità di Soddisfare Requisiti Non Funzionali:** I prototipi potrebbero non essere progettati per soddisfare requisiti non funzionali cruciali, come prestazioni, scalabilità o sicurezza. Pertanto, consegnare un prototipo "throw-away" come prodotto finale potrebbe non essere in grado di soddisfare questi aspetti critici.
- **Mancanza di Documentazione:** I prototipi "throw-away" sono spesso creati con lo scopo di esplorare idee e requisiti, quindi potrebbero mancare documentazione e dettagli essenziali che sono necessari per la manutenzione futura e la comprensione del funzionamento del sistema.
- **Degrado della Struttura:** Durante lo sviluppo di un prototipo, potrebbero essere apportate modifiche rapide per esplorare diverse possibilità. Queste modifiche potrebbero non rispettare le migliori pratiche di strutturazione del codice e potrebbero influire negativamente sulla qualità complessiva del software.
- **Norme di Qualità Mancanti:** Poiché un prototipo "throw-away" non è destinato a essere un prodotto finale, potrebbero non essere applicate le normali norme di qualità organizzative e di sviluppo. Questo potrebbe portare a un software di bassa qualità.

Punti Chiave sul Prototipaggio:

- **Dare un'Impressione Concreta:** L'utilizzo dei prototipi dà agli utenti finali un'idea tangibile delle capacità e delle funzionalità del prodotto, consentendo loro di fornire feedback più dettagliati.
- **Utilizzo Crescente del Prototipaggio:** Nell'attuale ambiente di sviluppo, in cui la velocità è spesso cruciale, il prototipaggio è sempre più utilizzato poiché consente uno sviluppo più rapido e iterativo.
- **Prototipaggio "Throw-away" per Comprendere i Requisiti:** Il prototipaggio "throw-away" è particolarmente utile per esplorare e comprendere meglio i requisiti del prodotto, identificando lacune e problematiche fin dalle fasi iniziali.
- **Sviluppo Rapido:** Lo sviluppo rapido dei prototipi è fondamentale, ma potrebbe richiedere alcune scelte come l'omissione di funzionalità non essenziali o il rilassamento temporaneo di vincoli non funzionali.
- **Programmazione Visuale:** La programmazione visuale è spesso utilizzata nei metodi di sviluppo dei prototipi perché consente di rappresentare in modo intuitivo le interazioni e le funzionalità del sistema.

In sintesi, i prototipi "throw-away" hanno un ruolo importante nell'esplorare e comprendere i requisiti, ma non sono adatti come prodotti finali a causa di problemi legati alla qualità, mancanza di documentazione e impossibilità di soddisfare requisiti non funzionali critici. Il prototipaggio in generale rimane uno strumento prezioso per lo sviluppo software rapido e iterativo, che aiuta a coinvolgere gli utenti e a comprendere meglio le esigenze del prodotto finale.

VISUAL PROGRAMMING

La programmazione visuale rappresenta un approccio allo sviluppo software che si basa sull'uso di **elementi grafici e interfacce utente** per creare il codice.

- **Sviluppo di Interfacce Utente:** Nel contesto della programmazione visuale, il prototipo viene creato mediante la progettazione di un'interfaccia utente utilizzando elementi grafici standard. I componenti vengono associati a questi elementi per definire il comportamento del prototipo.
- **Librerie di Componenti:** Esistono librerie di componenti che supportano il processo di sviluppo visuale. Questi componenti possono essere adattati alle specifiche esigenze dell'applicazione in fase di sviluppo.

Problemi con lo Sviluppo Visuale:

- **Difficoltà nel Coordinamento:** L'approccio visuale potrebbe comportare difficoltà nel coordinare lo sviluppo quando più membri del team lavorano su diverse parti dell'applicazione.
- **Assenza di un'Architettura Esplicita:** La programmazione visuale potrebbe mancare di un'architettura software esplicita, il che potrebbe complicare la gestione e la manutenzione a lungo termine del progetto.
- **Dipendenze Complesse:** Le dipendenze complesse tra le varie parti del programma possono portare a problemi di manutenzione e rendere difficile l'apportare modifiche senza causare effetti collaterali indesiderati.

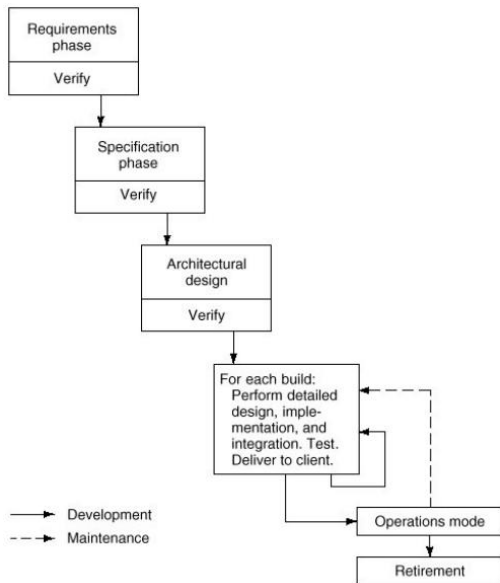
ITERAZIONE DEL PROCESSO

È comune che i requisiti di un progetto evolvano nel corso del tempo. L'iterazione è il processo di ritornare a fasi precedenti del ciclo di sviluppo per adattarsi a questi cambiamenti. L'iterazione può essere applicata a vari modelli di processo ed esistono due approcci: **modello incrementale** e **modello a spirale**.

MODELLO INCREMENTALE

Il modello incrementale è un approccio allo sviluppo software in cui il prodotto viene sviluppato e rilasciato per **incrementi successivi**. Ogni incremento aggiunge nuove funzionalità al prodotto. Questo modello include aspetti tipici del modello basato su **rapid prototyping** (l'utente può sperimentare l'utilizzo del prodotto contenente gli incrementi consegnati, mentre i restanti sono ancora in fase di sviluppo).

Può essere implementato in due versioni: con o senza **overall architecture** (architettura generale).

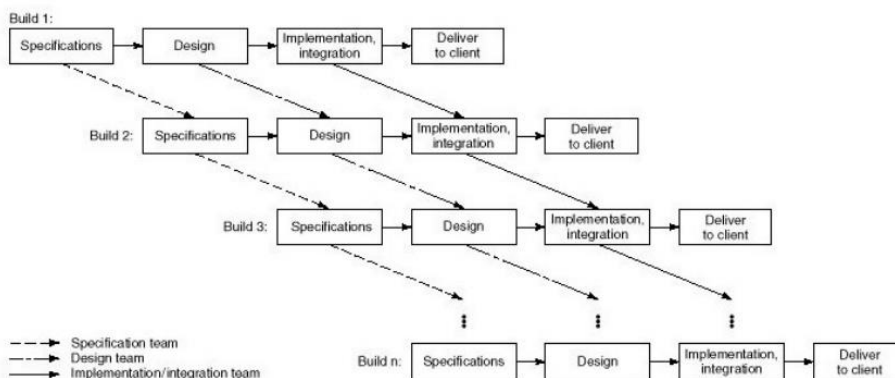


La **versione con overall architecture** si riferisce a un approccio nel modello incrementale in cui viene stabilita una struttura di architettura generale prima dell'inizio dello sviluppo incrementale. In altre parole, prima di iniziare a creare i singoli incrementi, si definisce un quadro architeturale che fornisce una visione globale di come il prodotto finale sarà strutturato.

- **Definizione dell'Architettura:** Prima di iniziare a sviluppare i singoli incrementi, viene definita un'architettura generale che stabilisce come le diverse parti del sistema interagiranno tra loro.
- **Guida per lo Sviluppo:** L'architettura generale funge da guida durante il processo di sviluppo. Gli sviluppatori hanno una visione chiara di come le parti del sistema si collegheranno e lavoreranno insieme, consentendo loro di sviluppare gli incrementi in modo coerente.
- **Minimizzazione dei Problemi di Integrazione:** L'approccio con overall architecture mira a minimizzare i problemi di integrazione che potrebbero emergere quando gli incrementi vengono combinati. Poiché l'architettura è già stata definita, è più probabile che i singoli incrementi si adattino in modo coeso all'interno della struttura complessiva.
- **Rischio e Complessità:** Sebbene l'approccio con overall architecture possa

ridurre il rischio di problemi di integrazione e aiutare a stabilire una base solida, richiede una pianificazione e un'analisi più approfondite iniziali per stabilire l'architettura generale. Questo può rendere l'avvio del progetto più lungo rispetto a un approccio meno pianificato.

- **Adattabilità:** Nonostante l'architettura globale venga definita all'inizio, è importante considerare che il modello incrementale prevede ancora flessibilità e adattabilità. Gli incrementi possono essere soggetti a modifiche in base al feedback degli utenti e all'evoluzione dei requisiti.



La **versione senza overall architecture** nel contesto del modello incrementale si riferisce a un approccio in cui non viene definita un'architettura generale completa prima di iniziare lo sviluppo incrementale.

- **Inizio Diretto con Incrementi:** In questo approccio, il processo di sviluppo inizia direttamente con la creazione dei singoli incrementi, senza una fase preliminare dedicata alla definizione di un'architettura generale.

- **Flessibilità e Adattabilità:** Questo approccio privilegia la flessibilità e la

capacità di adattarsi rapidamente ai cambiamenti. Gli sviluppatori sono in grado di concentrarsi su funzionalità specifiche e adattarsi alle esigenze in evoluzione dei requisiti e del feedback degli utenti.

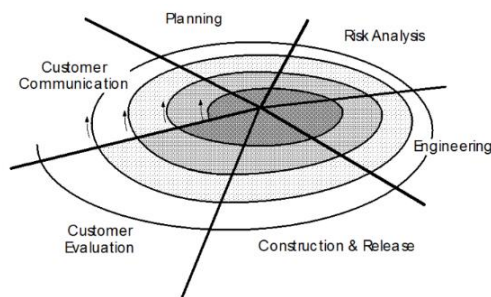
- **Rischio di Integrazione:** Poiché manca un quadro architeturale generale, potrebbero sorgere problemi di integrazione quando gli incrementi vengono combinati. La mancanza di una guida architeturale potrebbe portare a una struttura complessiva meno coerente.
- **Risultati Iniziali più Veloci:** Poiché l'attenzione si concentra direttamente sugli incrementi e sulle funzionalità specifiche, è possibile ottenere risultati iniziali più rapidi rispetto a un approccio con overall architecture.
- **Complessità Futura:** Mentre la mancanza di un'architettura globale potrebbe semplificare l'inizio dello sviluppo, potrebbe anche portare a una maggiore complessità in seguito quando gli incrementi dovessero essere integrati.

Confronto con il Modello a Cascata:

MODELLO A CASCATA	MODELLO INCREMENTALE
Il feedback del cliente è ottenuto solo alla fine del processo di sviluppo.	Il feedback del cliente è continuo durante tutto il processo di sviluppo grazie ai rilasci incrementali.
Le fasi procedono in una sequenza rigida (l'output di una costituisce input per la successiva).	Le fasi possono essere condotte in parallelo .
Le fasi di progetto dettagliato e codifica sono effettuate per l'intero prodotto .	La progettazione dettagliata e la codifica vengono effettuate su ogni singolo incremento .
Il team di sviluppo può essere composto da un numero elevato di persone.	Il lavoro è svolto da differenti team di sviluppo, ciascuno di piccole dimensioni.
I requisiti sono generalmente fissati dopo la fase di specifica.	I requisiti sono suddivisi in classi di priorità e possono essere facilmente modificati in base alle iterazioni.

In sintesi, il modello incrementale si basa su rilasci incrementali e coinvolgimento continuo dell'utente, il che lo rende adatto a progetti con requisiti in evoluzione e la necessità di un feedback costante. Rispetto al modello a cascata, il modello incrementale offre una maggiore adattabilità ai cambiamenti e una migliore gestione delle esigenze in evoluzione.

MODELLO A SPIRALE



Il modello a spirale è un approccio di sviluppo software che incorpora **iterazione** e **valutazione** continua per affrontare i rischi e i problemi nel corso del progetto. Viene rappresentato come una serie di fasi a spirale, ciascuna delle quali rappresenta un ciclo di sviluppo. Queste fasi sono:

- **Determinazione degli Obiettivi:** Definizione degli obiettivi del progetto, identificazione dei vincoli e delle restrizioni.
- **Identificazione e Risoluzione dei Rischi:** Identificazione dei rischi potenziali associati agli obiettivi del progetto. Si cercano modi per affrontarli e risolverli.
- **Sviluppo e Test:** Sviluppo del software e creazione di prototipi, seguiti da

test per valutare la funzionalità e la robustezza.

- **Valutazione del Cliente:** Valutazione del cliente dei risultati dell'iterazione corrente, inclusi i prototipi e i progressi.
- **Pianificazione:** Basandosi sulla valutazione del cliente, si decide se procedere con la prossima iterazione, pianificando ulteriori sviluppi e risoluzione dei rischi.

Il modello a spirale è adatto per progetti complessi in cui i rischi possono essere difficili da identificare fin dall'inizio. Ogni ciclo rappresenta un passaggio attraverso queste fasi, con l'obiettivo di migliorare il prodotto e affrontare i rischi prima che diventino problematici.

Modello a Spirale Semplificato (Versione Lineare):

In questa variante semplificata, il modello a spirale è semplificato a una sequenza lineare di attività. Manca la natura ciclica dei cicli a spirale. Le fasi sono simili a quelle del modello a spirale completo ma vengono eseguite in sequenza.

Modello a Spirale Semplificato:

Questa variante conserva le fasi a spirale ma riduce il numero di cicli. Invece di eseguire molteplici cicli a spirale, vengono eseguite solo poche iterazioni per coprire le diverse fasi. Questo riduce la complessità e il numero complessivo di cicli.

Modello Full-Spiral:

Il modello full-spiral è il modello a spirale completo, come descritto in precedenza. Prevede iterazioni multiple attraverso le diverse fasi in un approccio continuo e iterativo.

GESTIONE DEI RISCHI

La gestione dei rischi è un processo essenziale nello sviluppo software che mira a identificare, valutare e affrontare i rischi che possono influenzare un progetto. Un rischio è una possibile circostanza sfavorevole o evento che potrebbe influenzare il successo del progetto.

Categorie di Rischi:

- **Project Risks:** Questi riguardano le variabili interne al progetto stesso, come la gestione del personale, le modifiche nell'organizzazione o la disponibilità dei mezzi necessari.
 - Ad esempio, la perdita di personale esperto, il cambiamento della gestione dell'organizzazione o la disponibilità ritardata dell'hardware essenziale.

- **Product Risks:** Questi riguardano gli aspetti del prodotto in sviluppo, come cambiamenti nelle specifiche, ritardi nel completamento o problemi di prestazione con gli strumenti utilizzati per lo sviluppo.
 - Ad esempio, cambiamenti e ritardi nelle specifiche, sottostima delle dimensioni del sistema o prestazioni non soddisfacenti degli strumenti CASE (Computer-Aided Software Engineering).
- **Business Risks:** Questi sono legati al contesto commerciale in cui l'organizzazione opera, come cambiamenti tecnologici che potrebbero renderlo obsoleto o la competizione di prodotti simili sul mercato.
 - Ad esempio, il cambiamento della tecnologia sottostante o la competizione di prodotti concorrenti.

Processo di Gestione dei Rischi:

- **Identificazione dei Rischi:** In questa fase, si individuano e si elencano i potenziali rischi che potrebbero influire sul progetto, il prodotto o l'organizzazione. Questa fase richiede una visione approfondita del progetto e del suo ambiente.
- **Analisi dei Rischi:** Si valuta la probabilità che ciascun rischio si verifichi e le conseguenze che avrebbero se si verificassero. Questo aiuta a classificare i rischi in base alla loro gravità e impatto.
- **Pianificazione dei Rischi:** Per ciascun rischio identificato, si sviluppano piani per affrontarlo. Questi piani possono includere azioni preventive per evitare che il rischio si verifichi o contromisure per ridurre l'impatto se si verifica.
- **Monitoraggio dei Rischi:** Questa è una fase continua in cui i rischi vengono monitorati durante tutto il corso del progetto. Se nuove informazioni emergono o se cambiano le circostanze, i piani possono essere aggiornati o modificati di conseguenza.

Identificazione dei Rischi:

I rischi possono essere categorizzati in diverse categorie, ognuna delle quali rappresenta un'area di potenziale incertezza o problematica all'interno di un progetto. Le categorie includono **tecnologici**, **umani**, **organizzativi**, legati agli **strumenti**, ai **requisiti** e alle **stime**. Ogni categoria può avere rischi specifici associati ad essa.

Analisi dei Rischi:

L'analisi dei rischi coinvolge la valutazione della **probabilità** che un rischio si verifichi e degli effetti che avrebbe se si verificasse. La probabilità può essere classificata in categorie come **molto bassa (<10%)**, **bassa (10-25%)**, **moderata (25-50%)**, **alta (50-75%)** o **molto alta (>75%)**. Gli effetti possono essere categorizzati come **catastrofici**, **gravi**, **tollerabili** o **insignificanti**.

Ad esempio, il rischio di tagli al budget a causa di problemi finanziari dell'organizzazione è valutato come basso in probabilità ma con effetti catastrofici. Questo esempio dimostra come un rischio può essere valutato in base alla sua probabilità di verificarsi e all'entità dei danni che potrebbe causare.

Selezione dei Rischi Più Importanti:

Per individuare i rischi più importanti, è possibile considerare i rischi **catastrofici** e i rischi **gravi** con una probabilità superiore alla **moderata**. Questi rischi selezionati vengono poi **classificati** in ordine di importanza, consentendo al team di concentrare le risorse e l'attenzione sui rischi che potrebbero avere il maggiore impatto sul progetto.

Pianificazione dei Rischi:

La pianificazione dei rischi è il processo di sviluppare strategie per affrontare i rischi identificati durante il progetto. Queste strategie sono finalizzate a ridurre la probabilità di occorrenza dei rischi, minimizzarne l'impatto o affrontarli in modo efficace in caso di occorrenza.

Ecco alcune strategie di gestione dei rischi in relazione a diversi tipi di rischi:

- **Problemi Finanziari dell'Organizzazione:** Preparare un documento che dimostri come il progetto contribuisce agli obiettivi aziendali, al fine di sensibilizzare la direzione aziendale sull'importanza del progetto.
- **Problemi di Reclutamento:** Comunicare al cliente le possibili difficoltà di reclutamento e valutare l'opzione di acquistare componenti esterni per evitare ritardi.
- **Malattia del Personale:** Riorganizzare il team in modo da condividere il lavoro in modo più ampio e migliorare la comprensione dei compiti reciproci per affrontare l'assenza di personale.
- **Componenti Difettosi:** Sostituire i componenti potenzialmente difettosi con componenti affidabili ottenuti da fonti affidabili.
- **Cambiamenti nei Requisiti:** Creare un sistema di tracciabilità per valutare l'impatto dei cambiamenti dei requisiti e massimizzare l'isolamento delle informazioni durante la fase di progettazione.
- **Ristrutturazione Organizzativa:** Simile all'esempio precedente, preparare un documento per dimostrare come il progetto contribuisce agli obiettivi aziendali e sensibilizzare la direzione sull'importanza del progetto.
- **Prestazioni del Database:** Esaminare l'opzione di acquistare un database con prestazioni migliori per affrontare i problemi di prestazioni del database.
- **Sottostima del Tempo di Sviluppo:** Valutare l'acquisto di componenti esterni o l'uso di un generatore di programmi per accelerare lo sviluppo e affrontare la sottostima del tempo.

Monitoraggio dei Rischi:

Il monitoraggio dei rischi è un processo cruciale per garantire che i rischi identificati vengano valutati regolarmente nel corso del progetto. Coinvolge l'analisi costante di ciascun rischio per determinare se la sua probabilità di occorrenza sta diminuendo o aumentando. Questo processo aiuta a mantenere un controllo costante sui rischi e ad affrontarli in modo tempestivo.

MODELLO OBJECT-ORIENTED

Questo modello è incentrato sull'approccio di sviluppo **orientato agli oggetti**, in cui il software è progettato e strutturato intorno a oggetti, classi e relazioni. Questo approccio consente di creare software modulare, riutilizzabile e flessibile. Le fasi di sviluppo si concentrano sulla creazione e interazione degli oggetti, promuovendo la modularità e la gestione semplificata dei cambiamenti nei requisiti.

MODELLO DI INGEGNERIA SIMULTANEA O CONCORRENTE

Questo modello mira a ridurre tempi e costi di sviluppo attraverso un approccio sistematico e concorrente al progetto di un prodotto software e dei processi associati. A differenza dei modelli sequenziali, le **fasi di sviluppo** non si susseguono in ordine sequenziale, ma **coesistono** contemporaneamente. Questo favorisce la collaborazione tra le diverse fasi e consente di ottenere risultati più rapidamente.

MODELLO BASATO SU METODI FORMALI

Questo modello coinvolge l'applicazione di metodi formali per specificare il software in modo **matematicamente rigoroso**. Lo scopo principale è eliminare ambiguità, incompletezze ed inconsistenze nelle specifiche e nel codice, facilitando così la verifica e la validazione dei programmi attraverso tecniche matematiche. Un esempio di questa filosofia è la **Cleanroom Software Engineering**, sviluppata nel 1987, che si concentra sulla rilevazione tempestiva dei difetti tramite rigorosi processi formali.

MODELLO MICROSOFT – APPROCCIO SYNCH-AND-STABILIZE

La Microsoft, come molte altre organizzazioni nel settore del software, ha affrontato la necessità di migliorare la **qualità** dei prodotti software e di **ridurre tempi e costi** di sviluppo fin dagli anni '80. Per far fronte a queste sfide, ha sviluppato un modello di sviluppo che combina elementi **iterativi**, **incrementali** e **concorrenti**, consentendo di sfruttare la creatività delle persone coinvolte nello sviluppo.

L'approccio utilizzato dalla Microsoft, noto come "**synchronize-and-stabilize**", si basa su due principi fondamentali:

- **Sincronizzazione Quotidiana:** Le attività svolte da singole persone e piccoli team (da 3 a 8 persone) vengono sincronizzate quotidianamente. Questo avviene attraverso l'assemblaggio dei componenti software sviluppati (anche in modo parziale) in un "daily build". Questo build giornaliero rappresenta il prodotto in uno stato iniziale che viene sottoposto a test e correzioni per individuare e risolvere tempestivamente eventuali problemi.
- **Stabilizzazione Periodica:** Il prodotto viene stabilizzato in incrementi successivi, noti come "milestone", lungo tutto l'arco del progetto. Questo significa che il processo di stabilizzazione non avviene solo alla fine del progetto, ma periodicamente durante il suo avanzamento. Questo approccio mira a gestire in modo più efficace gli aspetti di qualità, rischi e cambiamenti nel corso del progetto.

CICLO DI SVILUPPO A 3 FASI

Il ciclo di sviluppo a 3 fasi è un approccio strutturato alla gestione del **processo di sviluppo** del software. Questo modello si basa su tre fasi chiave:

- **Fase di Pianificazione:**
 - In questa fase iniziale, viene definita la visione del prodotto, compresi gli obiettivi, le specifiche e la pianificazione.
 - Gli obiettivi includono la comprensione chiara di ciò che il prodotto software deve realizzare.
 - Le specifiche definiscono i dettagli funzionali e tecnici del prodotto.
 - La pianificazione comprende la stima dei tempi, la suddivisione delle attività e la creazione di un piano di sviluppo.
- **Fase di Sviluppo:**
 - La fase di sviluppo è suddivisa in 3 o 4 sottoprogetti sequenziali.
 - Ogni sottoprogetto è orientato alla realizzazione di una parte specifica del prodotto e culmina in un rilascio di milestone.
 - Questa fase è incentrata sulla progettazione, lo sviluppo e l'integrazione delle funzionalità del prodotto.
 - Ogni sottoprogetto viene completato con il rilascio di una versione funzionante del software.
- **Fase di Stabilizzazione:**
 - La fase di stabilizzazione segue la fase di sviluppo e si concentra sulla verifica e sulla validazione del prodotto.
 - Vengono eseguiti test interni ed esterni approfonditi per identificare difetti e assicurare che il software funzioni come previsto.

- Una volta completati i test e risolti i problemi, il prodotto è portato a uno stato stabile e pronto per il rilascio.
- Alla fine di questa fase, il software è pronto per essere distribuito agli utenti finali.

STRATEGIE E PRINCIPI – MODELLO MICROSOFT

Strategia per Definire Prodotto e Processo: Questa strategia pone l'accento sulla creatività come elemento essenziale nello sviluppo del software.

- Dividere il progetto in milestone (fasi chiave).
- Definire una "product vision" e una specifica funzionale in evoluzione.
- Selezionare funzionalità e priorità in base alle esigenze dell'utente.
- Definire un'architettura modulare per riflettere la struttura del prodotto.
- Assegnare task elementari e limitare le risorse.

Strategia per Sviluppo e Consegna dei Prodotti: Questa strategia mira a lavorare in modo parallelo con frequenti sincronizzazioni.

- Definire team paralleli e utilizzare il "daily build" per la sincronizzazione.
- Avere sempre un prodotto pronto per la consegna, con versioni per diverse piattaforme e mercati.
- Utilizzare lo stesso linguaggio di programmazione all'interno dello stesso sito di sviluppo.
- Testare continuamente il prodotto durante lo sviluppo.
- Utilizzare metriche per supportare le decisioni.

CONFRONTO TRA SYNC-AND-STABILIZE E MODELLO A CASCATA

SYNC-AND-STABILIZE	MODELLO A CASCATA
È iterativo e incrementale .	È sequenziale .
Coinvolge milestone periodiche di stabilizzazione.	Le fasi si susseguono in un flusso lineare .
Lavora in modo parallelo e sincronizzato .	Richiede il completamento di una fase prima di passare alla successiva.
Promuove la creatività e l'evoluzione delle specifiche .	Si basa su specifiche complete e fisse all'inizio.
Abbraccia il concetto di " daily build " e test continui.	Può essere meno adattabile ai cambiamenti dei requisiti.
Adatto a progetti che richiedono flessibilità e adattamento .	Adatto a progetti con requisiti chiari e stabili .

MODELLO NETSCAPE

Il processo di sviluppo software adottato dalla Netscape Communications Corporation, noto come il Modello Netscape segue un approccio di tipo "**synchronize-and-stabilize**" (sincronizzazione e stabilizzazione) e si adatta allo sviluppo di **applicazioni Internet** come browser e prodotti server. Questo significa che il processo di sviluppo è suddiviso in due fasi principali:

- **Fase di Sincronizzazione (Synchronize):** Durante questa fase, l'attenzione è rivolta a sviluppare e introdurre nuove funzionalità. Gli sviluppatori lavorano per creare e implementare nuove idee e caratteristiche nel software.
- **Fase di Stabilizzazione (Stabilize):** Dopo aver aggiunto le nuove funzionalità, la fase di stabilizzazione si concentra sulla risoluzione dei bug, sull'ottimizzazione delle prestazioni e sulla migliorare la qualità generale del software. In questa fase, il focus si sposta dalla creazione di nuove funzionalità alla correzione degli errori e all'assicurazione che il software sia affidabile e stabile.

METODI AGILI

Agile è un approccio allo sviluppo software che si è evoluto nei primi anni **2000** come reazione ai processi di sviluppo software rigidamente pianificati. Gli sviluppatori agili sostengono che questi processi possono essere troppo **restrittivi** e **incapaci** di adattarsi rapidamente ai cambiamenti. L'approccio Agile è incentrato sulla **flessibilità**, la **collaborazione** e la **consegna rapida** di valore. Il **Manifesto Agile**, formulato nel **2001**, definisce quattro valori fondamentali che gli sviluppatori agili ritengono di grande importanza:

- **Individui e interazioni più che processi e strumenti:** Gli aspetti umani e la comunicazione efficace sono prioritari rispetto a procedure rigide e strumenti tecnologici.
- **Software funzionante più che documentazione esaustiva:** L'obiettivo principale è consegnare software che funzioni correttamente, anziché dedicarsi a una documentazione eccessivamente dettagliata.
- **Collaborazione con il cliente più che negoziazione di contratti:** La collaborazione continua con il cliente è essenziale per comprendere le sue esigenze e soddisfarle, piuttosto che concentrarsi solo su contratti formali.
- **Rispondere ai cambiamenti più che seguire un piano:** L'Agile si basa sulla capacità di adattarsi rapidamente ai cambiamenti nelle esigenze o nelle circostanze, invece di attenersi rigidamente a piani prefissati.

Oltre ai valori, il Manifesto Agile include **dodici principi** che guidano l'implementazione degli approcci agili. Questi principi enfatizzano l'importanza della consegna continua di software funzionante, della collaborazione attiva con i clienti, dell'accoglienza dei cambiamenti nei requisiti e molto altro.

SCRUM – METODO AGILE

Scrum è uno dei framework* **agili** più popolari che aiuta i team a lavorare insieme in modo più efficiente e flessibile per produrre prodotti di alta qualità. Il suo **obiettivo** principale è quello di consegnare valore* in modo iterativo e incrementale, consentendo ai team di adattarsi rapidamente ai cambiamenti nelle esigenze dei clienti e nel contesto del progetto.

Ruoli:

- **Product Owner:** La persona responsabile di definire le esigenze del prodotto e di mantenere il Product Backlog aggiornato con le funzionalità e le attività da realizzare.
- **Scrum Master:** Il facilitatore del team, responsabile di aiutare il team a seguire Scrum e rimuovere gli ostacoli che potrebbero impedire il progresso.
- **Team di sviluppo:** Il gruppo di persone che effettivamente realizzano il lavoro, sviluppando le funzionalità e completando le attività.

Eventi:

- **Sprint:** Un periodo di tempo fisso (solitamente da una a quattro settimane) in cui il team lavora per consegnare un insieme di funzionalità.
- **Sprint Planning:** Una riunione all'inizio di ogni sprint in cui il team seleziona le attività dal Product Backlog da completare durante lo sprint.
- **Daily Scrum:** Una riunione quotidiana in cui il team condivide rapidamente cosa ha fatto, cosa farà e se ci sono ostacoli.
- **Sprint Review:** Una riunione alla fine di ogni sprint in cui il team mostra ciò che ha completato al Product Owner e agli stakeholder.
- **Sprint Retrospective:** Una riunione dopo la Sprint Review in cui il team riflette su ciò che è andato bene e su come migliorare.

Artefatti:

- **Product Backlog:** È una "lista di cose da fare" per il prodotto. Contiene tutte le funzionalità, i miglioramenti e le attività da realizzare in futuro, in ordine di priorità.
- **Sprint Backlog:** Una lista di attività selezionate dal Product Backlog per essere completate durante lo sprint corrente.
- **Incremento:** Il risultato del lavoro dello sprint, che include tutte le funzionalità e le attività completate.

L'approccio Scrum prevede cicli ripetitivi chiamati "**sprint**", in cui il team pianifica, sviluppa, testa e consegna un incremento di prodotto. Questo approccio consente al team di adattarsi rapidamente ai cambiamenti, ricevere feedback regolare e consegnare valore in modo continuo.

Un framework è un'infrastruttura di supporto predefinita e strutturata che fornisce un insieme di strumenti, librerie, linee guida e componenti riutilizzabili per semplificare lo sviluppo di software, applicazioni o altre soluzioni. In altre parole, è come un set di strumenti e istruzioni già pronte che ti aiutano a costruire qualcosa.

Quando si parla di "consegnare valore" in un contesto come lo sviluppo software o il project management, si intende sviluppare e rilasciare funzionalità o componenti che apportino un impatto positivo al destinatario finale.

Sprint:

Uno "sprint" è un periodo di tempo definito (tipicamente da 2 a 4 settimane) durante il quale il team di sviluppo lavora per consegnare un **incremento** funzionante del software. Gli sprint iniziano con una riunione di pianificazione durante la quale il team seleziona gli elementi da sviluppare dal Product Backlog e li sposta nello Sprint Backlog. Durante lo sprint, si tengono brevi riunioni quotidiane chiamate "stand-up meeting" per sincronizzare il lavoro e risolvere eventuali problemi. Alla fine di uno sprint, viene effettuata una revisione in cui l'incremento viene mostrato al Product Owner e agli stakeholder. Infine, si tiene una riunione di retrospettiva per riflettere sullo sprint e identificare possibili miglioramenti per il prossimo.

Definizione di "Done" (Fatto):

Nel contesto di Scrum e dell'approccio Agile, la "definizione di done" è un concetto fondamentale. Rappresenta un insieme di criteri che il team di sviluppo stabilisce per definire quando un elemento di lavoro è considerato **completato** e **pronto** per l'integrazione nel ramo principale del progetto. Questo aiuta a garantire che il lavoro consegnato sia di alta qualità e soddisfi gli standard stabiliti.

User Stories (Storie Utente):

Le "storie utente" sono un metodo comune utilizzato nello sviluppo **Agile**, spesso in combinazione con Scrum, per catturare ed esprimere i **requisiti** di un progetto in un formato semplice e comprensibile. Una "**storia utente**" è una breve descrizione di una funzionalità o un requisito del sistema dal punto di vista dell'utente finale. La struttura tipica di una storia utente è la seguente:

As a [ruolo dell'utente], I want [obiettivo dell'utente] so that [beneficio ottenuto]

Le storie utente sono brevi, concise e mirano a focalizzare l'attenzione sui bisogni degli utenti finali. Sono uno strumento chiave per comprendere e soddisfare i requisiti degli utenti in modo iterativo ed efficiente.

Le "storie utente" possono essere chiamate "**epiche**" quando rappresentano requisiti **complessi** che richiedono una suddivisione ulteriore in storie più piccole.

CAPABILITY MATURITY MODEL – CMM

Il Capability Maturity Model (CMM) è un framework di valutazione e miglioramento dei processi sviluppato dal Software Engineering Institute (SEI) negli anni '90. È stato progettato per **valutare** e **migliorare** la maturità dei processi di sviluppo software all'interno delle organizzazioni. Il CMM fornisce una scala a **cinque livelli di maturità**, ognuno dei quali rappresenta un grado crescente di disciplina e controllo nei processi di sviluppo.

1. **Initial (Iniziale):** I processi sono imprevedibili, non documentati e variano notevolmente. Il successo è spesso basato sull'eroismo individuale.
2. **Repeatable (Ripetibile):** I processi sono documentati in modo da poter essere ripetuti. Tuttavia, la gestione di progetti potrebbe essere ancora reattiva.
3. **Defined (Definito):** I processi sono definiti e standardizzati in tutta l'organizzazione. L'organizzazione compie sforzi per istituire pratiche coerenti e formali.
4. **Managed (Gestito):** I processi sono misurati e controllati. L'organizzazione usa metriche per il monitoraggio e il miglioramento continuo dei processi.
5. **Optimizing (Ottimizzato):** I processi sono soggetti a miglioramenti continui e innovazioni. L'organizzazione cerca costantemente modi per ottimizzare i processi esistenti e adattarsi ai cambiamenti.

Key Process Areas (KPA):

Ogni livello di maturità nel CMM è associato a diverse Key Process Areas (KPA), che rappresentano gli aspetti chiave che un'organizzazione deve affrontare per raggiungere quel livello di maturità. Ogni KPA viene descritta attraverso **obiettivi**, **responsabilità**, **risorse necessarie**, **attività da svolgere** e **metodi per monitorare e verificare l'implementazione**.

REQUISITI SOFTWARE

I requisiti software rappresentano le descrizioni dei **servizi** che un sistema software deve fornire, insieme ai **vincoli** da rispettare durante il processo di sviluppo e nell'operatività del software stesso. Sono essenziali per comprendere le **necessità** degli **utenti**, guidare lo **sviluppo** e garantire che il sistema soddisfi le **aspettative** e i **requisiti** del cliente.

Secondo la **definizione dello standard IEEE Std 610.12 del 1990**, i requisiti software possono essere visti come condizioni o capacità necessarie per risolvere un problema o raggiungere un obiettivo da parte dell'utente o come condizioni o capacità che devono essere soddisfatte o possedute da un sistema o componente di sistema per adempiere a un contratto, standard, specifica o altro documento formalmente imposto.

I requisiti vengono generati attraverso il processo di **ingegneria dei requisiti**, che coinvolge la comprensione delle esigenze degli utenti, la loro traduzione in requisiti specifici e la documentazione di tali requisiti in modo accurato e completo.

Tipi di Requisiti:

- **Requisiti Utente (User Requirements):** Questi requisiti sono descritti in linguaggio naturale e possono essere arricchiti con l'uso di diagrammi. Definiscono i servizi che il sistema deve fornire e i vincoli operativi. Sono scritti in collaborazione con il cliente e rappresentano la base per capire le aspettative dell'utente finale.
- **Requisiti di Sistema (System Requirements):** Questi requisiti sono specificati in dettaglio attraverso un documento strutturato. Questo documento descrive in modo approfondito i servizi che il sistema software deve fornire. Il documento dei requisiti di sistema diventa una sorta di "contratto" tra il cliente e il fornitore, definendo in modo chiaro cosa il sistema dovrà realizzare.

Definizioni:

- **Cliente (Customer, Client):** È la persona o l'organizzazione che paga per la fornitura del prodotto software. Il cliente stabilisce le esigenze e i requisiti del software.
- **Fornitore (Supplier, Contractor):** È la persona o l'organizzazione che sviluppa il software per il cliente. Deve realizzare il software in linea con i requisiti stabiliti.
- **Utente Finale (End-User):** È la persona che interagisce direttamente con il prodotto software. Non è necessariamente il cliente, ma è colui che userà effettivamente il sistema.

Categorie di Requisiti:

- **Requisiti Funzionali:** Questi requisiti descrivono le funzionalità specifiche che il sistema software deve fornire. Si concentrano su cosa il sistema deve fare e come deve rispondere a determinati input o scenari.
 - Ad esempio, la fornitura di un visualizzatore per i documenti, la possibilità di effettuare ricerche e l'associazione di un identificatore unico a ciascun ordine.
- **Requisiti Non Funzionali:** Questi requisiti descrivono le proprietà del sistema in relazione alle prestazioni, all'affidabilità, alla sicurezza e ad altre caratteristiche che non riguardano direttamente le funzionalità. Essi possono anche riguardare il processo di sviluppo, gli standard da seguire e le normative da rispettare.
 - Ad esempio, tempi di risposta specifici, conformità agli standard di progetto e restrizioni sulla divulgazione di informazioni personali.
- **Requisiti di Dominio:** Questi requisiti derivano dal contesto applicativo specifico in cui il sistema verrà utilizzato. Possono includere requisiti funzionali e non funzionali correlati a normative, procedure o regolamenti specifici del dominio applicativo.
 - Ad esempio, la conformità alle normative contabili o agli standard di interfaccia utente specifici per il dominio.

In sintesi, le categorie di requisiti aiutano a organizzare e distinguere diverse tipologie di informazioni necessarie per la definizione e lo sviluppo di un sistema software. I requisiti funzionali definiscono ciò che il sistema farà, i requisiti non funzionali definiscono come lo farà e le proprietà del sistema, mentre i requisiti di dominio tengono conto delle caratteristiche specifiche dell'ambiente in cui il sistema verrà utilizzato.

Classificazione dei Requisiti Non Funzionali:

- **Requisiti di Performance:** Definiscono le prestazioni richieste dal sistema, come tempi di risposta, velocità di elaborazione o capacità di gestire un certo volume di dati.
- **Requisiti di Sicurezza:** Riguardano la protezione dei dati, l'accesso autorizzato, l'integrità dei dati e la protezione da attacchi.
- **Requisiti di Affidabilità:** Definiscono quanto il sistema deve essere affidabile, inclusa la capacità di evitare guasti o di recuperare da essi.
- **Requisiti di Usabilità:** Si riferiscono all'esperienza dell'utente durante l'interazione con il sistema, come l'interfaccia utente intuitiva e la facilità d'uso.
- **Requisiti di Compatibilità:** Definiscono la capacità del sistema di operare con altre applicazioni o piattaforme.

- **Requisiti di Scalabilità:** Indicano la capacità del sistema di adattarsi a un aumento del carico o dei dati senza perdita di prestazioni.
- **Requisiti di Manutenibilità:** Definiscono quanto sia facile modificare, estendere o correggere il sistema nel tempo.
- **Requisiti di Conformità:** Si riferiscono alla necessità di rispettare normative, leggi o standard specifici.

Problemi con i Requisiti Software:

- **Ambiguità:** I requisiti possono essere interpretati in modi diversi da diverse parti interessate. Gli utenti e gli sviluppatori possono interpretare i requisiti in modi diversi, portando a risultati diversi.
- **Incompletezza:** I requisiti potrebbero non includere tutte le caratteristiche necessarie, portando a mancanze nel sistema finale.
- **Inconsistenza:** I requisiti potrebbero entrare in conflitto tra loro o contraddirsi, creando confusione e difficoltà nell'implementazione.

Verificabilità dei Requisiti:

I requisiti non funzionali devono essere espressi in modo **misurabile** e **quantificabile** per poter essere verificati.

Ad esempio, l'espressione "easy-to-use" è troppo generica e difficile da verificare, mentre una misura come "il tempo di risposta deve essere inferiore a 2 secondi" è quantificabile e verificabile.

DOCUMENTO DI SPECIFICA

Il **documento di analisi dei requisiti**, spesso chiamato "documento di specifica", è un documento ufficiale che rappresenta un aspetto cruciale nella fase di sviluppo del software. Esso contiene una **descrizione dettagliata** delle caratteristiche e dei requisiti del sistema che deve essere sviluppato. È un punto di riferimento essenziale per i team di sviluppo, gli utenti finali e altre parti coinvolte nel processo di sviluppo.

Il documento di specifica segue una struttura organizzata per assicurare **chiarezza** e **coerenza**. La struttura seguente è basata sullo **standard IEEE 830-1998, "IEEE Recommended Practice for Software Requirements Specifications"**:

- **Prefazione:** Questa sezione include informazioni sul pubblico previsto per il documento, la cronologia delle versioni precedenti e un riepilogo delle modifiche effettuate.
- **Introduzione:** Qui viene fornito uno scopo generale del sistema, una breve descrizione delle funzionalità principali e come il sistema interagisce con altri sistemi all'interno del contesto aziendale.
- **Glossario:** Una raccolta di definizioni di termini tecnici utilizzati all'interno del documento per garantire una comprensione uniforme da parte di tutti i lettori.
- **Definizione dei Requisiti Utente:** Questa sezione contiene i requisiti funzionali e non funzionali dal punto di vista degli utenti. Sono descritte le funzionalità che gli utenti si aspettano dal sistema.
- **Architettura del Sistema:** Fornisce una panoramica ad alto livello dei componenti del sistema e della loro interazione.
- **Specifiche dei Requisiti di Sistema:** Questa sezione dettaglia i requisiti funzionali e non funzionali del sistema. Definisce cosa il sistema deve fare e quali caratteristiche deve avere.
- **Modelli del Sistema:** Descrive i modelli che rappresentano le relazioni tra i componenti del sistema e l'ambiente in cui opera.
- **Evoluzione del Sistema:** Indica le supposizioni su cui si basa il sistema e le modifiche previste nel tempo, ad esempio, i cambiamenti nell'hardware o nelle esigenze degli utenti.
- **Appendici:** Questa sezione può contenere informazioni specifiche relative all'applicazione in sviluppo, come descrizioni dettagliate dell'hardware o del database utilizzati.
- **Indice:** Include una tabella dei contenuti, un indice alfabetico o altre indicazioni che facilitano la navigazione nel documento.

PROCESSO DI INGEGNERIA DEI REQUISITI

Il processo di ingegneria dei requisiti (requirements engineering) è una fase cruciale nello sviluppo del software in cui si raccolgono, analizzano e documentano le esigenze degli utenti e si traducono in requisiti chiari e dettagliati. Questo processo può variare in base al dominio applicativo, alle persone coinvolte e all'organizzazione che sviluppa il sistema software. Tuttavia, ci sono una serie di attività comuni a tutti i processi di ingegneria dei requisiti:

- **Studio di Fattibilità (Feasibility Study)**
- **Identificazione e Analisi dei Requisiti (Requirements Elicitation and Analysis)**
- **Specificazione dei Requisiti (Requirements Specification)**
- **Convalida dei Requisiti (Requirements Validation)**
- **Gestione dei Requisiti (Requirements Management)**

STUDIO DI FATTIBILITÀ

Lo studio di fattibilità è la **fase iniziale** del processo di ingegneria dei requisiti. Durante questa fase, vengono valutate le possibilità di sviluppo del sistema in base a considerazioni strategiche, tecniche ed economiche. Le informazioni necessarie per lo studio di fattibilità vengono raccolte attraverso **colloqui** con diverse parti coinvolte, tra cui client manager, esperti di settore, esperti tecnologici e utenti finali.

Report di Fattibilità:

Lo studio di fattibilità produce un report che fornisce un'analisi approfondita dell'opportunità di sviluppare il sistema software. Il report risponde a **domande cruciali**, come se il sistema contribuirà agli obiettivi strategici del cliente, se può essere sviluppato con le tecnologie e rispettando vincoli di tempo e costo, e se può essere integrato con altri sistemi esistenti.

ATTIVITÀ DI IDENTIFICAZIONE E ANALISI DEI REQUISITI

L'identificazione e l'analisi dei requisiti sono fondamentali nella fase di ingegneria dei requisiti e comportano la comprensione dettagliata delle **esigenze** degli **utenti** e la loro traduzione in **requisiti specifici** e chiari. Queste attività sono spesso svolte attraverso interazioni con il cliente e gli utenti finali, coinvolgendo vari ruoli all'interno e all'esterno dell'organizzazione.

- **Comprensione del Dominio:** L'analista deve acquisire conoscenze approfondite sul dominio applicativo in cui il sistema sarà utilizzato. Questo può implicare la comprensione dei processi aziendali, delle esigenze degli utenti e delle sfide specifiche del settore.
- **Raccolta dei Requisiti:** Gli stakeholder vengono coinvolti per identificare e definire i requisiti utente. Questo coinvolge interviste, sondaggi, workshop e altre tecniche per raccogliere informazioni dettagliate sulle esigenze.
- **Classificazione:** I requisiti raccolti vengono suddivisi in gruppi logici e coerenti. Questo aiuta a organizzare i requisiti e a gestirli in modo più efficace.
- **Risoluzione dei Conflitti:** Eventuali conflitti o contraddizioni tra requisiti vengono identificati e risolti. Questo assicura che i requisiti siano coerenti e allineati tra loro.
- **Assegnazione delle Priorità:** Gli stakeholder assegnano priorità ai requisiti in base all'importanza e all'urgenza. Questo aiuta a stabilire quali requisiti devono essere soddisfatti prima.
- **Verifica dei Requisiti:** I requisiti vengono controllati per assicurarsi che siano completi, non ambigui e coerenti. Ciò include anche la verifica che soddisfino le esigenze degli stakeholder.

Tecniche di Identificazione dei Requisiti:

- **Ethnography:** Un metodo che coinvolge l'osservazione diretta e l'interazione con gli utenti per comprendere le loro esigenze e il contesto in cui operano.
- **Casi d'Uso (Basati su Scenari):** Scenari dettagliati di come il sistema verrà utilizzato, aiutando a identificare i requisiti funzionali.
- **Prototipazione:** Creazione di prototipi di parti del sistema per mostrare agli utenti e ai clienti potenziali funzionalità e interazioni, raccogliendo feedback per definire meglio i requisiti.

Tecniche di Analisi dei Requisiti:

- **Semi-formali**, basate su modelli del sistema e usate dai metodi di analisi strutturata o analisi orientata agli oggetti
- **Formali** (basate su Petri Net, FSM, Z, etc.)

CONVALIDA DEI REQUISITI

La convalida dei requisiti è una fase cruciale nella fase di ingegneria dei requisiti. L'obiettivo principale è **verificare** se il **documento** dei requisiti, derivato dall'analisi, rappresenta correttamente le aspettative del cliente e se contiene requisiti chiari, completi e coerenti. Scoprire **errori** in questa fase è essenziale per evitare costosi **rework** e problemi nelle fasi successive del ciclo di vita del progetto.

Controlli da Effettuare:

- **Validità:** I requisiti devono rappresentare correttamente le esigenze degli stakeholder e del cliente. Devono rispondere alle vere necessità e obiettivi.
- **Consistenza:** I requisiti non devono entrare in conflitto tra loro o contraddirsi. Devono essere armoniosi e in accordo tra loro.
- **Completezza:** Tutti gli aspetti rilevanti del sistema devono essere coperti dai requisiti. Non dovrebbero esserci omissioni significative.
- **Realizzabilità:** I requisiti devono essere tecnologicamente fattibili e realizzabili entro i vincoli del progetto.
- **Verificabilità:** Deve essere possibile verificare se i requisiti sono stati soddisfatti attraverso test e altre attività di verifica.

Tecniche di Convalida dei Requisiti:

- **Revisioni Informali:** Incontri informali in cui gli stakeholder esaminano il documento dei requisiti e forniscono feedback.
- **Revisioni Formali:** Processi strutturati di revisione che includono walkthrough* e ispezioni per analizzare in modo dettagliato il documento dei requisiti.
- **Prototipazione:** Creazione di prototipi di parti del sistema per ottenere feedback dagli utenti e dai clienti sulla correttezza dei requisiti.
- **Generazione dei Test-Case:** Sviluppo di casi di test che riflettono i requisiti. La loro generazione può evidenziare eventuali ambiguità o mancanze.
- **Analisi di Consistenza Automatizzata:** Per requisiti formali, possono essere utilizzati strumenti di analisi formale per verificare la consistenza e la correttezza dei requisiti.

Un walkthrough è un processo in cui un gruppo di persone, come sviluppatori, tester o stakeholder, esaminano attentamente un prodotto, un documento o una parte del progetto per identificare errori, problemi o miglioramenti potenziali.

GESTIONE DEI REQUISITI

La gestione dei requisiti è un processo di identificazione e controllo delle **modifiche** che i requisiti subiscono durante il ciclo di vita del progetto.

I requisiti di un sistema possono essere classificati in base alla loro evoluzione:

- **Requisiti stabili:** Sono i requisiti che hanno una bassa probabilità di subire modifiche nel tempo. Questi sono spesso i requisiti principali e fondamentali che non dovrebbero cambiare frequentemente.
- **Requisiti volatili:** Sono i requisiti che hanno una probabilità elevata di subire modifiche nel tempo. Questi possono essere ulteriormente suddivisi in:
 - **Mutabili:** Le modifiche sono legate a cambiamenti nell'ambiente operativo, come nuovi requisiti legali o normativi.
 - **Emergenti:** Le modifiche sono causate da una migliore comprensione del sistema software, poiché il team acquisisce una visione più chiara delle esigenze degli utenti o dei dettagli tecnici.
 - **Conseguenziali:** Le modifiche sono legate all'introduzione di sistemi informatici nel flusso di lavoro aziendale, che richiedono adeguamenti nel sistema software.
 - **Di compatibilità:** Le modifiche sono legate a cambiamenti nei sistemi e nei processi aziendali, che possono richiedere modifiche per garantire che il sistema software rimanga compatibile con l'ambiente circostante.

Gestione delle Modifiche di Requisiti:

Le modifiche dei requisiti vanno opportunamente pianificate mediante:

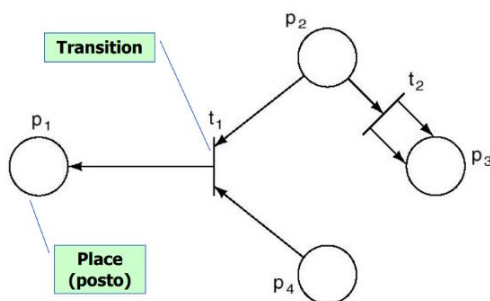
- **Identificazione Univoca dei Requisiti:** Ciascun requisito deve avere un'identificazione univoca per semplificare la tracciabilità e il controllo delle modifiche.
- **Gestione delle Modifiche:** Le modifiche vanno gestite in modo strutturato, includendo analisi dei costi, analisi dell'impatto e analisi della realizzazione.
- **Politiche di Tracciabilità:** Deve essere possibile tracciare le relazioni tra i requisiti e tra i requisiti e il progetto software. Ciò aiuta a capire come le modifiche influenzano l'intero sistema.
- **Uso di Tool CASE:** Strumenti di ingegneria del software (CASE) possono essere utilizzati per supportare la gestione delle modifiche, tracciabilità e controllo dei requisiti nel ciclo di vita del progetto.

SPECIFICHE FORMALI VS INFORMALI

Le specifiche dei requisiti possono essere espresse sia in forma **formale** che **informale**. Ogni approccio ha vantaggi e svantaggi, a seconda delle esigenze del progetto e delle preferenze dell'organizzazione. Ecco una panoramica delle due approcci:

	INFORMALI	FORMALI
VANTAGGI	• Facili da comprendere per gli stakeholder non tecnici. • Flessibilità nell'espressione dei requisiti. • Facilitano la comunicazione diretta con gli utenti e il cliente. • Semplici da scrivere e da interpretare inizialmente.	• Elevato grado di precisione e dettaglio. • Riducono l'ambiguità e le interpretazioni diverse. • Supportano l'analisi formale e l'automazione della verifica. • Adatte per sistemi critici o complessi.
SVANTAGGI	• Possono essere ambigue o soggette a interpretazioni diverse. • Manca di precisione matematica e dettaglio tecnico. • Non è adatta per requisiti complessi che richiedono chiarezza assoluta.	• Potrebbero essere difficili da comprendere per gli stakeholder non tecnici. • Richiedono competenze specializzate per la scrittura e l'interpretazione. • Potrebbero richiedere più tempo per essere create e gestite.

SPECIFICHE FORMALI CON PETRI NET



Le **reti di Petri** sono un modello **matematico** e **grafico** utilizzato per descrivere **sistemi concorrenti***. Sono costituite da i seguenti elementi principali:

- **Posti**: I posti rappresentano le condizioni o gli stati in cui può trovarsi un sistema. Possono contenere un certo numero di "gettoni" che rappresentano risorse, oggetti o stato.
- **Transizioni**: Le transizioni rappresentano gli eventi o le azioni che possono verificarsi all'interno del sistema. Per attivare una transizione, i posti devono contenere abbastanza gettoni per soddisfare i "pre-requisiti" di attivazione della transizione.
- **Archi**: Gli archi collegano i posti alle transizioni e viceversa. Gli archi indicano

il flusso dei gettoni tra posti e transizioni, determinando quale condizione deve essere soddisfatta per attivare una transizione.

Le reti di Petri possono essere rappresentate graficamente attraverso diagrammi in cui i **posti** sono rappresentati da **cerchi**, le **transizioni** da **rettangoli** e gli **archi** da **freccie**.

* I sistemi concorrenti, anche noti come sistemi paralleli, sono sistemi informatici o reali in cui diverse attività, processi o entità vengono eseguiti simultaneamente o in parallelo. Questi sistemi gestiscono più attività contemporaneamente, consentendo loro di condividere risorse, lavorare insieme o agire indipendentemente.*

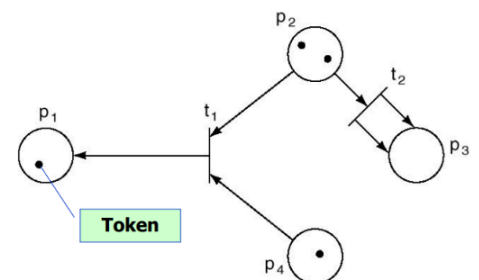
Le **specifiche formali** con le **reti di Petri** implicano l'utilizzo delle reti di Petri come strumento per descrivere e definire in modo preciso il comportamento di un sistema. Questo approccio consente di rappresentare il funzionamento del sistema in modo dettagliato e accurato, fornendo una base per l'analisi, la verifica e la simulazione delle proprietà del sistema stesso.

Marked Petri Net:

Una Marked Petri Net (rete di Petri marcata) è un tipo di rete di Petri che incorpora il concetto di "**marcature**". Le marcature rappresentano l'assegnazione di "**gettoni**" (di solito rappresentati come punti) ai posti all'interno della rete di Petri. Questi gettoni indicano la presenza di risorse, dati o lo stato del sistema in un determinato momento.

In una Marked Petri Net:

- **Posti**: Rappresentano condizioni, stati o posizioni nel sistema. Ogni posto può contenere un certo numero di gettoni (marcature), che simboleggiano risorse o oggetti.
- **Transizioni**: Rappresentano eventi, azioni o processi che possono verificarsi nel sistema. Le transizioni vengono attivate quando hanno il numero necessario di gettoni nei posti di input.
- **Archetti**: Collegano posti a transizioni e transizioni a posti. Indicano il flusso dei gettoni e stabiliscono le condizioni necessarie affinché una transizione possa scattare.



- **Marcatore:** Una marcatura è una distribuzione specifica di gettoni tra i posti nella Rete di Petri. Rappresenta lo stato attuale del sistema. Le marcature cambiano quando le transizioni vengono attivate, i gettoni vengono consumati e ne vengono creati di nuovi.

Le Marked Petri Nets sono utili per comprendere la **concorrenza**, l'**allocazione di risorse**, la **sincronizzazione** e i potenziali **punti critici** nei sistemi. Sono particolarmente utili quando si vuole studiare come i gettoni si spostano attraverso una rete di posti e transizioni e come il sistema reagisce a diverse sequenze di eventi.

SPECIFICHE FORMALI CON FINITE STATE MACHINE (FSM)

Le Finite State Machine (FSM), o Macchine a Stati Finiti, sono un modello matematico utilizzato per rappresentare il comportamento di sistemi che possono trovarsi in uno dei diversi stati finiti. In questo contesto, le FSM possono essere utilizzate per esprimere specifiche formali dei requisiti in modo preciso.

Nel contesto delle specifiche formali, una FSM può essere utilizzata per rappresentare il **comportamento** di un **sistema** attraverso gli **stati** che può assumere e le **transizioni** tra di essi. Ogni **stato** rappresenta una **condizione** specifica del sistema, mentre le **transizioni** rappresentano gli **eventi** o le **azioni** che causano il passaggio da uno stato all'altro. Le FSM possono essere utilizzate per specificare sequenze di azioni, condizioni di errore e comportamenti del sistema in modo dettagliato.

SPECIFICHE FORMALI CON LINGUAGGIO Z

S
declarations
predicates

Il **linguaggio Z** è un linguaggio formale basato sulla **logica** dei **predicati** del **primo ordine** e utilizzato per la specifica formale dei requisiti e il design di sistemi software. Il linguaggio Z è noto per la sua capacità di esprimere specifiche in modo preciso e non ambiguo, consentendo la rappresentazione formale di proprietà del sistema.

È ampiamente utilizzato nell'ingegneria del software per specificare i requisiti dei sistemi, modellare il loro comportamento e verificare le proprietà desiderate.

Le **specifiche formali** con il **linguaggio Z** sono un modo per descrivere in modo preciso e rigoroso i **requisiti** e il **comportamento** di un sistema utilizzando la **notazione matematica** del linguaggio Z. Queste specifiche consentono di eliminare **ambiguità**, **fraintendimenti** e **interpretazioni erranee** durante lo sviluppo e la verifica dei sistemi.

SPECIFICHE SEMI-FORMALI: MODELLI DEL SISTEMA

Le specifiche semi-formali rappresentano un **approccio intermedio** tra le specifiche informali e quelle completamente formali. Queste specifiche utilizzano **modelli astratti** per descrivere le **proprietà** e il **comportamento** di un sistema software in un modo **più strutturato** rispetto alle specifiche **informali**, ma **meno rigoroso** delle specifiche completamente **formali**. I modelli del sistema vengono utilizzati per rappresentare diverse prospettive e aspetti del sistema stesso.

Un **modello del sistema** è una rappresentazione astratta del sistema software che si intende sviluppare. Questo modello cattura le caratteristiche essenziali del sistema, come le sue funzioni, il comportamento e le interazioni con l'ambiente.

L'uso di modelli dei sistemi software è formalizzato all'interno di metodi di analisi dei requisiti (**specifici**) del software che fanno uso di tecniche **semi-formali**.

Esistono diversi approcci per analizzare i requisiti del software. Alcuni sono orientati alle **procedure (analisi strutturata)**, mentre altri si basano sull'orientamento agli **oggetti (analisi orientata agli oggetti)**.

Per ottenere una **comprensione completa** di un sistema, è spesso necessario creare diversi modelli che rappresentano il sistema da diversi punti di vista. Questi possono includere la rappresentazione delle informazioni, delle funzioni e del comportamento dinamico.

Tipi di Modelli del Sistema:

Per descrivere la specifica semi-formale di un sistema software si usano 3 tipi di modelli:

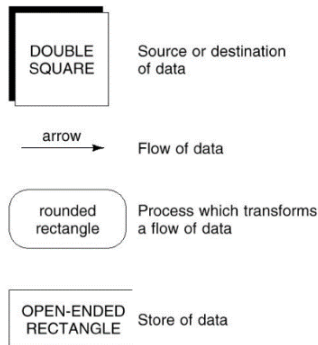
- **Modello dei Dati:** Questo modello rappresenta gli aspetti statici e strutturali dei dati all'interno del sistema. Include entità, attributi, relazioni e strutture dati. I modelli dei dati sono utilizzati per definire i requisiti relativi ai dati e alla loro organizzazione.
 - Esempi: Entity-Relationship Diagrams (ERD), Class Diagrams (UML).
- **Modello Comportamentale:** Questo modello rappresenta gli aspetti funzionali e comportamentali del sistema, descrivendo come il sistema interagisce con gli utenti e come le diverse parti del sistema interagiscono tra loro.
 - Esempi: Data Flow Diagrams (DFD), Use Case Diagrams, Activity Diagrams, Interaction Diagrams (UML).
- **Modello Dinamico:** Questo modello rappresenta gli aspetti dinamici del sistema, cioè come il sistema cambia e reagisce alle varie condizioni e eventi. È utilizzato per descrivere lo stato e il comportamento del sistema nel tempo.
 - Esempi: State Diagrams (UML).

ENTITY RELATIONSHIP DIAGRAM (ERD)

L'Entity Relationship Diagram (ERD) è un tipo di diagramma utilizzato per rappresentare graficamente le **relazioni** tra le **entità** in un sistema o in un database. Gli ERD sono comunemente utilizzati nel contesto della progettazione dei database per visualizzare come le diverse tabelle (enti) sono collegate tra loro attraverso le relazioni. Gli ERD sono costituiti da tre componenti principali: **entità, attributi e relazioni**.

- **Relazioni Uno-a-Molti:** Questa relazione indica che ogni istanza dell'entità da un lato può essere associata a molte istanze dell'entità dall'altro lato, ma ogni istanza dell'entità dall'altro lato è associata solo a un'istanza dell'entità da un lato.
- **Relazioni Molti-a-Molti:** Questa relazione indica che molte istanze dell'entità da un lato possono essere associate a molte istanze dell'entità dall'altro lato. Spesso, nelle implementazioni reali, le relazioni molti-a-molti vengono gestite attraverso l'introduzione di una terza tabella (detta tabella di giunzione o tabella di associazione) per gestire l'associazione tra le entità.

DATA FLOW DIAGRAM (DFD)



I Data Flow Diagram (DFD) sono strumenti **grafici** utilizzati per rappresentare visivamente il **flusso dei dati** attraverso un sistema o un processo. Sono spesso utilizzati nell'analisi dei sistemi e nell'ingegneria del software per modellare come i **dati** vengono acquisiti, elaborati, trasformati e distribuiti all'interno di un sistema. I DFD sono costituiti da quattro elementi principali:

- **Processi:** Sono le attività o le operazioni che trasformano i dati. I processi sono rappresentati da cerchi o rettangoli con una descrizione breve del loro scopo.
- **Flussi di dati:** Sono le frecce che rappresentano il movimento dei dati tra i processi, gli entità esterne e i depositi dei dati. I flussi di dati indicano il percorso dei dati e spesso sono etichettati con il nome del dato.
- **Entità esterne:** Rappresentano le fonti o le destinazioni dei dati esterne al sistema in esame. Possono essere altre applicazioni, utenti, database, ecc.
- **Depositi dati:** Sono le posizioni in cui i dati vengono memorizzati. Questi possono essere database, file o altre forme di archiviazione dei dati.

I DFD sono utili per comprendere il flusso dei dati all'interno di un sistema, identificare aree di miglioramento, individuare potenziali problemi di progettazione e stabilire una base per la progettazione e lo sviluppo successivo. Sono particolarmente efficaci per modellare sistemi complessi in modo chiaro e strutturato.

- **Primo Raffinamento:** Nei DFD, il primo raffinamento comporta la suddivisione dei processi di alto livello in processi più dettagliati. Questo livello di raffinamento mostra come i processi principali vengono suddivisi in sotto-processi più piccoli e dettagliati. Questo aiuta a definire meglio le attività che avvengono all'interno del sistema.
- **Secondo Raffinamento:** Nel secondo raffinamento, i sotto-processi ottenuti dal primo raffinamento vengono ulteriormente suddivisi in attività più dettagliate. Questo livello di dettaglio aiuta a definire con precisione le operazioni che avvengono all'interno di ogni processo.

STRUCTURED SYSTEM ANALYSIS (SSA)

L'Analisi Strutturata dei Sistemi (Structured System Analysis - SSA) è un metodo di **analisi strutturata** introdotto da Gane e Sarson nel 1979. Questo metodo è progettato per **analizzare** e **progettare** sistemi complessi suddividendoli in componenti più piccole. L'approccio di SSA prevede un processo graduale di perfezionamento, in cui ogni passaggio affina l'analisi e la progettazione del sistema. Ecco una panoramica dei 9 passaggi coinvolti nel metodo SSA:

1. **Disegnare il Diagramma di Flusso dei Dati (DFD)**
 - a. Identificare flussi di dati, fonti, destinazioni e processi in base ai requisiti.
 - b. Perfezionare il DFD aggiungendo nuovi flussi di dati e dettagli ai flussi esistenti.
2. **Decidere Quali Sezioni Computerizzare e Come**
 - a. Utilizzare l'analisi costo-beneficio per determinare quali sezioni del DFD automatizzare.
 - b. Decidere tra operazioni batch* ed elaborazione online*.
3. **Determinare i Dettagli dei Flussi di Dati**
 - a. Specificare elementi dati per ogni flusso di dati e perfezionarli passo dopo passo.
4. **Definire la Logica dei Processi**
 - a. Definire la logica dei processi utilizzando tecniche come gli alberi decisionali.
5. **Determinare i Depositi di Dati**
 - a. Definire il contenuto di ciascun deposito di dati e la sua rappresentazione.
 - b. Specificare livelli di accesso dati usando un diagramma di accesso immediato ai dati (DIAD).
6. **Definire le Risorse Fisiche**
 - a. Specificare l'organizzazione e la struttura di file, supporti di archiviazione e tabelle di database.

7. Determinare le Specifiche di Input/Output

- Specificare moduli di input, schermate di output e dettagli delle stampe.

8. Determinare le Dimensioni

- Calcolare il volume di input, la frequenza dei report, le dimensioni dei record e dei file.

9. Determinare i Requisiti Hardware

- Determinare le necessità di archiviazione massiva, i requisiti di backup, le caratteristiche dei terminali e altro.
- Valutare l'hardware esistente e stimare i costi per il nuovo hardware.

* Le operazioni in batch (batch operations) si riferiscono a un metodo di elaborazione in cui un gruppo di attività o operazioni viene raccolto ed eseguito insieme in un'unica sequenza, senza intervento diretto dell'utente. L'elaborazione online (on-line processing) si riferisce all'esecuzione immediata di operazioni in risposta alle azioni dell'utente o agli eventi del sistema. In questo caso, l'utente interagisce direttamente con il sistema e ottiene una risposta immediata dopo aver inviato un'operazione o un'azione.*

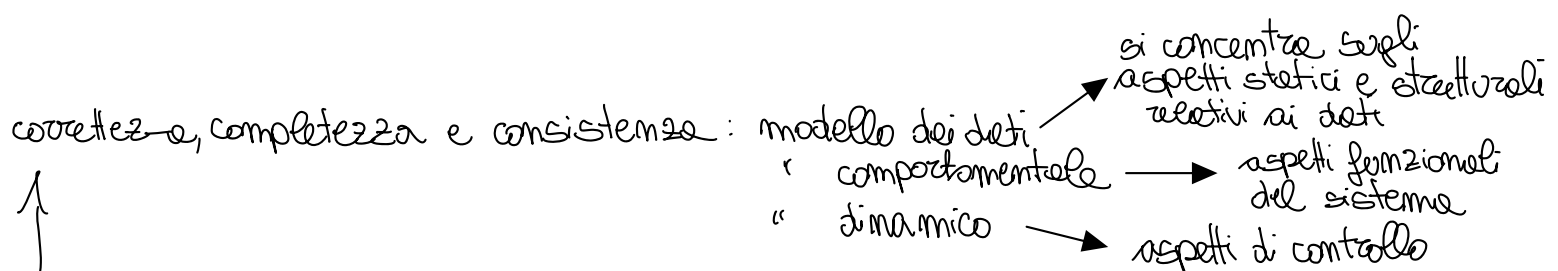
Output di SSA:

Il risultato del processo SSA è un **documento di specifica** dettagliato che delinea la **progettazione** e i **requisiti** del sistema. Una volta approvato dal cliente, questo documento viene consegnato al team di progettazione per procedere con lo sviluppo del software.

Limitazioni di SSA:

Nonostante fornisca un approccio strutturato all'analisi e alla progettazione, SSA ha limitazioni:

- Non può determinare con precisione i tempi di risposta o le dimensioni della **CPU**.
- Gli aspetti di tempistica e prestazioni non sono affrontati con **precisione**.



ANALISI ORIENTATA AGLI OGGETTI (OOA)

L'**Analisi Orientata agli Oggetti** (Object Oriented Analysis - OOA) è una fase del processo di sviluppo software che si focalizza sulla definizione di **COSA** un prodotto software deve fare. Questa fase è distinta dalla fase successiva, la **Progettazione Orientata agli Oggetti** (Object Oriented Design - OOD), che si concentra su **COME** implementare ciò che è stato definito in fase di OOA. Insieme, OOA e OOD aiutano a sviluppare un sistema orientato agli oggetti che sia ben strutturato, modulare e in grado di soddisfare i requisiti dell'utente.

METODI DI OOA

I metodi di Analisi Orientata agli Oggetti (OOA) sono approcci strutturati e sistematici utilizzati per guidare lo sviluppo di software orientato agli oggetti. Questi metodi offrono un insieme di procedure, tecniche e strumenti per affrontare la fase di OOA in modo efficace.

- **Input:** Gli **input** di un metodo di OOA sono costituiti dall'**insieme dei requisiti utente**, che sono raccolti nel documento di analisi dei requisiti. Questi requisiti rappresentano le necessità e le aspettative degli utenti per il sistema software.
- **Output:** Gli **output** di un metodo di OOA sono rappresentati dai modelli del sistema. Questi modelli definiscono la specifica del prodotto software e rappresentano gli aspetti statici e funzionali del sistema. Questi modelli sono solitamente inclusi nel documento di analisi dei requisiti. *requisiti di sistema*
- **Notazioni Visuali:** I metodi di OOA fanno ampio uso di notazioni visive, come i diagrammi UML (Unified Modeling Language), per rappresentare graficamente le classi, le relazioni, i casi d'uso, le sequenze e altre informazioni rilevanti.
- **Parallelismo e Iterazione:** Lo sviluppo dei modelli di OOA non è un processo sequenziale. Invece, i vari modelli vengono sviluppati in parallelo e si influenzano a vicenda. I metodi di OOA seguono un **approccio iterativo**. Durante le iterazioni successive, i modelli vengono raffinati, dettagliati e migliorati in base ai feedback, alle nuove comprensioni e ai cambiamenti dei requisiti.

Notazioni per OOA e OOD:

Nell'Analisi e Progettazione Orientata agli Oggetti (OOA/OOD), ciascun metodo fa uso di specifiche notazioni visive per rappresentare i modelli del sistema. Queste notazioni aiutano gli sviluppatori a comunicare e visualizzare le caratteristiche strutturali e comportamentali del software in modo chiaro e conciso. Tuttavia, l'adozione di diverse notazioni da parte dei vari metodi poteva creare confusione e difficoltà nell'interazione tra team diversi.

Per affrontare questa sfida, è stato introdotto il **Unified Modeling Language (UML)**, che è diventato uno standard per la rappresentazione visuale dei modelli di sistemi software orientati agli oggetti.

UNIFIED MODELING LANGUAGE (UML)

UML è un linguaggio standard per la descrizione visuale dei sistemi software orientati agli oggetti. È stato introdotto nel 1997 e adottato come standard dall'Object Management Group (OMG). UML fornisce un insieme di notazioni grafiche, chiamate "**diagrammi**", che rappresentano diverse prospettive e aspetti dei modelli del sistema ed è composto da nove formalismi di base, che corrispondono a diversi tipi di diagrammi, ciascuno con una semantica e una notazione specifica.

Formalismi UML:

I diagrammi UML sono rappresentazioni grafiche utilizzate nell'Analisi e Progettazione Orientata agli Oggetti per descrivere visivamente diversi aspetti e prospettive di un sistema software. Questi diagrammi consentono agli sviluppatori di comunicare in modo chiaro e conciso le caratteristiche strutturali e comportamentali del sistema.

- **Diagramma dei casi d'uso (Use Case Diagram):** Evidenzia i casi d'uso, che rappresentano le interazioni tra gli attori (utenti o altri sistemi) e il sistema. Mostra come gli utenti utilizzano il sistema e quali funzionalità vengono svolte in risposta a determinati scenari.
- **Diagramma delle classi (Class Diagram):** Rappresenta le classi del sistema con i loro attributi, operazioni e le relazioni tra di esse. Descrive la struttura statica del sistema e le associazioni tra le classi.
- **Diagramma degli stati (State Diagram):** Illustra il comportamento dinamico di un oggetto o di una classe attraverso vari stati e transizioni tra di essi in risposta agli eventi. È utile per modellare il ciclo di vita degli oggetti e il loro comportamento in base alle variazioni di stato.
- **Diagramma delle attività (Activity Diagram):** Mostra flussi di attività, decisioni e azioni nel sistema. È utile per rappresentare processi aziendali, workflow o sequenze di attività.
- **Diagramma di sequenza (Sequence Diagram):** Visualizza le interazioni tra oggetti in sequenza temporale, mostrando i messaggi scambiati tra di essi e l'ordine in cui avvengono. Evidenzia come le diverse parti del sistema cooperano durante l'esecuzione di un caso d'uso.
- **Diagramma di collaborazione (Collaboration Diagram):** Rappresenta le interazioni tra oggetti, mostrando gli oggetti coinvolti e i messaggi scambiati tra di essi. Sottolinea le relazioni tra le istanze delle classi coinvolte nell'interazione.

- **Diagramma degli oggetti (Object Diagram):** Mostra un'istantanea specifica dello stato dei vari oggetti e delle loro relazioni in un determinato momento. È simile al diagramma di classe, ma rappresenta oggetti reali invece di classi astratte.
- **Diagramma dei componenti (Component Diagram):** Rappresenta la struttura e le dipendenze tra i componenti software, come librerie, moduli o file sorgente. Utile per la progettazione e la comunicazione tra sviluppatori.
- **Diagramma di deployment (Deployment Diagram):** Mostra la disposizione fisica dei componenti software su nodi hardware e le connessioni tra di essi. Utile per pianificare la distribuzione e l'implementazione del sistema.

MODELLO DEI DATI

Il modello dei dati rappresenta l'organizzazione logica e strutturale dei dati che verranno elaborati all'interno di un sistema software. Nel contesto dell'approccio ad oggetti, le strutture dati vengono definite attraverso gli oggetti, i cui stati sono determinati dai valori degli attributi e dalle associazioni tra oggetti.

Il modello dei dati è specificato attraverso il **formalismo dei diagrammi delle classi (Class Diagrams)** nell'UML. Questi diagrammi consentono di definire:

- **Classi:** Rappresentano le entità fondamentali del sistema, ciascuna delle quali può contenere attributi (dati) e operazioni (comportamenti).
- **Attributi di ciascuna classe:** Sono le proprietà che definiscono le caratteristiche di ciascuna classe.
- **Operazioni di ciascuna classe:** Rappresentano le azioni che una classe può eseguire o le funzioni che può fornire.
- **Associazioni tra classi:** Indicano come le diverse classi interagiscono tra loro e possono essere unidirezionali o bidirezionali.

La costruzione del modello dei dati è un processo **iterativo** ed **incrementale**, il che significa che viene sviluppato gradualmente e può essere affinato nel tempo. Questo processo è altamente **creativo** e richiede una profonda comprensione del dominio applicativo.

Nella fase iniziale di costruzione, è importante concentrarsi sulle **entity classes**. Queste classi definiscono le entità principali nel dominio applicativo e sono fondamentali per la creazione di un modello accurato.

Le **control classes (classi di controllo)** e le **boundary classes (classi di confine)** vengono introdotte successivamente nel processo di costruzione del modello. Le prime gestiscono la logica interna del sistema, mentre le seconde rappresentano l'interfaccia utente del sistema.

Le **operazioni** di ciascuna classe vengono identificate progressivamente a partire dal **modello comportamentale**, che evidenzia come gli oggetti interagiscono nel sistema. Inizialmente, le operazioni possono essere trascurate in favore di una comprensione più approfondita delle interazioni e delle relazioni tra le classi.

APPROCCIO PER L'IDENTIFICAZIONE DELLE CLASSI

Approccio Noun Phrase:

Questo approccio si basa sull'analisi delle **frasi nominali** presenti nel documento dei requisiti utente.

Una frase nominale* è una frase in cui il sostantivo ha un ruolo predominante rispetto alla parte verbale. I **sostantivi** all'interno di queste frasi vengono considerati potenziali **candidate classes**, cioè classi che potrebbero essere parte del sistema da sviluppare. L'analisi inizia identificando i sostantivi all'interno delle frasi nominali presenti nel documento dei requisiti. Questi sostantivi vengono considerati possibili candidati per le classi del sistema software. Successivamente, le candidate classi vengono raggruppate in tre categorie:

- **Irrelevant:** Queste sono le classi che non appartengono al dominio applicativo del sistema e quindi possono essere scartate. Non hanno rilevanza per il progetto.
- **Relevant:** Queste classi sono rilevanti per il dominio applicativo del sistema. Esse evidenziano caratteristiche che sono importanti per la realizzazione del software. Possono rappresentare entity classes che definiscono concetti chiave all'interno del dominio.
- **Fuzzy:** Queste classi non possono essere immediatamente classificate come rilevanti o irrilevanti. Potrebbero mancare informazioni sufficienti per determinarne la rilevanza. Queste classi vengono considerate in modo più approfondito in seguito, quando saranno disponibili ulteriori dettagli.

È importante sottolineare che l'approccio noun phrase si basa sull'assunzione che l'insieme di requisiti utente sia **completo e corretto**. Questo approccio può essere un modo utile per identificare le classi in modo iniziale, ma spesso viene affiancato da altri approcci per ottenere una visione più completa e accurata delle classi necessarie per il sistema.

*Una **frase nominale** è una "frase" o gruppo di parole che ruota attorno a un sostantivo o a un pronome, dando maggiori dettagli o contesto a quella parola chiave. Le frasi nominali sono spesso utilizzate per chiarire o espandere il significato di un sostantivo all'interno di una frase più ampia.

Esempio. "Il grande elefante grigio" - In questa frase nominale, "il grande elefante grigio" è la frase che descrive il sostantivo "elefante". Gli aggettivi "grande" e "grigio" forniscono dettagli sul tipo di elefante.*

Approccio Common Class Patterns:

Questo approccio si basa sulla teoria della **classificazione** e si concentra sull'identificazione di classi predefinite o pattern comuni che si verificano spesso nei sistemi software. Invece di derivare classi dalle frasi nominali dei requisiti utente, come nell'approccio noun phrase, questo approccio cerca di identificare classi basate su categorie di concetti che sono comunemente presenti nei sistemi.

Le **candidate classes** vengono identificate a partire da gruppi di classi predefinite, chiamate "**common class patterns**". Questi pattern rappresentano concetti generali che possono essere applicati a vari domini. Alcuni esempi di common class patterns potrebbero includere:

- **Concept:** rappresenta la prenotazione di risorse o servizi.
- **Events:** rappresenta eventi o azioni che si verificano nel sistema.
- **Organization:** rappresenta organizzazioni o aziende.
- **People:** rappresenta persone coinvolte nel sistema.
- **Places:** rappresenta luoghi o sedi.

Questo approccio non è necessariamente sistematico e può essere utilizzato come guida per identificare classi comuni. Tuttavia, a differenza dell'approccio noun phrase, non si basa sul documento dei requisiti utente. Poiché i nomi delle classi possono essere interpretati in modi diversi, l'approccio common class patterns potrebbe causare **problemi di interpretazione** e richiedere un'adeguata comprensione del dominio applicativo per essere efficace. L'approccio common class patterns può essere un punto di partenza utile per l'identificazione delle classi, ma spesso viene integrato con altri approcci per ottenere una visione completa delle classi necessarie per il sistema.

Approccio Use Case Driven:

Questo approccio si basa sul presupposto che gli **use case diagram**, insieme alla descrizione testuale degli scenari di funzionamento dei casi d'uso, forniscono una base solida per identificare le classi del sistema. Gli use case diagram rappresentano come gli attori (utenti o sistemi esterni) interagiscono con il sistema, mentre le descrizioni testuali degli scenari forniscono dettagli su come il sistema deve rispondere a tali interazioni.

L'approccio use case driven include i seguenti passaggi:

- **Sviluppo degli Use Case Diagram:** Prima di tutto, vengono creati gli use case diagram per rappresentare le interazioni tra attori e il sistema. Gli use case diagram evidenziano i casi d'uso principali e le interazioni tra attori e il sistema stesso.
- **Descrizione degli Scenari:** Per ciascun caso d'uso, viene fornita una descrizione dettagliata degli scenari di funzionamento. Queste descrizioni testuali rappresentano una sequenza di azioni che un attore svolge interagendo con il sistema e il modo in cui il sistema risponde a tali azioni.
- **Identificazione delle Classi:** Durante l'analisi degli scenari di funzionamento, le classi coinvolte nell'esecuzione degli scenari emergono naturalmente. Le classi vengono identificate considerando gli oggetti che compaiono nei vari passaggi degli scenari e le loro interazioni.
- **Associazioni e Attributi:** Oltre all'identificazione delle classi, vengono anche considerate le associazioni tra le classi e gli attributi che devono essere associati a ciascuna classe per supportare le funzionalità descritte negli scenari.
- **Raffinamenti Successivi:** L'approccio use case driven è iterativo e prevede raffinamenti successivi. Man mano che gli scenari vengono esplorati e dettagliati, le classi e le loro relazioni possono essere riviste e aggiornate.

Questo approccio è simile all'approccio noun phrase nel senso che entrambi si basano sui **requisiti utente** per identificare le classi. Tuttavia, l'approccio use case driven si concentra specificamente sugli use case e sugli scenari di funzionamento, garantendo che le classi siano direttamente rilevanti per le interazioni degli attori con il sistema.

Approccio CRC (Class-Responsibility-Collaborators):

Questo approccio coinvolge una serie di **riunioni** di gruppo in cui gli analisti, i progettisti e gli sviluppatori lavorano insieme per identificare le classi del sistema. L'approccio prende il nome dall'acronimo CRC, che sta per Class, Responsibility e Collaborators. Il processo segue questi passaggi:

- **Riunioni di Gruppo:** Il processo inizia con riunioni in cui vengono coinvolti membri del team di sviluppo. Ogni riunione è focalizzata su un particolare aspetto del sistema.
- **Creazione di Card CRC:** Durante le riunioni, vengono create card CRC per rappresentare le classi del sistema. Ogni card CRC include tre compartimenti:
 - **Classe:** Il nome della classe che sta per essere identificata.
 - **Responsabilità:** Le responsabilità che la classe avrà nel sistema. Questo include le funzioni che la classe dovrà svolgere.

- **Collaboratori:** Il nome delle altre classi con cui la classe in oggetto collaborerà per svolgere le sue responsabilità.
- **Discussione e Analisi:** Durante la creazione delle card CRC, il gruppo discute e analizza come le classi collaborano tra loro e quali sono le responsabilità di ciascuna classe.
- **Identificazione di Attributi e Operazioni:** Durante le discussioni, vengono identificati anche gli attributi e le operazioni associati a ciascuna classe, sebbene questi dettagli possano essere ulteriormente rifiniti in fasi successive.
- **Raffinamenti Successivi:** L'approccio CRC è iterativo, quindi le card possono essere riviste e aggiornate in fasi successive a mano a mano che il team sviluppa una comprensione più completa delle classi e delle loro interazioni.

Approccio Misto (Mixed):

Questo è un approccio che **combina** elementi da diversi approcci precedenti. Potrebbe iniziare con l'identificazione delle classi basata sull'esperienza dell'analista o sui modelli comuni di classi. Successivamente, potrebbero essere aggiunte classi utilizzando l'approccio Noun Phrase o Use Case Driven. Infine, l'approccio CRC può essere utilizzato per verificare e rifinire l'insieme delle classi identificate.

L'approccio misto offre un approccio flessibile e adattabile all'identificazione delle classi, consentendo al team di sviluppo di sfruttare i punti di forza di diverse tecniche per ottenere una visione più completa e accurata del sistema in fase di analisi.

L'obiettivo principale è raggiungere una rappresentazione delle classi che rifletta al meglio le esigenze del dominio e le interazioni all'interno del sistema.

LINEE GUIDA PER L'IDENTIFICAZIONE DELLE ENTITY CLASSES

Le linee guida per l'identificazione delle **entity classes** sono fondamentali durante la fase di **Object-Oriented Analysis (OOA)**. Le entity classes rappresentano le entità principali del dominio applicativo e costituiscono il cuore del modello dei dati.

- **Statement of Purpose:** Ogni classe deve avere un chiaro e ben definito "statement of purpose" o scopo. Questo significa che ogni classe dovrebbe avere una ragione specifica per esistere nel sistema. Questo scopo dovrebbe essere descritto in modo chiaro e conciso, in modo che sia evidente quale ruolo la classe gioca all'interno del sistema.
- **Insieme di Istanze:** Una classe dovrebbe rappresentare un insieme di oggetti o istanze del mondo reale. Questi oggetti dovrebbero condividere le stesse proprietà e comportamenti descritti dalla classe. Tuttavia, le "singleton classes", che rappresentano oggetti per i quali è prevista una singola istanza, di solito non sono considerate entity classes, poiché non rappresentano una collezione di istanze simili.
- **Insieme di Attributi:** Ogni classe dovrebbe prevedere un insieme di attributi che rappresentano le proprietà o le caratteristiche degli oggetti della classe. Gli attributi descrivono lo stato dell'oggetto e sono utilizzati per memorizzare informazioni specifiche. È importante considerare un insieme significativo di attributi che siano pertinenti per il dominio.
- **Differenza tra Classi e Attributi:** È importante distinguere tra le entità che dovrebbero essere modellate come classi e quelle che dovrebbero essere modellate come attributi. Le classi rappresentano concetti significativi e persistenti all'interno del dominio, mentre gli attributi rappresentano caratteristiche specifiche di un'istanza di classe.
- **Insieme di Operazioni:** Sebbene inizialmente le operazioni possano essere trascurate nella fase iniziale dell'identificazione delle classi, è importante prevedere un insieme di operazioni che una classe può eseguire. Queste operazioni rappresentano i servizi che la classe offre per manipolare i dati o interagire con altre parti del sistema. Le operazioni possono essere derivate implicitamente dallo "statement of purpose" della classe.

Seguendo queste linee guida, è possibile identificare in modo efficace e accurato le entity classes all'interno del modello dei dati, assicurandosi che riflettano il dominio applicativo e le sue esigenze.

CASI DI STUDIO

I casi di studio forniscono scenari di esempio realistici in cui è possibile applicare i principi dell'analisi orientata agli oggetti, identificare le classi, le relazioni e i comportamenti rilevanti e costruire modelli adeguati per rappresentare il sistema. Ogni caso di studio presenta sfide specifiche che richiedono una comprensione approfondita delle esigenze del dominio e l'applicazione di tecniche di analisi appropriate.

University Enrolment:

Questo caso di studio riguarda un sistema per la gestione delle iscrizioni universitarie. Si tratta di un'applicazione che supporta il processo di iscrizione degli studenti a corsi universitari. Il sistema deve essere in grado di gestire programmi di studio personalizzati, verificare i prerequisiti dei corsi, gestire le iscrizioni e risolvere eventuali problemi come sovrapposizioni di orari.

- **Scenari:**
 - L'università offre corsi di laurea di primo e secondo livello (undergraduate e postgraduate) sia a studenti a tempo pieno che a studenti part-time.
 - La struttura dell'università prevede divisioni che contengono dipartimenti. Ogni corso di laurea è amministrato da una singola divisione, ma un corso di laurea può includere corsi provenienti da altre divisioni.

- **Caratteristiche del Sistema:**
 - Il sistema di iscrizione universitaria deve consentire la creazione di programmi di studio personalizzati per gli studenti.
 - I programmi di studio possono avere corsi prerequisiti, corsi obbligatori e restrizioni, come sovrapposizioni di orario o dimensioni massime delle classi.
- **Obiettivi del Sistema:**
 - Aiutare nelle attività pre-iscrizione, come l'invio di voti degli esami dell'ultimo semestre agli studenti e fornire istruzioni per l'iscrizione.
 - Gestire le procedure di iscrizione, tra cui l'accettazione dei programmi di studio proposti dagli studenti.
 - Effettuare la convalida per i prerequisiti dei corsi, sovrapposizioni di orario, dimensioni delle classi e approvazioni speciali.
- **Considerazioni Aggiuntive:**
 - Alcune problematiche possono richiedere la consultazione con consulenti accademici o docenti responsabili delle offerte dei corsi. Questo implica un coinvolgimento attivo di docenti esperti nel processo di iscrizione.
 - Il sistema deve gestire i dettagli del programma di studio di ciascuno studente in modo accurato e affidabile, considerando tutte le regole e i requisiti specifici.
 - La flessibilità nel sistema è cruciale per adattarsi alle esigenze dei diversi studenti e dei diversi corsi.

Nell'analisi orientata agli oggetti, questo caso di studio richiederebbe l'identificazione delle classi (come "Student", "Course", "Program", "Division", ecc.) e delle loro proprietà, associazioni e comportamenti. I diagrammi UML, come i diagrammi dei casi d'uso, i diagrammi delle classi e i diagrammi di sequenza, possono essere utilizzati per rappresentare in modo chiaro ed esaustivo il sistema e le sue dinamiche.

Video Store:

In questo caso di studio, il sistema riguarda una videoteca che offre il noleggio di film in formato fisico (videocassette e dischi). Il sistema deve gestire l'inventario dei film, i dati dei clienti, le prenotazioni dei film, il noleggio e la restituzione dei film. È anche necessario gestire le richieste dei clienti e fornire informazioni sui film disponibili.

- **Scenari:**
 - Il negozio di noleggio offre noleggio di videocassette e dischi a clienti.
 - Ogni videocassetta e disco ha un codice a barre univoco.
 - Anche i membri del negozio hanno codici a barre univoci per identificare i loro account.
- **Funzionalità del Sistema:**
 - Il sistema deve consentire ai clienti di noleggiare videocassette e dischi.
 - I clienti possono effettuare prenotazioni su video specifici da ritirare in una data specifica.
 - Deve rispondere alle richieste dei clienti, inclusa la possibilità di rispondere a domande sui film che il negozio non ha in magazzino, ma potrebbe ordinare su richiesta.
- **Caratteristiche del Sistema:**
 - Il sistema deve essere in grado di tenere traccia delle videocassette e dei dischi disponibili, nonché delle copie noleggate e delle prenotazioni.
 - Deve gestire gli account dei clienti e tenere traccia dei video noleggiati da ciascun cliente.
 - Deve gestire le prenotazioni dei video e assicurarsi che i clienti ricevano le copie prenotate in tempo.
- **Considerazioni Aggiuntive:**
 - Il sistema deve essere in grado di gestire la complessità delle prenotazioni, delle copie noleggate e delle restituzioni, considerando anche le scadenze.
 - Il sistema deve avere una funzionalità di ricerca e ricerca avanzata per cercare film specifici o ottenere informazioni su film non disponibili.
 - La gestione degli account dei clienti e il processo di noleggio devono essere user-friendly e intuitivi.
 - La gestione accurata delle informazioni sulle copie disponibili, i clienti e i video è essenziale per fornire un servizio efficiente e di qualità.

Nell'analisi orientata agli oggetti, questo caso di studio richiederebbe l'identificazione delle classi (come "Customer", "Video", "Reservation", ecc.) e delle loro proprietà, associazioni e comportamenti. I diagrammi UML, come i diagrammi dei casi d'uso, i diagrammi delle classi e i diagrammi di sequenza, possono essere utilizzati per rappresentare in modo chiaro ed esaustivo il sistema e le sue dinamiche.

Contact Management:

Questo caso di studio riguarda una società di ricerca di mercato che ha bisogno di un sistema per gestire i contatti con i clienti. Il sistema dovrebbe gestire i dati dei contatti dei clienti, tenere traccia delle comunicazioni passate e pianificare le attività future con i clienti. Si tratta di un'applicazione di gestione delle relazioni con i clienti (CRM).

- **Scenari:**
 - L'azienda di ricerca di mercato ha una base di clienti consolidata composta da organizzazioni che acquistano rapporti di analisi di mercato.
 - L'azienda è costantemente alla ricerca di nuovi clienti per espandere il proprio business.
- **Funzionalità del Sistema:**
 - Il sistema deve consentire la gestione dei contatti in tre categorie: prospect (clienti potenziali), actual (clienti attuali) e past (clienti passati).
 - Gli impiegati dell'azienda, in particolare del Dipartimento di Servizio Clienti, devono essere in grado di accedere al sistema.
 - Il sistema deve consentire la pianificazione flessibile e la ri-pianificazione delle attività legate ai contatti.
 - Deve permettere la collaborazione tra gli impiegati per acquisire nuovi clienti e mantenere le relazioni esistenti.
- **Caratteristiche del Sistema:**
 - Il sistema deve essere in grado di tenere traccia delle informazioni sui contatti e di archivarle in base alla loro categoria.
 - Deve gestire i dati dei clienti, inclusi i dettagli di contatto, le informazioni aziendali e lo storico delle interazioni.
 - Dovrebbe consentire agli impiegati di pianificare e registrare attività come chiamate, riunioni e follow-up per ciascun contatto.
 - Deve garantire la sicurezza e la gestione delle autorizzazioni per gli utenti con diversi livelli di accesso.
- **Considerazioni Aggiuntive:**
 - Il sistema dovrebbe offrire funzionalità di ricerca e filtro per individuare rapidamente i contatti desiderati.
 - La capacità di tracciare le interazioni passate con i clienti può aiutare a fornire un servizio personalizzato.
 - Gli strumenti di collaborazione, come la condivisione di note e informazioni, potrebbero migliorare l'efficienza delle attività di vendita e di gestione dei clienti.
 - La flessibilità nella pianificazione delle attività e la possibilità di aggiornarle in tempo reale sono fondamentali per adattarsi alle mutevoli esigenze aziendali.

Nell'ambito dell'analisi orientata agli oggetti, questo caso di studio richiederebbe l'identificazione delle classi (come "Contact", "Employee", "Activity", ecc.) e delle loro proprietà, associazioni e interazioni. I diagrammi UML, come i diagrammi dei casi d'uso, i diagrammi delle classi e i diagrammi di sequenza, potrebbero essere utilizzati per modellare il sistema in modo chiaro e completo.

Telemarketing:

In questo caso di studio, una società benefica sta cercando di raccogliere fondi attraverso la vendita di biglietti della lotteria. Il sistema deve supportare le attività di telemarketing, pianificando le chiamate in base alle priorità, registrando gli esiti delle conversazioni, gestendo le richieste di contributi e incentivando la partecipazione dei clienti attraverso campagne speciali.

- **Scenari:**
 - L'organizzazione benefica vende biglietti della lotteria per raccogliere fondi per cause importanti.
 - Le campagne vengono lanciate per sostenere cause benefiche attualmente rilevanti.
 - Vengono rivolte iniziative ai sostenitori passati per promuovere l'acquisto di biglietti o donazioni.
- **Funzionalità del Sistema:**
 - Il sistema deve consentire l'effettuazione di chiamate telefoniche per promuovere la vendita di biglietti della lotteria.
 - Dovrebbe supportare fino a cinquanta operatori di telemarketing che lavorano simultaneamente.
 - Le chiamate telefoniche devono essere pianificate in base a priorità predefinite e altre restrizioni conosciute.
 - Il sistema deve effettuare automaticamente le chiamate pianificate e gestire i risultati delle chiamate.
- **Caratteristiche del Sistema:**
 - Il sistema deve tracciare i dettagli delle chiamate effettuate, comprese le informazioni sugli operatori, i sostenitori chiamati e gli esiti delle chiamate.
 - Dovrebbe essere in grado di re-schedulare le chiamate non riuscite e pianificare ulteriori richiami telefonici per i sostenitori interessati.
 - Il sistema deve consentire agli operatori di registrare gli ordini dei biglietti e qualsiasi modifica ai dati dei sostenitori.
 - Dovrebbe supportare iniziative speciali di premi per l'acquisto in blocco di biglietti o per l'attrazione di nuovi sostenitori.
- **Considerazioni Aggiuntive:**
 - Il sistema dovrebbe rispettare le leggi e le regolamentazioni sulla privacy e sulle comunicazioni commerciali.

- La capacità di visualizzare un elenco dei sostenitori passati e le loro preferenze potrebbe aiutare a indirizzare le offerte in modo mirato.
- La possibilità di analizzare i dati delle chiamate e gli esiti potrebbe fornire informazioni utili per ottimizzare le strategie di telemarketing.
- L'integrazione con un sistema di gestione delle donazioni potrebbe semplificare la registrazione dei contributi dei sostenitori.

Nel contesto dell'analisi orientata agli oggetti, questo caso di studio richiederebbe l'identificazione delle classi (come "Supporter", "TicketOrder", "Telemarketer", ecc.) e delle loro proprietà, associazioni e interazioni. I diagrammi UML, come i diagrammi dei casi d'uso, i diagrammi delle classi e i diagrammi di sequenza, potrebbero essere utilizzati per modellare il sistema in modo chiaro e completo.

LINEE GUIDA PER LA SPECIFICA DELLE CLASSI

Nomi di classe:

- Assegnare a ciascuna classe un nome significativo che rifletta il suo ruolo nel dominio applicativo.
- Adottare una convenzione standard per i nomi delle classi, come ad esempio l'utilizzo di nomi singolari e parole multiple con l'iniziale di ciascuna parola in maiuscolo (notazione CamelCase).
- Limitare la lunghezza dei nomi delle classi a un massimo di 30 caratteri per mantenere la leggibilità.

Attributi e operazioni:

- Iniziare con l'identificazione degli attributi che sono rilevanti per caratterizzare gli stati di interesse degli oggetti all'interno del dominio applicativo.
- Utilizzare una convenzione standard per assegnare nomi agli attributi, come ad esempio utilizzare lettere minuscole e separare le parole con un carattere di underscore (snake_case).
- Ritardare l'aggiunta di operazioni alle classi fino a quando non è disponibile il modello comportamentale, dal quale le operazioni dovrebbero essere derivate in modo coerente.

IDENTIFICAZIONE DELLE ASSOCIAZIONI

- Gli **attributi** che rappresentano relazioni tra le classi devono essere identificati come associazioni. Ad esempio, se hai un attributo che rappresenta un oggetto di un'altra classe, dovrebbe essere modellato come un'associazione con quella classe.
- Le associazioni **ternarie** (coinvolgenti tre classi) dovrebbero essere convertite in cicli di associazioni binarie, in modo da evitare complicazioni di interpretazione. Questo significa che dovresti rappresentare le connessioni tra le tre classi come una serie di associazioni binarie.
- Nei **cicli** di associazioni, puoi valutare se alcune associazioni possono essere eliminate e gestite come associazioni derivate. Tuttavia, a volte per ragioni di efficienza, vengono introdotte associazioni ridondanti.

Specifica delle associazioni:

- Per assegnare **nomi** alle associazioni, utilizza la stessa convenzione adottata per gli attributi, cioè utilizza lettere minuscole e separa le parole con un carattere di underscore (snake_case).
- Assegna nomi di ruolo (**rolename**) alle estremità dell'associazione. Questi nomi di ruolo diventeranno i nomi degli attributi nelle classi all'estremità opposta dell'associazione. Questo aiuta a chiarire il significato e il ruolo delle associazioni.
- Determina la **molteplicità** delle associazioni. Questo indica quante istanze di una classe sono associate con un'altra classe attraverso un'associazione. Puoi utilizzare numeri o simboli come "*", "1", "0..1", "0..*", ecc., per rappresentare la molteplicità.

AGGREGAZIONE

L'aggregazione è un concetto chiave nei diagrammi delle classi di UML che rappresenta una relazione di tipo "**whole-part**" o di contenimento tra classi. Questo tipo di relazione descrive come un oggetto di una classe "**contenitore**" (o classe composta, detta anche superset class) è composto da uno o più oggetti di classi "**contenute**" (o classi componente, dette anche subset classes).

L'aggregazione può assumere quattro significati differenti:

- **ExclusiveOwns**: Questo significato sottolinea una dipendenza di esistenza tra la classe contenitore e le classi contenute. In altre parole, se l'oggetto della classe contenitore cessa di esistere, anche gli oggetti delle classi contenute cesseranno di esistere. Questo tipo di aggregazione è anche transitivo (se un oggetto della classe contenuta è parte di un altro oggetto, allora è parte anche dell'oggetto contenitore originale). È importante notare che l'aggregazione in UML non è simmetrica; cioè l'oggetto contenuto non ha una conoscenza diretta dell'oggetto contenitore.

- **Owns:** Questo significato indica che l'oggetto della classe contenitore possiede gli oggetti della classe contenuta. Non esiste una dipendenza di esistenza tra le due classi, ma l'oggetto contenitore è responsabile della creazione e distruzione degli oggetti contenuti. In questo caso, non c'è una relazione di proprietà fissa tra le classi.
- **Has:** In questo caso, l'aggregazione rappresenta il fatto che l'oggetto della classe contenitore "ha" o "contiene" gli oggetti delle classi contenute. Non c'è dipendenza di esistenza e né una relazione di proprietà fissa.
- **Member:** Questo significato rappresenta una relazione di appartenenza. Ad esempio, se una riunione ha un presidente, allora il presidente è membro della riunione. Non c'è una particolare proprietà fissa associata a questa relazione, eccetto l'appartenenza stessa.

Specifica di Aggregazione in UML:

La specifica di aggregazione in UML include due tipi distinti di aggregazione: l'aggregazione e la composizione. Questi due tipi riflettono le diverse semantiche e i significati delle relazioni di aggregazione tra le classi.

- **Aggregazione:**
 - **Semantica:** Aggregazione con riferimento (by-reference semantics) – L'aggregazione rappresenta una relazione meno stretta tra la classe contenitore e quella contenuta. In questo tipo di relazione, l'oggetto contenitore fa riferimento all'oggetto contenuto, ma non ne possiede la responsabilità diretta per il ciclo di vita. In altre parole, l'oggetto contenuto può esistere indipendentemente dalla classe contenitore.
 - **Notazione:** Un diamante vuoto è posto sulla linea che collega la classe contenitore alla classe contenuta.
 - Corrisponde alle aggregazioni "Has" e "Member".
 - **Esempi:** Un'automobile "ha" un motore o un dipartimento "ha" dipendenti. Questa relazione riflette l'idea che gli oggetti contenuti possono esistere indipendentemente dalla classe contenitore.
- **Composizione:**
 - **Semantica:** Aggregazione con valore (by-value semantics) – La composizione rappresenta una relazione più stretta tra la classe contenitore e quella contenuta. In questo caso, l'oggetto contenitore possiede direttamente l'oggetto contenuto e ne gestisce il ciclo di vita. L'oggetto contenuto esiste solo all'interno dell'oggetto contenitore e non può esistere al di fuori di esso.
 - **Notazione:** Un diamante solido è posto sulla linea che collega la classe contenitore alla classe contenuta.
 - Corrisponde alle aggregazioni "ExclusiveOwns" e "Owns".
 - **Esempi:** Un computer "possiede" una CPU, oppure una casa "possiede" le sue stanze. In questo caso, l'oggetto contenuto è strettamente legato all'oggetto contenitore e non può esistere autonomamente.

Questi due tipi di aggregazione riflettono la profondità e la natura delle relazioni tra le classi. L'aggregazione permette una relazione più debole tra la classe contenitore e quella contenuta, mentre la composizione implica una relazione più forte, in cui l'oggetto contenitore possiede direttamente e gestisce gli oggetti contenuti.

EREDITARIETA' (GENERALIZZAZIONE)

L'ereditarietà, anche chiamata generalizzazione in UML, è un concetto chiave nella programmazione orientata agli oggetti che consente di modellare la condivisione di attributi e operazioni tra diverse classi.

- **Superclasse e Sottoclassi:** L'ereditarietà coinvolge una relazione tra una classe più generica chiamata superclasse (o classe padre) e una o più classi specializzate chiamate sottoclassi (o classi figlie). La superclasse contiene le caratteristiche comuni che verranno ereditate dalle sottoclassi.
- **Ereditare Attributi e Operazioni:** Una sottoclasse eredita gli attributi e le operazioni definite nella superclasse. Ciò significa che le sottoclassi possono utilizzare e avere accesso alle stesse proprietà e comportamenti della superclasse.
- **Sostituibilità:** Un concetto importante nell'ereditarietà è la sostituibilità. Un oggetto istanziato da una sottoclasse può essere utilizzato in qualsiasi contesto in cui è richiesto un oggetto della superclasse.
- **Polimorfismo:** L'ereditarietà supporta il polimorfismo, che consente alle sottoclassi di fornire implementazioni specifiche per le operazioni ereditate dalla superclasse. Questo significa che la stessa operazione può essere implementata in modo diverso da diverse sottoclassi, ma può essere richiamata usando la stessa interfaccia.
- **Overriding:** Le sottoclassi possono sovrascrivere (override) le implementazioni ereditate di operazioni dalla superclasse. Questo consente alle sottoclassi di personalizzare il comportamento delle operazioni in base alle loro esigenze.
- **Specializzazione:** Le sottoclassi rappresentano una specializzazione o una raffinazione della superclasse. Possono aggiungere attributi e operazioni aggiuntive o sovrascrivere quelle ereditate.

Specifica di Ereditarietà in UML:

Nell'UML, l'ereditarietà viene rappresentata con una relazione di tipo "is-a". Questo indica che una classe (sottoclasse) è una specializzazione di un'altra classe (superclasse). L'associazione "is-a" viene spesso chiamata "is-a-kind-of" per evidenziare la relazione di specializzazione. Ecco come viene rappresentata l'ereditarietà in UML:

- Una freccia diretta collega la sottoclasse alla superclasse.

- La freccia punta dalla sottoclasse alla superclasse.
- La sottoclasse è posta all'estremità della freccia.
- La freccia rappresenta la relazione di ereditarietà tra le classi.

OBJECT DIAGRAM

Un diagramma degli oggetti (object diagram) è una rappresentazione grafica delle **istanze** (oggetti) di classi definite nel modello di dati. Questo tipo di diagramma viene utilizzato per mostrare le relazioni e le interazioni tra le istanze di diverse classi, offrendo una visione concreta del sistema in un dato momento. Gli object diagram sono utili per diversi scopi:

- **Rappresentare relazioni complesse tra classi:** Gli object diagram sono utilizzati per mostrare come diverse istanze di classi interagiscono tra loro. Questo può essere utile per comprenderne il funzionamento in situazioni specifiche.
- **Illustrare modifiche alle istanze durante l'evoluzione del sistema:** Gli object diagram consentono di rappresentare come gli oggetti cambiano nel tempo, man mano che il sistema si evolve. Ciò può aiutare a identificare cambiamenti nel comportamento o nelle relazioni degli oggetti.
- **Mostrare la collaborazione tra oggetti:** Gli object diagram possono essere usati per illustrare come le diverse istanze di classi lavorano insieme per raggiungere un obiettivo specifico. Questo può contribuire a una comprensione più chiara dei flussi di lavoro e delle interazioni tra gli oggetti.

In sostanza, gli object diagram offrono una visione dettagliata e concreta del sistema in termini delle sue istanze attive. Mostrano come le classi e le loro istanze interagiscono e collaborano per svolgere determinate attività o funzioni.

MODELLO COMPORTAMENTALE

Il modello comportamentale rappresenta l'aspetto **funzionale** del sistema, descrivendo come gli oggetti collaborano e interagiscono tra loro per fornire i servizi offerti dal sistema. Questo modello è composto da vari formalismi che aiutano a catturare diversi aspetti delle dinamiche del sistema. Alcuni dei principali formalismi utilizzati nel modello comportamentale sono:

- **Use Case Diagram:** Questo diagramma rappresenta gli scenari di utilizzo del sistema, mostrando le interazioni tra attori (utenti o altri sistemi) e i casi d'uso (funzionalità) che il sistema offre. Aiuta a identificare le principali funzionalità del sistema e le interazioni tra gli attori e le funzionalità.
- **Activity Diagram:** Questo diagramma descrive il flusso di elaborazione all'interno di un caso d'uso o di una funzionalità. Rappresenta le attività, le decisioni e le interazioni tra le attività. È particolarmente utile per modellare i processi aziendali, i flussi di lavoro e le sequenze di azioni.
- **Sequence Diagram:** Questo diagramma visualizza l'interazione tra gli oggetti nel tempo, mostrando l'ordine dei messaggi scambiati tra gli oggetti. È particolarmente utile per rappresentare scenari di interazione e per mostrare come gli oggetti collaborano per raggiungere uno specifico obiettivo.
- **Collaboration Diagram:** Questo diagramma rappresenta l'interazione tra gli oggetti, mostrando i messaggi scambiati tra di essi e le associazioni tra le istanze delle classi. È utile per mostrare le relazioni e le interazioni complesse tra oggetti diversi.

Il modello comportamentale viene sviluppato in modo **iterativo** ed **incrementale**, lavorando in **parallelo** con il modello dei dati. Le informazioni dal modello dei dati vengono utilizzate per identificare le operazioni e le classi aggiuntive necessarie per il sistema, come le control classes (classi che gestiscono la logica del sistema) e le boundary classes (classi che rappresentano l'interfaccia utente).

USE CASE DIAGRAM

Un diagramma dei casi d'uso è uno strumento di modellazione utile nella fase di analisi dei requisiti. Esso aiuta a catturare le funzionalità principali del sistema e le interazioni con gli attori (utenti o altri sistemi esterni).

- **Livelli di Astrazione:** I diagrammi dei casi d'uso possono essere sviluppati a vari livelli di dettaglio, da una panoramica ad alto livello fino ai dettagli più specifici delle funzionalità. Questi livelli di astrazione aiutano a rappresentare sia le viste generali del sistema che le dettagliate interazioni.
- **Rappresentazione di Funzionalità Complete:** Ogni caso d'uso rappresenta una funzionalità completa del sistema, inclusi il flusso principale, eventuali sottoflussi alternativi e le eccezioni gestite. Questo fornisce una visione completa del comportamento del sistema in risposta alle azioni degli attori.
- **Visibilità Esterna:** I casi d'uso rappresentano le funzionalità visibili dall'esterno del sistema. In altre parole, mostrano come gli attori interagiscono con il sistema per ottenere risultati specifici.
- **Comportamento Ortogonale:** Ciascun caso d'uso è progettato per essere eseguito in modo indipendente dagli altri. Questo significa che le interazioni all'interno di un caso d'uso non interferiscono con gli altri casi d'uso. Ciò semplifica la modellazione e l'analisi dei requisiti.

- **Origine da Attori:** Ogni caso d'uso ha un attore iniziale che ne è la fonte. Questo attore può essere un utente umano o un altro sistema esterno. Una volta che il caso d'uso è stato avviato, può interagire con altri attori e oggetti all'interno del sistema.
- **Risultato Significativo per Attori:** Ogni caso d'uso deve produrre un risultato significativo per l'attore che lo inizia. Questo significa che deve portare a un risultato utile, come ad esempio un servizio fornito all'utente.

Identificazione dei Casi d'Uso:

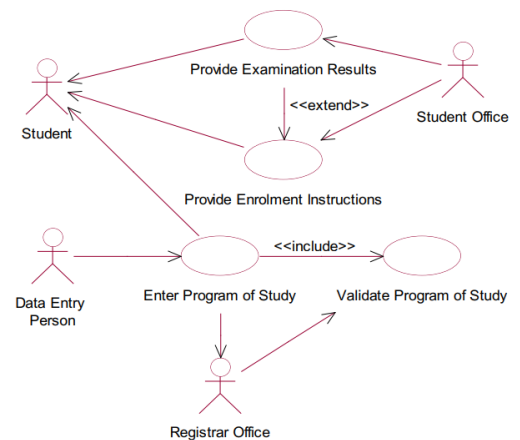
L'identificazione dei casi d'uso è una parte cruciale della fase di analisi dei requisiti e aiuta a definire le principali funzionalità e interazioni del sistema.

- **Insieme dei Requisiti Utente:** L'insieme dei casi d'uso nasce spesso dalla comprensione dei requisiti utente. Ogni caso d'uso rappresenta un modo specifico in cui gli utenti interagiscono con il sistema per soddisfare un certo obiettivo.
- **Attori e Obiettivi:** Gli attori rappresentano le diverse categorie di utenti o sistemi esterni che interagiscono con il sistema. Identificare gli obiettivi degli attori aiuta a stabilire quali compiti devono essere supportati dal sistema.
- **Domande Guida:** L'identificazione può essere facilitata facendosi guidare dalle seguenti domande:
 - Quali sono i compiti principali svolti da ciascun attore?
 - Un attore accede o modifica l'informazione nel sistema?
 - L'attore rappresenta il tramite mediante cui il sistema viene informato di modifiche apportate in altri sistemi?
 - L'attore deve essere informato di eventuali cambiamenti avvenuti nel sistema?
- **Necessità degli Attori:** Durante la fase di OOA, è fondamentale identificare le necessità degli attori e come queste si traducono in comportamenti specifici del sistema. Questo aiuta a creare una rappresentazione accurata delle interazioni utente-sistema.

Specifiche di Use Case Diagram:

Si possono rappresentare quattro tipi di relazioni:

- **Associazione:** Rappresenta l'interazione tra un attore e un caso d'uso. Questo tipo di relazione indica che un attore è coinvolto nell'esecuzione di un caso d'uso specifico.
 - Ad esempio, un attore "Studente" potrebbe essere associato al caso d'uso "Registrare a un corso".
- **Include:** Questa relazione indica che un caso d'uso (chiamato caso d'uso incluso) è sempre necessario per completare un altro caso d'uso (chiamato caso d'uso di base). Il caso d'uso incluso rappresenta una sequenza di azioni comuni a più casi d'uso.
 - Ad esempio, un caso d'uso "Eseguire pagamento" potrebbe includere il caso d'uso "Verificare saldo".
- **Extend:** Questa relazione indica che un caso d'uso (chiamato caso d'uso esteso) può aggiungere comportamenti opzionali a un altro caso d'uso (chiamato caso d'uso estendente). Il caso d'uso esteso rappresenta una sequenza di azioni che si verificano solo in determinate condizioni.
 - Ad esempio, un caso d'uso "Prenotare camera d'albergo" potrebbe estendersi per includere un caso d'uso "Applicare sconto" se il cliente ha un codice sconto.
- **Generalizzazione:** Questa relazione è simile al concetto di ereditarietà nelle classi. Rappresenta una relazione di tipo "è-un" tra un caso d'uso più generico (super-caso d'uso) e uno o più casi d'uso più specifici (sotto-casi d'uso).
 - Ad esempio, un caso d'uso generico "Gestire utente" potrebbe essere specializzato in casi d'uso come "Aggiungere utente" e "Eliminare utente".

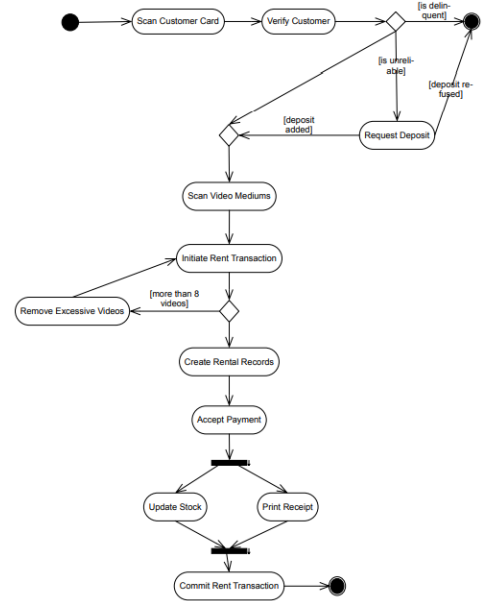


ACTIVITY DIAGRAM

L'activity diagram è un potente strumento per rappresentare il **flusso di esecuzione** delle attività all'interno di un sistema software. È utilizzato sia nella fase di OOA che di OOD, consentendo di rappresentare sia il flusso di attività dei casi d'uso durante la fase di analisi che l'implementazione delle operazioni definite nel class diagram durante la fase di progettazione.

Specifica di Activity Diagram:

- **Evento:** Rappresenta l'innesco di un'attività e può essere esterno (come un input dall'utente) o interno (come il completamento di un'azione). Gli eventi esterni spesso corrispondono all'origine di un caso d'uso.
 - Ad esempio, "Clicca sul pulsante 'Invia'" potrebbe essere un evento esterno che avvia l'attività di invio di un'e-mail.
- **Action State:** Rappresenta un'azione da eseguire. Può essere un'azione elementare (come "Invia email") o un'azione composta (come un'azione più dettagliata che può essere decomposta ulteriormente).
 - Ad esempio, "Invia email" potrebbe essere un "action state" che rappresenta l'azione di invio di un'e-mail all'interno del flusso di attività.
- **Transition:** Rappresenta il passaggio da uno stato all'altro, causato dal completamento di un'azione. Può essere condizionato da una guard condition, che determina se la transizione è attivata.
 - Sono frecce che collegano lo stato di partenza allo stato di destinazione
- **Barra di Sincronizzazione (Fork/Join):** Utilizzata per modellare flussi di esecuzione concorrenti. La barra di fork rappresenta la divisione dei flussi concorrenti, mentre la barra di join rappresenta il punto in cui i flussi concorrenti si riuniscono.
 - Viene rappresentata da una linea spezzata che si divide in più frecce/con frecce che si riuniscono.
- **Nodo Decisionale (Branch/Merge):** Utilizzato per modellare flussi alternativi. Il nodo decisionale rappresenta una scelta tra diverse alternative, mentre il nodo di merge indica il punto in cui i flussi alternativi si ricongiungono.
 - È rappresentato da un rombo e contiene le opzioni decisionali, ciascuna collegata da una freccia etichettata con una condizione.

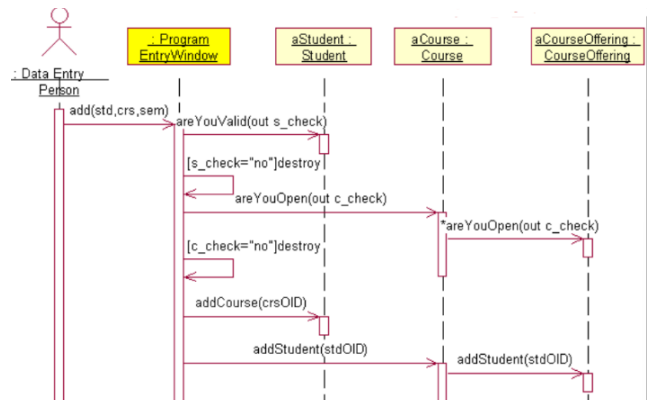


DIAGRAMMI DI INTERAZIONE

I diagrammi di interazione, che includono i **sequence diagram** e i **collaboration diagram**, sono strumenti essenziali per rappresentare le interazioni tra gli oggetti all'interno di un sistema.

Sequence Diagram:

- **Scambio di Messaggi:** Il sequence diagram rappresenta lo scambio di messaggi tra gli oggetti nel tempo. Gli oggetti sono disposti in ordine orizzontale lungo l'asse verticale, mentre le frecce orizzontali rappresentano i messaggi scambiati tra gli oggetti.
- **Ordine Temporale:** La disposizione degli oggetti e delle frecce segue l'ordine temporale in cui avviene lo scambio dei messaggi. Gli oggetti chiamanti inviano messaggi agli oggetti chiamati, e il diagramma mostra chiaramente l'ordine delle chiamate.
- **Attivazioni:** Le barre verticali chiamate "attivazioni" rappresentano il periodo di tempo in cui un oggetto è impegnato nell'esecuzione di un'azione o nella gestione di un messaggio. L'altezza dell'attivazione può indicare la durata dell'azione.



Collaboration Diagram:

- **Scambio di Messaggi e Relazioni:** Il collaboration diagram mostra le interazioni tra gli oggetti attraverso relazioni e messaggi. Gli oggetti sono rappresentati da "blocchi" e le frecce indicano i messaggi scambiati tra gli oggetti.
- **Relazioni tra Oggetti:** Le relazioni tra gli oggetti sono rappresentate da linee tratteggiate o continue tra i blocchi. Queste relazioni possono essere associazioni, aggregazioni o altre relazioni definite nel class diagram.

Equivalenza tra Sequence e Collaboration Diagram:

I sequence diagram e i collaboration diagram sono due rappresentazioni diverse ma equivalenti delle interazioni tra oggetti. È possibile trasformare un sequence diagram in un collaboration diagram e viceversa in modo coerente, poiché entrambi rappresentano le stesse informazioni relative alle interazioni.

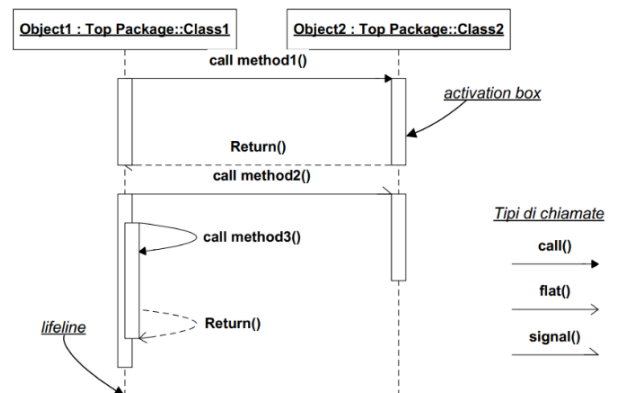
Specifica di Sequence Diagram:

Le attività dell'activity diagram vengono mappate come **messaggi** (di tipo "richiesta esecuzione attività") in un sequence diagram. Esistono due tipi principali di messaggi:

- **Signal:**
 - I messaggi di tipo "signal" rappresentano chiamate **asincrone** tra oggetti. In una chiamata asincrona, l'oggetto mittente invia un messaggio di "signal" all'oggetto destinatario, ma non attende una risposta immediata.
 - Dopo aver inviato un messaggio di "signal", l'oggetto mittente può continuare la sua esecuzione senza aspettare una risposta dall'oggetto destinatario.
 - I messaggi di "signal" sono spesso utilizzati per rappresentare eventi che scatenano attività o reazioni nell'oggetto destinatario. Ad esempio, una notifica di avvenuta registrazione potrebbe essere inviata come un messaggio di "signal".
- **Call:**
 - I messaggi di tipo "call" rappresentano chiamate **sincrone** tra oggetti. In una chiamata sincrona, l'oggetto mittente invia un messaggio di "call" all'oggetto destinatario e attende una risposta.
 - L'oggetto mittente blocca la sua esecuzione fino a quando non riceve una risposta dall'oggetto destinatario. La risposta può includere valori di ritorno o altri dettagli rilevanti.
 - I messaggi di "call" sono utilizzati quando l'oggetto mittente ha bisogno di ottenere una risposta immediata dall'oggetto destinatario per procedere con l'esecuzione.

Notazione per Sequence Diagram:

- **Partecipanti (Oggetti):** Gli oggetti coinvolti nell'interazione sono rappresentati da rettangoli verticali (chiamati "lifelines"). Il nome dell'oggetto è scritto sopra o all'interno del rettangolo, seguito da un separatore e dal nome della classe tra parentesi, ad esempio: "Oggetto : Classe".
- **Messaggi:** I messaggi scambiati tra gli oggetti sono rappresentati da frecce tra le lifelines. Le frecce possono essere dirette o tratteggiate. Le frecce dirette indicano i messaggi sincroni, mentre le frecce tratteggiate indicano i messaggi asincroni. L'etichetta del messaggio, che rappresenta il nome del messaggio e, opzionalmente, gli argomenti o i valori passati, è posizionata sopra la freccia.
- **Attivazione dell'oggetto:** L'attivazione dell'oggetto viene rappresentata da una barra verticale, chiamata "barra di attivazione", che si estende sulla lifeline dell'oggetto. Indica il periodo durante il quale l'oggetto è attivo e sta eseguendo le operazioni.
- **Messaggi di ritorno:** I messaggi di ritorno, che rappresentano le risposte agli altri messaggi, sono rappresentati da frecce con un'etichetta che indica il valore di ritorno o la risposta.



INTERFACCIA PUBBLICA DI CLASSE

Durante la fase di **analisi (OOA)**, è fondamentale definire con attenzione l'interfaccia pubblica di ciascuna classe. L'interfaccia pubblica di una classe definisce le **operazioni** che la classe rende disponibili per essere utilizzate da altre classi.

La **signature** di un'operazione include:

- **Nome dell'Operazione:** Il nome che identifica l'operazione.
- **Lista degli Argomenti Formali:** I parametri di input richiesti dall'operazione, ciascuno con il proprio tipo e nome.
- **Tipo di Ritorno:** Il tipo di dato che l'operazione restituirà come risultato.

Implementazione dell'Operazione:

Durante la fase di progettazione (OOD), si procede alla definizione dell'implementazione concreta dell'operazione. Questo include la scrittura del codice che definisce cosa fa effettivamente l'operazione quando viene eseguita. L'implementazione dettagliata può includere algoritmi, logica e interazioni con altri oggetti del sistema.

Scope delle Operazioni:

Le operazioni possono avere diverse "scope" in base a come agiscono all'interno della classe:

- **Instance Scope:** Le operazioni di istanza operano su un oggetto specifico (istanza) della classe e possono accedere ai suoi attributi di istanza.
- **Class (Static) Scope:** Le operazioni di classe agiscono a livello di classe, piuttosto che su istanze specifiche, e possono accedere solo agli attributi statici della classe.

Carattere \$ per le Operazioni di Classe (Static):

Nella notazione UML (Unified Modeling Language), un carattere "\$" che precede il nome di un'operazione indica che quell'operazione agisce a livello di classe (statico). Questo è particolarmente utile per distinguere tra operazioni di istanza e operazioni di classe.

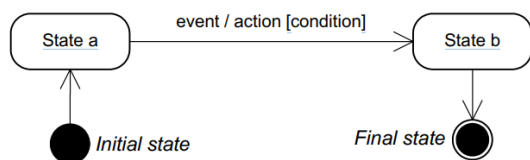
IDENTIFICAZIONE DELLE OPERAZIONI

L'identificazione delle operazioni all'interno di un sistema software può avvenire in diversi modi, tra cui l'analisi dei **sequence diagram** e l'applicazione di **criteri aggiuntivi** come il criterio **CRUD**. Questi approcci possono aiutare a definire le operazioni principali che le classi del sistema devono supportare. Ecco come funzionano entrambi i metodi:

- **Identificazione dalle Sequence Diagram:** Nei sequence diagram, i messaggi inviati tra gli oggetti rappresentano le chiamate di metodi tra le classi. Analizzando i sequence diagram, si possono individuare quali operazioni vengono eseguite dalle classi in diversi scenari di interazione. Questo metodo è particolarmente utile per identificare le operazioni che sono coinvolte nei casi d'uso specifici.
- **Criterio CRUD:** Il criterio CRUD è un'acronimo che sta per Create, Read, Update e Delete. Questo criterio definisce le operazioni fondamentali che un sistema deve supportare per gestire le informazioni. Ogni classe di dati dovrebbe supportare queste quattro operazioni:
 - **Create** (Creazione): Consente di creare una nuova istanza dell'oggetto.
 - **Read** (Lettura): Consente di ottenere informazioni sullo stato dell'oggetto.
 - **Update** (Aggiornamento): Consente di modificare lo stato dell'oggetto esistente.
 - **Delete** (Eliminazione): Consente di rimuovere un'istanza dell'oggetto dal sistema.

MODELLO DINAMICO

Il modello dinamico si concentra sul comportamento dinamico degli oggetti all'interno di una classe nel corso del tempo. In altre parole, rappresenta come gli oggetti di una classe interagiscono e cambiano stato in risposta agli eventi e alle azioni. Questo modello è utile per comprendere come il sistema si comporta nel suo flusso operativo e come gli oggetti reagiscono agli stimoli esterni o agli eventi interni.



State Diagrams:

I "State Diagrams" (Diagrammi di Stato) sono uno strumento comune utilizzato per rappresentare il modello dinamico. Un diagramma di stato è costituito da una serie di stati, eventi e transizioni. Gli **stati** rappresentano i diversi stati in cui un oggetto può trovarsi, mentre le **transizioni** collegano gli

stati e vengono scatenate da eventi specifici. Un diagramma di stato include anche le **azioni** associate agli eventi e le **condizioni** che devono essere soddisfatte per attivare una transizione. Le condizioni possono determinare se una transizione è possibile in base allo stato attuale dell'oggetto.

Applicazioni del Modello Dinamico:

Il modello dinamico è particolarmente utile per le applicazioni **scientifiche** e **real-time**, in cui è importante comprendere come gli oggetti cambiano stato nel corso del tempo.

In sintesi, il modello dinamico e i diagrammi di stato offrono una visione chiara e dettagliata di come gli oggetti all'interno di una classe interagiscono e cambiano nel tempo, consentendo agli sviluppatori di comprendere meglio il comportamento del sistema e di prendere decisioni informate nella progettazione e nell'implementazione.

GESTIONE DELLA COMPLESSITA' NEI MODELLI DI OOA

La gestione della complessità è un aspetto cruciale nello sviluppo di sistemi software di grandi dimensioni. I modelli di analisi orientata agli oggetti (OOA) spesso coinvolgono numerose classi con relazioni interconnesse, e questa complessità può rendere difficile la comprensione, la manutenzione e l'evoluzione del sistema. L'introduzione di **gerarchie di classi** e la **stratificazione dei modelli** sono tecniche che possono aiutare a affrontare questa complessità.

Gerarchie di Classi:

Le gerarchie di classi consentono di organizzare le classi in modo più strutturato, raggruppandole secondo relazioni di ereditarietà. Introducendo classi di base (superclassi) e classi derivate (sottoclassi), è possibile catturare le caratteristiche comuni e differenziali tra le classi. Questo permette di evitare la duplicazione del codice e semplifica il modello, rendendolo più gestibile. Le gerarchie di classi possono anche favorire la modularità e la riusabilità del codice. Le classi di base possono implementare comportamenti di alto livello, mentre le classi derivate possono specializzarsi e aggiungere dettagli specifici. Inoltre, le operazioni definite nella classe di base possono essere ereditate e sovrascritte nelle classi derivate.

Stratificazione dei Modelli:

La stratificazione dei modelli implica la suddivisione del sistema in livelli o strati logici. Ogni strato ha uno scopo specifico e una responsabilità chiara. Questo approccio permette di isolare le funzionalità e ridurre le interdipendenze tra le diverse parti del sistema. La stratificazione aiuta a creare una struttura modulare e coesa, in cui ogni strato è **indipendente** dagli altri e può essere modificato senza influenzare gli altri. Inoltre, questa suddivisione semplifica la manutenzione, in quanto è più facile individuare e risolvere i problemi in un determinato strato senza dover toccare tutto il sistema.

UML PACKAGE

Un **package** è un meccanismo utilizzato nell'Unified Modeling Language (UML) per organizzare e raggruppare elementi come classi, casi d'uso, componenti e altri elementi correlati in un sistema software.

- **Annidamento dei Package:** I package possono essere annidati all'interno di altri package, creando una struttura gerarchica. Questo permette di organizzare ulteriormente gli elementi in sottogruppi e di creare una gerarchia di organizzazione.
- **Visibilità e Comunicazione:** Le classi all'interno di un package possono comunicare tra loro usando relazioni di dipendenza, associazioni e altre interazioni. La visibilità dei membri delle classi (ad esempio attributi e operazioni) può essere controllata mediante i livelli di visibilità (public, protected, private) definiti nel linguaggio di programmazione o nel paradigma di sviluppo.
- **Gestione della Complessità:** L'uso di package aiuta a gestire la complessità di grandi sistemi software, suddividendo il sistema in unità più gestibili e facilitando la navigazione all'interno del modello. Inoltre, i package possono riflettere la struttura organizzativa dell'applicazione o i domini di interesse specifici.
- **Collaborazione tra Package:** I package possono comunicare tra loro attraverso dipendenze di package o attraverso l'uso di classi di altri package. Questo consente la separazione delle preoccupazioni e favorisce l'incapsulamento e la modularità.

Package Diagram:

I package possono essere creati all'interno di diversi tipi di diagrammi, come il class diagram o il use case diagram. I package possono contenere classi, interfacce, casi d'uso e altri elementi del modello.

Si possono specificare due tipi principali di relazioni tra i package:

- **Generalization (Gerarchia):** Questa relazione indica che un package è una specializzazione di un altro package. Implica una relazione di ereditarietà tra i contenuti dei package. Quando un package eredita elementi da un altro package, la gerarchia tra i package è spesso accompagnata da una dipendenza tra i contenuti dei package.
- **Dependency (Dipendenza):** Questa relazione indica che un package dipende da un altro per qualche motivo. Le dipendenze possono includere l'utilizzo di elementi del package di destinazione o l'accesso ai suoi membri. Le relazioni di dipendenza tra package possono includere usage dependency, access dependency e visibility dependency.

APPROCCIO BCE

L'approccio BCE (Boundary-Control-Entity) è un modo di organizzare il **design** e la **struttura** di un sistema software in base a tre categorie principali di classi che collaborano per fornire funzionalità complessive. Questo approccio mira a suddividere le responsabilità all'interno del sistema in modo da ottenere un design ben strutturato e modulare. Le tre categorie di classi nell'approccio BCE sono:

- **Boundary Package:** Questa componente rappresenta l'interfaccia utente dell'applicazione. Include tutti gli elementi che interagiscono direttamente con l'utente, come le schermate, i form, i widget e i controlli grafici. Il Boundary si occupa di ricevere le interazioni dell'utente, visualizzare i dati e fornire feedback visivo. Non contiene logica di business o elaborazione dei dati. Lo scopo principale del Boundary è quello di trasmettere le richieste dell'utente al Controller e visualizzare i dati ottenuti dall'Entity.
- **Control Package:** Le classi del package Control rappresentano il "cervello" del sistema. Questa componente gestisce la logica di controllo dell'applicazione. Riceve i comandi e le azioni provenienti dalla Boundary e inoltra le richieste appropriate all'Entity per elaborazione. Il Controller contiene la logica di business, le regole di validazione e le azioni da intraprendere in risposta alle richieste dell'utente. Il Controller è responsabile di gestire l'interazione tra la Boundary e l'Entity. Le classi Control possono essere considerate come le azioni e le attività che gestiscono i casi d'uso.

- **Entity Package:** Le classi del package Entity contengono le strutture dati, le regole di business e l'accesso ai dati. L'Entity elabora le richieste provenienti dal Controller, interagisce con il database o altre fonti di dati, e restituisce i risultati al Controller. L'Entity è separata dalla Boundary per garantire che la logica di business e l'accesso ai dati siano isolati dall'interfaccia utente.

PIANIFICAZIONE

GESTIONE DI PROGETTI SOFTWARE

La gestione di progetti software è un'attività fondamentale nel processo di sviluppo di software. Lo sviluppo di un prodotto software è un'operazione complessa che coinvolge molte persone, processi e risorse. La gestione di un progetto software implica la pianificazione, il monitoraggio e il controllo di tutte queste componenti durante l'intero ciclo di vita del progetto, dall'inizio alla consegna del software finito.

Il Software Project Management Plan (SPMP) è un documento chiave in questo contesto. Esso guida la gestione del progetto software, definendo obiettivi, vincoli, risorse disponibili, pianificazione delle attività e le responsabilità dei membri del team.

Le quattro "P" nella gestione di progetti software:

La gestione efficace di un progetto software si basa su quattro aspetti chiave, spesso chiamati "quattro P":

- **Persone:** Le persone sono l'elemento più importante in un progetto software di successo. La gestione delle risorse umane è fondamentale per assegnare ruoli e responsabilità ai membri del team, garantendo che abbiano le competenze necessarie e motivandoli per ottenere risultati di alta qualità. La gestione delle persone è spesso basata su modelli come il "People Management - Capability Maturity Model".
- **Prodotto:** Questo riguarda l'identificazione delle caratteristiche del software da sviluppare. Questo include definire obiettivi, specifiche di dati, funzionalità, comportamenti principali, alternative e vincoli tecnici o di progetto.
- **Processo:** Questo riguarda la definizione del quadro di riferimento all'interno del quale viene pianificato e sviluppato il prodotto software. In altre parole, si tratta di stabilire il metodo e le pratiche di sviluppo che saranno utilizzate per raggiungere gli obiettivi del progetto.
- **Progetto:** Questo riguarda l'organizzazione e la pianificazione delle attività specifiche necessarie per sviluppare il software. Questo include l'allocazione delle risorse, la definizione dei compiti, la stima dei tempi e dei costi, e il monitoraggio costante del progresso.

L'**organizzazione del team** è un aspetto critico della gestione di progetti software. È importante assegnare il lavoro in modo efficiente, tenendo conto delle capacità e delle interazioni tra i membri del team. L'obiettivo è ottenere il miglior risultato possibile in termini di qualità e tempistiche, evitando ritardi dovuti a eccessive interazioni o alla mancanza di competenze specifiche. La legge di Brooks sottolinea che l'aggiunta di più risorse in un progetto non sempre accelererà lo sviluppo e potrebbe persino ritardarlo a causa delle complessità aggiuntive.

Pianificazione di progetti software:

La pianificazione dei progetti software è un processo chiave nell'ambito della gestione dei progetti software. Il suo obiettivo è fornire un quadro di riferimento che consenta di controllare, determinare l'avanzamento e monitorare lo sviluppo del progetto software. Questo processo è motivato dalla necessità di sviluppare prodotti software nei tempi e nei costi previsti, rispettando al contempo i requisiti di qualità desiderati. Le componenti fondamentali della pianificazione dei progetti software includono:

- **Scoping (raggio d'azione):** Questo step consiste nell'acquisire una comprensione completa del problema da risolvere e del lavoro da svolgere. È fondamentale definire chiaramente l'ambito del progetto per evitare cambiamenti e allargamenti eccessivi durante lo sviluppo.
- **Stime:** Durante questa fase, vengono previsti tempi, costi e sforzi necessari per completare il progetto. Queste stime servono come base per la pianificazione delle risorse e per valutare la fattibilità del progetto.
- **Rischi:** È importante identificare i rischi potenziali associati al progetto software e definire strategie per gestirli. La gestione dei rischi aiuta a prevenire problemi imprevisti che potrebbero causare ritardi o costi aggiuntivi.
- **Schedule:** Questa parte della pianificazione coinvolge l'allocazione delle risorse disponibili e la definizione dei punti di controllo temporali nel progetto. La creazione di una timeline chiara aiuta a monitorare l'andamento del progetto.
- **Strategia di controllo:** Qui si stabilisce un quadro di riferimento per il controllo della qualità e il controllo dei cambiamenti nel corso del progetto. È importante garantire che il prodotto software mantenga gli standard di qualità desiderati e che i cambiamenti vengano gestiti in modo controllato per evitare impatti negativi sul progetto.

STIME NEI PROGETTI SOFTWARE

Le stime nei progetti software sono un aspetto fondamentale della pianificazione e della gestione dei progetti. L'obiettivo principale delle stime di tempi, costi ed effort è ridurre al minimo l'incertezza e limitare i rischi associati a una stima errata. Ci sono diverse tecniche utilizzate per migliorare l'affidabilità e l'accuratezza delle stime nei progetti software:

- **Stime basate su progetti simili (Expert Judgment by Analogy):** Questa tecnica si basa sull'esperienza di esperti nel settore. Gli esperti confrontano il progetto attuale con progetti simili già completati in passato e utilizzano queste analogie per effettuare stime. Questo metodo è utile quando ci sono progetti precedenti con caratteristiche simili che possono servire da punto di riferimento.

- **Tecniche di scomposizione (Approccio bottom-up):** Queste tecniche coinvolgono la suddivisione del progetto in componenti più piccole, come task o funzioni, e la stima di allocazione dell'effort per ciascuna componente. Le stime possono basarsi su metriche come il numero di righe di codice (LOC) o i Function Point (FP) per valutare la complessità e l'effort richiesto per ciascuna parte del progetto. Questo approccio è particolarmente utile per progetti complessi e dettagliati.
- **Modelli algoritmici empirici:** Queste tecniche di stima si basano su dati storici raccolti da progetti software precedenti. I modelli utilizzano relazioni matematiche tra variabili indipendenti (come LOC o FP stimati) e la variabile da stimare (come l'effort, il costo o la durata del progetto). Un esempio comune è il modello COCOMO (CONstructive COSt MOdel) che utilizza LOC stimati per calcolare il costo e l'effort.

FUNCTION POINT- FP

I Function Point (FP) sono una misura ponderata della funzionalità del software proposta da Albrecht negli anni '70 e '80. I Function Point misurano la quantità di funzionalità in un sistema software basandosi sulla specifica del sistema (stima prima dell'implementazione effettiva del software). Il calcolo dei Function Point avviene in due fasi principali:

- **Calcolo dell'Unadjusted Function Point Count (UFC):** Questa fase coinvolge il conteggio delle diverse categorie di dati e transazioni presenti nel sistema software.
- **Moltiplicazione dell'UFC per un Technical Complexity Factor (TCF):** Una volta calcolato l'UFC, viene applicato un fattore di complessità tecnica per riflettere la complessità aggiuntiva del sistema. Questo fattore considera vari aspetti come la complessità dell'architettura, la performance, l'usabilità e altri fattori tecnici che influenzano la complessità complessiva del progetto.

Il risultato finale, il Function Point (FP), è ottenuto moltiplicando l'UFC per il TCF: **FP = UFC × TCF**

Categorie di conteggio dei dati (UFC):

- **Internal Logical Files (ILF):** Questi rappresentano gruppi di dati o informazioni di controllo generate, utilizzate o mantenute dal sistema software stesso.
- **External Interface Files (EIF):** Questi rappresentano gruppi di dati o informazioni di controllo condivise o scambiate tra applicazioni, inclusi gli input e gli output provenienti da altre applicazioni o sistemi.

Categorie di conteggio delle transazioni (TCF):

- **External Inputs (EI):** Questi rappresentano gli elementi forniti dall'utente che descrivono dati orientati all'applicazione, informazioni di controllo o output di altri sistemi che entrano in un'applicazione e cambiano lo stato dei suoi file logici interni.
- **External Outputs (EO):** Questi rappresentano tutti i dati o le informazioni di controllo unici prodotti dal sistema software, ad esempio rapporti e messaggi.
- **External Inquiries (EQ):** Questi rappresentano tutte le combinazioni uniche di input/output, in cui un input genera un output immediato senza cambiare lo stato dei file logici interni.

Il calcolo del Technical Complexity Factor (TCF) coinvolge l'assegnazione di un valore numerico a ciascuno dei **14 fattori di degree of influence**, in base al grado di influenza che hanno sul progetto software. Questi valori numerici variano da 0 (nessuna influenza) a 5 (influenza essenziale). Una volta assegnati i valori, si può calcolare il TCF sommando questi valori.

1. Affidabile backup e ripristino
2. Comunicazione dati
3. Elaborazione dati distribuita
4. Prestazioni
5. Configurazione ad utilizzo intensivo
6. Inserimento dati online
7. Facilità operativa
8. Aggiornamento online
9. Interfaccia complessa
10. Elaborazione complessa
11. Riutilizzabilità
12. Facilità di installazione
13. Siti multipli
14. Agevolare il cambiamento

Una volta assegnati i valori a ciascun fattore, si sommano per ottenere il TCF.

$$TCF = 0.65 + 0.01 \sum_{j=1}^{14} F_j$$

La formula indica che il TCF varia da 0.65 (se tutti i F_j sono impostati a 0) a 1.35 (se tutti i F_j sono impostati a 5). In altre parole, il TCF può essere regolato in base alla complessità tecnica attraverso i valori dei fattori F_j . La somma di $F_j * 0.01$ contribuisce all'aggiustamento complessivo della formula. Quindi, il risultato del TCF sarà nell'intervallo da 0.65 a 1.35, con una possibile variazione del $\pm 35\%$ a seconda dei valori specifici dei fattori F_j utilizzati nel calcolo.

FP vs LOC

- **Function Points (FP):** Misurano la funzionalità e la complessità funzionale di un sistema software in base alle sue caratteristiche e ai suoi requisiti funzionali. Sono indipendenti dal linguaggio di programmazione e rappresentano una misura più astratta della dimensione del software.
- **Lines of Code (LOC):** Misurano la dimensione del software contando il numero di linee di codice sorgente scritte. Questa metrica è fortemente dipendente dal linguaggio di programmazione utilizzato e non tiene conto della funzionalità o della complessità del software.

Relazione tra FP e LOC:

Molti studi hanno cercato di stabilire una relazione tra FP e LOC per aiutare a stimare il lavoro necessario per sviluppare il software. Tuttavia, questa relazione non è diretta o costante, poiché dipende da vari fattori, tra cui la complessità del software, le pratiche di programmazione, il linguaggio di programmazione e altro ancora.

Classificazione dei linguaggi di programmazione:

Per facilitare il confronto tra FP e LOC, i linguaggi di programmazione sono stati classificati in diverse categorie o livelli in base alla loro relazione media tra LOC e FP. Ad esempio, alcuni linguaggi possono richiedere più LOC per implementare una funzionalità equivalente a un certo numero di FP, mentre altri linguaggi possono richiederne meno.

Questa classificazione può essere utile per stimare la dimensione del software in LOC sulla base dei FP calcolati e viceversa.

Jones' Backfiring:

Il termine "Jones' Backfiring" si riferisce a una situazione in cui l'uso esclusivo delle LOC come metrica per la dimensione del software può portare a risultati errati o a previsioni inaccurate in termini di sforzo di sviluppo. Questo fenomeno può verificarsi quando si ignorano i fattori di complessità e funzionalità che non sono catturati dalle LOC.

In sintesi, sebbene ci siano sforzi per stabilire una relazione tra FP e LOC, queste due metriche rappresentano concetti diversi nella misurazione del software. FP è orientato alla funzionalità e alla complessità, mentre LOC è una misura della dimensione del codice sorgente. La scelta tra FP e LOC come metrica dipende dagli obiettivi specifici di misurazione e di gestione del progetto, nonché dalle caratteristiche del software in questione.

COCOMO

Il COCOMO (CONstructive COSt MOdel) è un modello introdotto da Boehm nel 1981 per determinare il valore dello sforzo (effort) necessario nello sviluppo software. Il valore ottenuto per lo sforzo viene successivamente utilizzato per determinare la durata e i costi di sviluppo del progetto. COCOMO si compone di tre modelli distinti:

- **Basic:** Utilizzato per stime iniziali del progetto.
- **Intermediate:** Applicato dopo aver suddiviso il sistema in sottosistemi.
- **Advanced:** Applicato dopo aver suddiviso in moduli ciascun sottosistema.

La stima dello sforzo viene effettuata sulla base di due fattori principali:

- **Stima delle dimensioni del progetto in KLOC:** KLOC sta per migliaia di linee di codice. Questa stima rappresenta la dimensione del progetto in termini di quantità di codice da sviluppare.
- **Stima del modo di sviluppo del prodotto:** Misura il livello intrinseco di difficoltà nello sviluppo e può essere di tre tipi:
 - Organico (per prodotti di piccole dimensioni)
 - Semidetached (per prodotti di dimensioni intermedie)
 - Embedded (per prodotti complessi)

Nel 1995 è stata introdotta una versione più flessibile e sofisticata chiamata **COCOMO II** rispetto alla versione precedente, al fine di adattarsi meglio alle dinamiche e alla complessità dei progetti software moderni.

Ecco un **esempio** semplificato di utilizzo del modello COCOMO.

- **Passo 1 - Determinare l'effort nominale**

Inizialmente, si stima l'effort nominale utilizzando la formula seguente: $Effort\ Nominale = 3.2 \times (KLOC)^{1.05} MM$

Dove KLOC rappresenta le migliaia di righe di codice e MM sta per "Man-Months" (mese-uomo), una misura dell'effort di sviluppo necessario.

Ad esempio, se si prevede di scrivere un software con 33.000 linee di codice (33 KLOC), l'effort nominale sarà calcolato come segue: $Effort\ Nominale = 3.2 \times (33)^{1.05} \approx 126\ MM$

- **Passo 2 - Ottenere la stima dell'effort effettivo**

Successivamente, si applica un fattore moltiplicativo C basato su 15 cost drivers per ottenere la stima dell'effort effettivo. Questi cost drivers riflettono varie influenze e complessità associate al progetto.

L'effort effettivo è calcolato come segue: $Effort\ Effettivo = Effort\ Nominale \times C$

Ad esempio, se si applica un moltiplicatore C di 1.15, l'effort effettivo sarà: $Effort\ Effettivo = 126\ MM \times 1.15 \approx 145\ MM$

Il valore del moltiplicatore C dipenderà dalla valutazione dei 15 cost drivers specifici del progetto. Questi cost drivers includono fattori come la complessità dell'architettura, la produttività del personale, la complessità del software, l'esperienza del team e altri.

Il modello COCOMO include una stima del tempo necessario per la consegna del prodotto (product delivery) in base alla dimensione del progetto E. Ecco le formule per la stima del tempo T in mesi per i diversi modi di sviluppo:

- **Modo organico:** $T = 2.5 * E^{0.38}$ mesi
- **Modo semi-detached:** $T = 2.5 * E^{0.35}$ mesi
- **Modo embedded:** $T = 2.5 * E^{0.32}$ mesi

Dove T è il tempo stimato per la consegna del prodotto in mesi ed E rappresenta la dimensione del progetto, spesso misurato in KLOC (migliaia di linee di codice).

Queste formule consentono di stimare il tempo necessario per la consegna del prodotto in base alla complessità intrinseca del progetto, che può variare a seconda del modo di sviluppo utilizzato (organico, semi-detached o embedded).

PIANIFICAZIONE TEMPORALE

La pianificazione temporale è una fase cruciale in qualsiasi progetto e coinvolge la definizione di un piano dettagliato che indica quando e come saranno eseguite le attività pianificate. La pianificazione temporale consiste nel definire una "rete di task" in base ai seguenti principi fondamentali:

- **Ripartizione:** Questo principio implica la suddivisione del processo e del prodotto in parti più gestibili. In pratica, significa scomporre il progetto in attività più piccole e gestibili, spesso chiamate "task" o "attività".
- **Interdipendenza:** Identificare le dipendenze tra le diverse attività. Alcuni task potrebbero dipendere da altri per essere completati, mentre altri possono essere eseguiti parallelamente. Comprendere le relazioni di dipendenza è essenziale per stabilire un ordine logico delle attività.
- **Allocazione di risorse:** Determinare il numero di persone coinvolte nel progetto, la quantità di sforzo richiesto e le date di inizio e fine per ciascun task. Questo aiuta a stabilire una stima realistica delle risorse necessarie per portare a termine il progetto.
- **Responsabilità definite:** Assegnare chiaramente le responsabilità a ciascun task. Ogni attività dovrebbe essere associata a una persona o a un gruppo responsabile del suo completamento.
- **Risultati previsti:** Definire chiaramente i risultati attesi al termine di ciascuna attività. Questo contribuisce a mantenere il focus sulle consegne e sulla qualità del lavoro svolto.
- **Punti di controllo (Milestone):** Identificare punti di controllo importanti durante il progetto, spesso associati a task specifici o gruppi di task. Le milestone sono utilizzate per valutare il progresso, controllare la qualità e apportare eventuali correzioni di rotta.

Strumenti di pianificazione:

- **Diagramma PERT (Program Evaluation and Review Technique):** È un grafo che rappresenta il flusso delle attività in un progetto. Ogni nodo nel grafo rappresenta un task, mentre gli archi indicano le dipendenze tra i task.
 - Utilizzo:
 - Determina il cammino critico, che è la sequenza di task che determina la durata minima del progetto. Questi task hanno un impatto significativo sulla durata totale del progetto.
 - Fornisce una stima del tempo di completamento di ciascun task utilizzando modelli statistici.
 - Indica i limiti temporali di inizio e termine per ciascun task.
 - Vantaggi:
 - Visualizza chiaramente le dipendenze tra le attività.
 - Aiuta a identificare le attività critiche che influenzano la durata complessiva del progetto.
- **Carta di Gantt:** È un diagramma a barre che mostra la durata temporale delle attività lungo un asse temporale. Ogni barra rappresenta un task e la sua lunghezza indica la durata prevista.
 - Utilizzo:
 - Visualizza l'allocazione temporale delle attività in modo chiaro e intuitivo.
 - Fornisce una panoramica visiva delle date di inizio e fine di ciascun task.
 - Vantaggi:

- Facile da comprendere e comunicare, sia internamente al team che esternamente agli stakeholder.
- Permette di individuare sovrapposizioni e ritardi.

Entrambi gli strumenti sono spesso utilizzati in combinazione. Mentre il Diagramma PERT si concentra sulle relazioni di dipendenza tra le attività e sul cammino critico, la Carta di Gantt fornisce una visione più semplice e immediata delle scadenze e della durata delle attività nel tempo. Integrare i due approcci può offrire una visione più completa e dettagliata della pianificazione del progetto.

SOFTWARE PROJECT MANAGEMENT PLAN (SPMP)

Un piano di gestione del progetto software (SPMP) è un documento dettagliato che delinea come il progetto software sarà pianificato, eseguito, controllato e chiuso.

- **Project Scope (Ambito del Progetto):** Questa sezione definisce i limiti e gli obiettivi del progetto. Indica cosa è incluso nel progetto e cosa è escluso.
 - Contenuti possibili:
 - Dichiarazione del problema o dell'opportunità.
 - Descrizione dei deliverable (prodotti consegnabili) del progetto.
 - Limitazioni e vincoli del progetto.
- **Estimates (Stime):** Questa sezione affronta le stime relative a diverse risorse del progetto, come tempo, costo e sforzo umano.
 - Stime di durata per ciascuna attività.
 - Stime di costo per risorse e attrezzature.
 - Stime di sforzo umano necessario.
- **Risks (Rischi):** Identifica e analizza i rischi potenziali che potrebbero influenzare il successo del progetto.
 - Elenco di rischi specifici e delle relative probabilità e impatti.
 - Strategie di mitigazione dei rischi.
 - Piani di risposta a situazioni di emergenza.
- **Schedule (Pianificazione):** Definisce la sequenza temporale delle attività del progetto, indicando quando ogni attività inizia e finisce.
 - Diagramma di Gantt o altro strumento di visualizzazione temporale.
 - Sequenza delle attività e delle dipendenze tra di esse.
 - Milestone (punti di controllo) critici.
- **Control strategy (Strategia di Controllo):** Indica come il progetto sarà monitorato e controllato durante la sua esecuzione.
 - Procedure di monitoraggio delle attività.
 - Strumenti e metriche di controllo.
 - Fasi di revisione e valutazione.

La struttura generale di un Software Project Management Plan (SPMP) basato sugli standard dell'IEEE (Institute of Electrical and Electronics Engineers), in particolare lo standard IEEE Std. 1058-1998 è costituito dalle seguenti sezioni principali:

- **Overview:**
 - Project Summary: Una panoramica generale del progetto.
 - Evolution of the Plan: Informazioni sulla storia e l'evoluzione del piano.
- **References:** Elenco di riferimenti e documenti utilizzati nella preparazione del piano.
- **Definitions:** Definizioni di termini chiave utilizzati nel documento.
- **Project Organization:**
 - External Interfaces: Descrizione delle interfacce esterne con altre entità.
 - Internal Structure: Struttura interna del team di progetto.
 - Roles and Responsibilities: Descrizione dei ruoli e delle responsabilità delle persone coinvolte.
- **Managerial Process Plans:**
 - Start-up Plan: Piani per l'avvio del progetto, inclusi piani di stima, pianificazione delle risorse e formazione del personale.
 - Work Plan: Piani relativi alle attività di lavoro, inclusi l'allocazione di risorse e di budget.
 - Control Plan: Piani per il controllo dei requisiti, del cronogramma, del budget, della qualità, e la raccolta di metriche.
 - Risk Management Plan: Piani per la gestione dei rischi nel progetto.
 - Closeout Plan: Piani per la chiusura del progetto.
- **Technical Process Plans:**
 - Process Model: Descrizione del modello di processo utilizzato.
 - Methods, Tools and Techniques: Utilizzo di metodi, strumenti e tecniche nel progetto.
 - Infrastructure Plan: Piani relativi all'infrastruttura necessaria per il progetto.

- Product Acceptance Plan: Piani per l'accettazione del prodotto finale.
- **Supporting Process Plans:**
 - Configuration Management Plan: Piani per la gestione della configurazione.
 - Verification and Validation Plan: Piani per la verifica e la convalida del prodotto.
 - Documentation Plan: Piani relativi alla documentazione del progetto.
 - Quality Assurance Plan: Piani per garantire la qualità del prodotto.
 - Reviews and Audits: Procedure per revisioni e audit.
 - Problem Resolution Plan: Piani per la risoluzione dei problemi.
 - Subcontractor Management Plan: Piani per la gestione dei subappalti.
 - Process Improvement Plan: Piani per il miglioramento continuo dei processi.
- **Additional Plans:** Altri piani specifici o particolari necessari per il progetto.

ESERCIZIO OOA: SISTEMA SW PER ONLINE SHOPPING – 09/01/2023

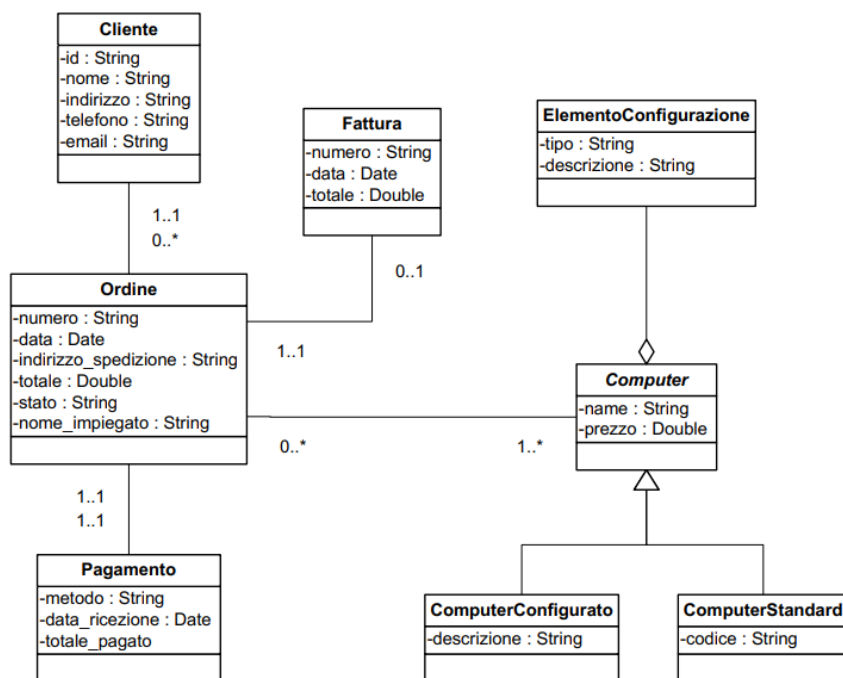
REQUISITI UTENTE

- Il sistema software deve supportare l'azienda X che vende computer online
- I clienti che accedono al sistema possono scegliere di acquistare un computer in configurazione standard o costruire una specifica configurazione selezionando i singoli elementi (ad es.: processore, disco, RAM, etc.)
- Per effettuare l'ordine, il cliente deve fornire le informazioni necessarie per la spedizione e per il pagamento
- Il cliente può usare il sistema per verificare online lo stato dell'ordine
- Il computer nella configurazione scelta viene inviato al cliente assieme alla relativa fattura (se richiesta)

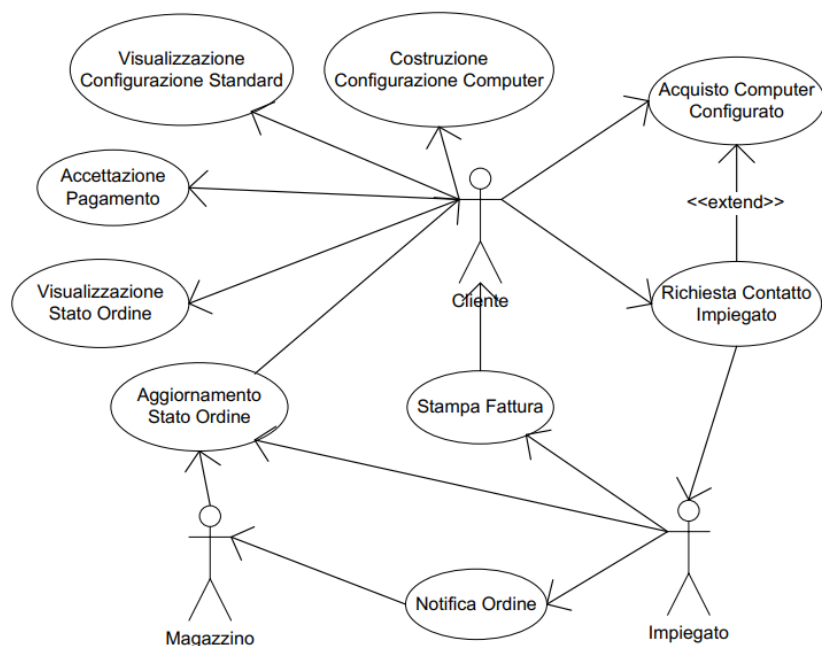
SVILUPPARE LA SPECIFICA SECONDO OOA

1. Si produca inizialmente il modello dei dati costruendo un class diagram in cui le operazioni di ciascuna classe possono essere omesse. Per ciascuna associazione devono invece essere specificate le molteplicità, mentre i nomi di ruolo possono essere omessi.
2. Successivamente, si produca una porzione di modello comportamentale identificando attori e casi d'uso e specificando la descrizione di un caso d'uso a scelta, sia in forma testuale che usando un activity diagram.
3. A partire dal caso d'uso identificato, si produca un sequence diagram che mostri una possibile interazione tra gli oggetti del sistema.
4. Infine, a partire dal sequence diagram, si produca un raffinamento del class diagram iniziale, identificando le operazioni ed eventuali classi, associazioni o attributi aggiuntivi.

CLASS DIAGRAM



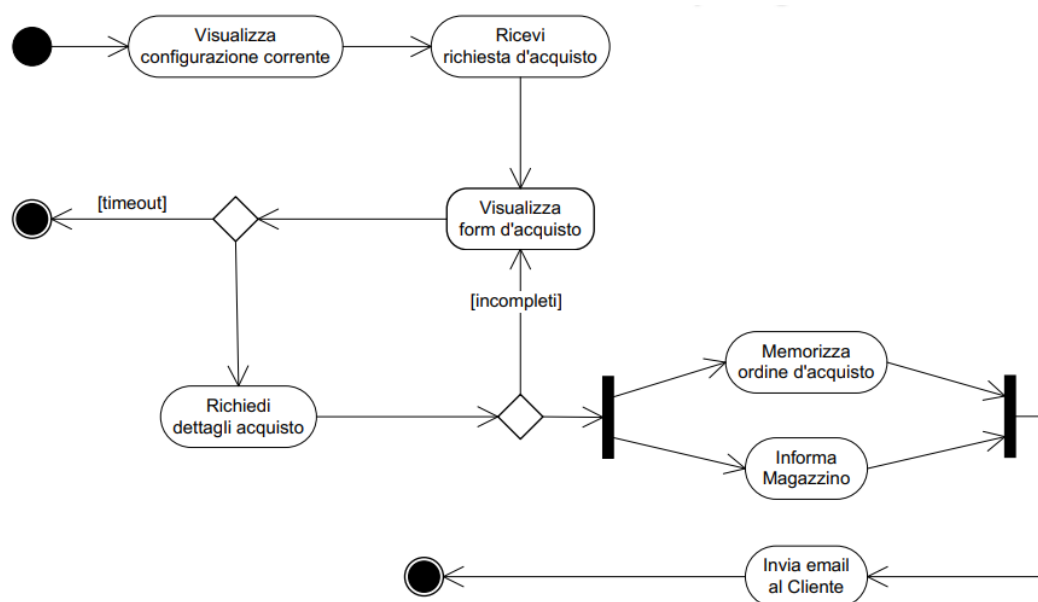
USE CASE DIAGRAM



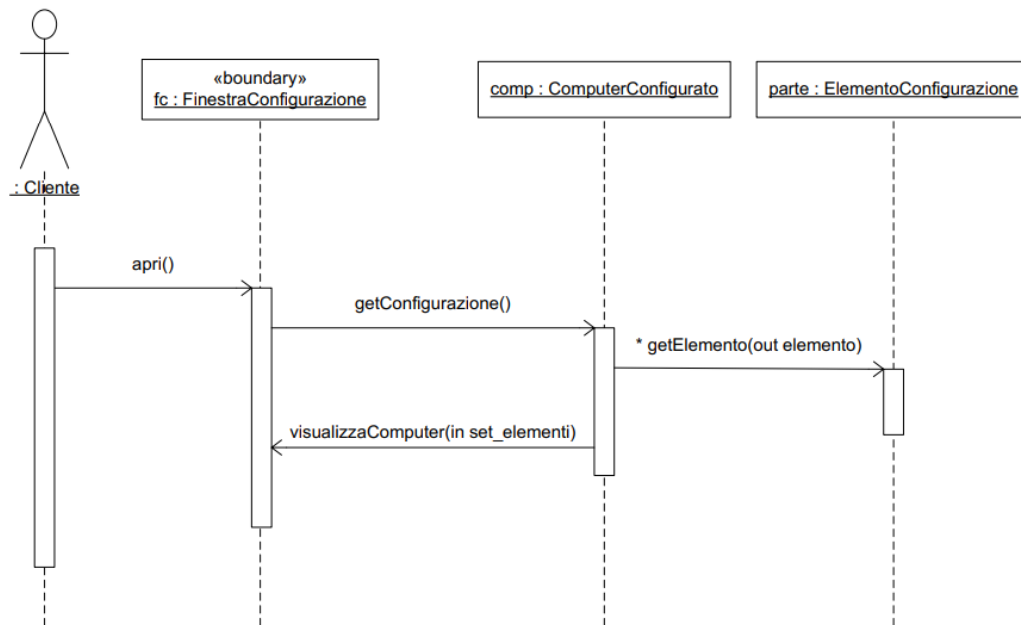
Descrizione testuale:

Brief Description	Il caso d'uso permette al Cliente di inserire un ordine di acquisto....
Actors	Cliente
Preconditions	L'interfaccia utente mostra a video i dettagli del computer configurato, insieme al prezzo relativo...
Main Flow	Il sistema assegna un numero unico e l'identificatore del cliente all'ordine di acquisto e ne memorizza le informazioni nel database...
Alternative Flows	Il Cliente attiva la funzionalità di acquisto prima di aver fornito tutte le informazioni necessarie...
Postconditions	L'ordine di acquisto viene registrato nel database di sistema.

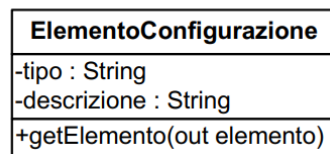
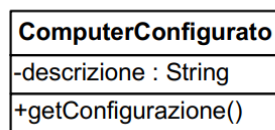
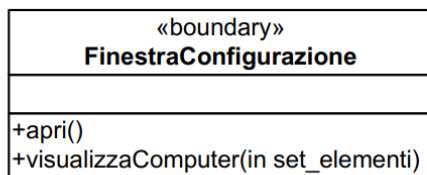
DESCRIZIONE FLUSSO CON ACTIVITY DIAGRAM



SEQUENCE DIAGRAM



RAFFINAMENTO CLASS DIAGRAM



INGEGNERIA DEL SOFTWARE

MODULO 2

PROGETTO

FASE DI PROGETTO

La fase di progetto è una tappa chiave nel ciclo di vita dello sviluppo del software. Questa fase si concentra sul passaggio dalla definizione di "cosa" deve essere realizzato, come specificato nell'analisi dei requisiti, a "come" la realizzazione dovrebbe avvenire. La fase di progetto prende in input il documento di specifica (analisi dei requisiti) e produce un documento di progetto che guida la successiva fase di codifica.

La fase di progetto può essere suddivisa in due sotto-fasi principali:

- **Progetto Architettuale (o Preliminare):** Durante questa fase, il sistema software complessivo viene suddiviso in sottosistemi più gestibili. Questa suddivisione è spesso nota come decomposizione modulare. L'obiettivo è definire un'architettura generale del sistema.
- **Progetto Dettagliato:** In questa fase, ogni sottosistema identificato nella fase di progetto architettuale viene progettato in dettaglio. Si fa una scelta più specifica di algoritmi, strutture dati e altro necessario per implementare ciascun modulo.

PRINCIPI DI PROGETTAZIONE

Stepwise refinement: Si tratta di una strategia di progettazione nell'ambito della programmazione strutturata proposta da Wirth nel 1971. Questa strategia è utilizzata per sviluppare un programma in modo graduale e organizzato, partendo da una visione generale e affinando i dettagli progressivamente.

- **Definizione della Strategia:** Il procedere per raffinamenti successivi è una strategia top-down, il che significa che inizia con una visione generale e si sposta gradualmente verso i dettagli più specifici.
- **Specifiche iniziali:** Si inizia con la specifica di una funzione o di dati senza descrivere il funzionamento interno della funzione o la struttura interna dei dati. In questa fase, si ha una visione ad alto livello.
- **Processo di Raffinamento:** Il raffinamento è un processo iterativo che consiste nell'aggiungere livelli di dettaglio crescenti ad ogni passo. Man mano che ci si sposta attraverso i passaggi di raffinamento successivi, si sviluppa una visione sempre più dettagliata e completa del sistema.
- **Legge di Miller:** Afferma che in un dato momento una persona può concentrarsi su al massimo 7 ± 2 "chunk" di informazioni. Questo sottolinea l'importanza di suddividere il problema in parti più gestibili e concentrarsi su dettagli limitati in ogni fase del processo.
- **Complementarità con l'Astrazione:** Mentre l'astrazione coinvolge l'isolamento dei dettagli non rilevanti e la focalizzazione sugli aspetti cruciali, il raffinamento si occupa di aggiungere progressivamente dettagli per completare l'implementazione.

Astrazione: Significa isolare e concentrarsi sugli aspetti essenziali di un'entità, ignorando i dettagli non rilevanti. In altre parole, si tratta di estrarre la sostanza fondamentale o l'essenza di un concetto o di un'entità.

- **Livelli di Astrazione:** Il concetto di livello di astrazione è stato introdotto da Dijkstra nel 1968, specialmente nell'ambito dei sistemi operativi, per descriverne l'architettura a strati. Ogni livello rappresenta un punto di vista o una visione specifica del sistema, con un grado crescente di dettagli.
- **Processo Software e Raffinamento:** Nel contesto del processo software, ogni passo rappresenta un raffinamento del livello di astrazione della soluzione. Prima di decidere come implementare qualcosa, ci si concentra su cosa è e cosa fa un'entità del sistema software.
- **Tipi Principali di Astrazione:**
 - **Astrazione Procedurale:** Coinvolge la focalizzazione sugli aspetti procedurali di un sistema. Ad esempio, l'uso di funzioni in linguaggi di programmazione come il linguaggio C rappresenta un'astrazione procedurale.
 - **Astrazione dei Dati:** Coinvolge la focalizzazione sugli aspetti relativi ai dati, spesso implementata attraverso concetti come l'incapsulamento dei dati.
- **Data Encapsulation:** Questo concetto implica che i dati e le operazioni ad essi associate sono raggruppati in un'unica entità.

Modularità: Si riferisce alla suddivisione di un sistema software in moduli distinti e autonomi. Questo approccio contrasta con l'idea di un unico blocco monolitico di codice.

- **Vantaggi:**
 - **Facilità di Manutenzione:** I moduli autonomi consentono di concentrarsi su porzioni specifiche del codice, semplificando la manutenzione e le modifiche.
 - **Facilità di Correzione:** Gli errori sono spesso limitati a un singolo modulo, rendendo più facile individuarli e correggerli senza influenzare altre parti del sistema.

- **Facilità di Comprensione:** La suddivisione del codice in moduli rende più semplice la comprensione del sistema nel suo complesso, in quanto è possibile concentrarsi su un modulo alla volta.
- **Riusabilità:** I moduli ben progettati possono essere riutilizzati in diversi contesti, promuovendo la riusabilità del codice.

Decomposizione modulare: È una pratica di progettazione del software che mira a suddividere un sistema complesso in moduli più gestibili. Questi moduli costituiscono unità di codice indipendenti che possono essere compilati e utilizzati separatamente.

- **Caratteristiche di un Modulo:**
 - Contiene istruzioni, logica di elaborazione e strutture dati.
 - Può essere compilato separatamente e memorizzato in una libreria software.
 - Può essere incluso in un programma, utilizzando segmenti di modulo identificati da un nome e da una lista di parametri.
 - Può utilizzare altri moduli.
- **Obiettivi:**
 - **Massima coesione interna ai moduli:** Un modulo dovrebbe concentrarsi su una specifica funzionalità e avere una forte coesione interna.
 - **Minimo grado di accoppiamento tra i moduli:** L'interazione tra moduli dovrebbe essere ridotta al minimo indispensabile.
 - Questi obiettivi aumentano la comprensibilità, la manutenibilità, l'estensibilità e la riusabilità del software.

Coesione e Accoppiamento rispetto a Modularità:

- La coesione (cohesion) si riferisce al grado in cui gli elementi all'interno di un modulo sono strettamente correlati tra loro. Un modulo con alta coesione esegue un insieme ben definito di attività correlate, mentre un modulo con bassa coesione potrebbe svolgere attività diverse e poco correlate. L'obiettivo è massimizzare la coesione all'interno di ciascun modulo.
- L'accoppiamento (coupling) si riferisce al grado di dipendenza tra moduli. Un basso accoppiamento implica una minima dipendenza tra i moduli, il che significa che le modifiche in un modulo hanno un impatto limitato sugli altri. L'obiettivo è minimizzare l'accoppiamento tra i moduli.

Coesione: Misura il grado di interazione interna al modulo tra le azioni di una funzione. Esistono 7 livelli di coesione, che variano da quello più debole a quello più forte:

- **Coincidental (Livello 1):** Non esiste una relazione significativa tra gli elementi del modulo. Le attività sono raggruppate insieme casualmente e non c'è una logica o un collegamento intrinseco tra di esse.
- **Logica (Livello 2):** Gli elementi del modulo sono correlati solo in modo logico. Uno di essi viene selezionato e utilizzato dal modulo chiamante, ma non c'è una stretta connessione tra le attività del modulo.
- **Temporale (Livello 3):** Gli elementi del modulo sono correlati in base a un ordine temporale o sequenza di esecuzione. Le attività avvengono in un certo ordine, ma potrebbero non essere strettamente connesse concettualmente.
- **Procedurale (Livello 4):** Gli elementi del modulo sono correlati in base a una sequenza predefinita di passi da eseguire. Questo implica una connessione più forte rispetto alla coesione temporale, poiché le attività seguono un ordine ben definito.
- **Communicational (Livello 5):** Gli elementi del modulo sono correlati in base a una sequenza di passi che vengono eseguiti sulla stessa struttura dati. Le attività all'interno del modulo comunicano attraverso la condivisione di dati.
- **Informational (Livello 6):** Ogni elemento del modulo ha una porzione di codice indipendente con un proprio punto di ingresso e uscita. Tutti gli elementi agiscono sulla stessa struttura dati, ma eseguono attività indipendenti.
- **Funzionale (Livello 7):** Tutti gli elementi del modulo sono strettamente correlati perché svolgono una singola funzione o responsabilità. Questo rappresenta il livello più elevato di coesione e riflette una progettazione ottimale del modulo.

In generale, l'obiettivo è massimizzare la coesione di un modulo, poiché una coesione più elevata generalmente porta a moduli più facili da capire, mantenere e riusare. Un modulo altamente coeso è spesso un modulo più autonomo e indipendente, il che contribuisce a migliorare la qualità complessiva del software.

Coupling: Misura il grado di dipendenza o interconnessione tra i moduli di un sistema software. Un basso accoppiamento è generalmente auspicabile, poiché implica una minore dipendenza tra i moduli, rendendo il sistema più flessibile, manutenibile e riutilizzabile. Al contrario, un alto accoppiamento può portare a un sistema più difficile da modificare e gestire. Esistono 5 livelli di accoppiamento:

- **Content (Livello 1):** Un modulo fa riferimento direttamente al contenuto interno di un altro modulo. Questo è il livello più alto di accoppiamento e generalmente è da evitare.
- **Common (Livello 2):** Due moduli accedono alla stessa struttura dati. Ciò implica una dipendenza diretta sulla rappresentazione interna dei dati condivisi.

- **Control (Livello 3):** Un modulo controlla esplicitamente l'esecuzione di un altro modulo. Questo accoppiamento si verifica quando un modulo determina l'ordine di esecuzione di un altro modulo.
- **Stamp (Livello 4):** Due moduli si scambiano una struttura dati come argomento, ma utilizzano solo alcuni dei suoi elementi. Ciò implica una dipendenza parziale sulla struttura dati condivisa.
- **Data (Livello 5):** Due moduli si scambiano argomenti omogenei, come argomenti semplici o strutture dati di cui utilizzano tutti gli elementi. Questo rappresenta il livello più basso di accoppiamento.

Information Hiding: Questo concetto è stato introdotto da Parnas nel 1971 e costituisce la base per i concetti di astrazione procedurale e astrazione dei dati. Questa tecnica di progettazione si concentra sulla definizione dei moduli in modo che i dettagli implementativi, sia procedurale che relativi ai dati, non siano accessibili da altri moduli che non ne hanno necessità.

- **Obiettivo:** Nascondere i dettagli implementativi per evitare che altri moduli dipendano da tali dettagli. Ciò consente di isolare le implementazioni specifiche e di ridurre l'accoppiamento tra i moduli.
- **Vantaggi:**
 - Migliora la modularità e la manutenibilità del sistema.
 - Facilita i cambiamenti senza influenzare altri moduli.
 - Favorisce la separazione delle responsabilità.
- **Applicazione in Fasi di Testing e Manutenzione:** I vantaggi della tecnica di information hiding diventano evidenti durante le fasi di testing e manutenzione, quando è possibile apportare modifiche a una parte del sistema senza dover considerare gli effetti su altri moduli.

Riusabilità: Si riferisce all'utilizzo di componenti sviluppati per un prodotto all'interno di un prodotto differente. Può riguardare non solo moduli o frammenti di codice, ma anche progetti, parti di documentazione, insiemi di dati di test o stime di costi e durata.

- **Vantaggi:**
 - Riduzione dei costi e dei tempi di produzione del software.
 - Aumento dell'affidabilità grazie all'uso di componenti già convalidati.
- **Applicazione nella Fase di Progetto:** La riusabilità può applicarsi a diversi livelli.
 - Moduli software.
 - Application framework: incorpora la logica di controllo di un progetto.
 - Design pattern: identifica soluzioni di progetto ricorrenti.
 - Architetture software che comprendono moduli, framework e design pattern.

OBJECT ORIENTED DESIGN- OOD

Object-Oriented Design è una fase del processo di sviluppo del software che segue l'Object-Oriented Analysis (OOA). La fase di OOD è suddivisa in due sottofasce principali:

- **Fase Preliminare (o Architetturale, o di Sistema) di OOD:** Questa fase definisce la strategia generale per costruire una soluzione che risolva il problema specificato durante l'Object-Oriented Analysis (OOA). Le decisioni prese durante questa fase riguardano l'organizzazione complessiva del software, inclusa l'architettura di sistema.
- **Fase Dettagliata (o degli Oggetti) di OOD:** Questa fase fornisce la definizione completa delle classi e delle associazioni che devono essere implementate, così come le strutture dati e gli algoritmi dei metodi che implementano le operazioni delle classi. Durante questa fase, si aggiungono dettagli tecnici alla soluzione hardware/software, definendo come il software deve essere implementato.

Secondo un approccio di sviluppo iterativo e incrementale, il modello OOA viene "trasformato" nel modello OOD. Questa trasformazione aggiunge dettagli tecnici alla soluzione, specificando come il software deve essere implementato. Questo approccio consente di affinare gradualmente la progettazione in base a feedback e requisiti in evoluzione.

ARCHITETTURA DI SISTEMA

L'architettura di sistema (system architecture) definisce la struttura dei componenti del sistema software insieme alle relazioni tra tali componenti e ai principi che guidano la progettazione e l'evoluzione del sistema.

Sistemi Software Distribuiti: I sistemi software distribuiti prevedono che l'elaborazione sia distribuita su un insieme di host di esecuzione indipendenti, collegati da un'infrastruttura di rete (locale o a vasta area).

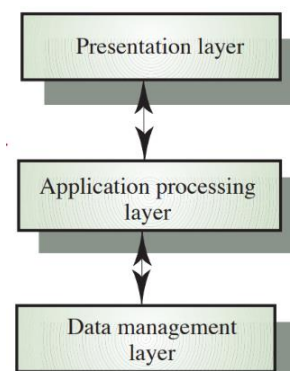
- **Set di Host di Esecuzione Indipendenti:** L'elaborazione è distribuita su più host di esecuzione, visti dagli utenti come un singolo host.
- **Infrastruttura di Rete:** Gli host sono collegati da un'infrastruttura di rete, che può essere di tipo locale (LAN) o di vasta area (WAN).
- **Ruolo della Tecnologia Middleware:** La tecnologia middleware gioca un ruolo essenziale nella transizione da architetture centralizzate a distribuite. Middleware è il software che fornisce la connettività tra applicazioni distribuite. Si trova tra i livelli di applicazione e sistema operativo, offrendo servizi per stabilire l'interazione tra i processi delle varie applicazioni eseguite su host di rete.

Architetture Client/Server (C/S): Le architetture client/server sono un modello di progettazione comune nei sistemi distribuiti. In questo modello, i processi coinvolti possono essere distinti in due ruoli principali:

- **Ruolo del Processo Cliente:** Il processo cliente interagisce direttamente con l'utente e svolge le seguenti funzioni:
 - Fornisce l'interfaccia utente per raccogliere le richieste dell'utente.
 - Inoltra le richieste ai server utilizzando la tecnologia middleware.
 - Visualizza le risposte dei server all'utente attraverso l'interfaccia utente.
- **Ruolo del Processo Server:** Il processo server (o il set di processi eseguiti su un dato host) fornisce servizi ai clienti e svolge le seguenti funzioni:
 - Risponde alle richieste dei clienti (non è il server che inizia la conversazione con il cliente).
 - Nasconde la complessità dell'intero sistema C/S all'utente.
 - Un server può a sua volta agire come un cliente, inoltrando la richiesta iniziale a un server secondario, senza far sapere al cliente o all'utente della catena di inoltri.

Livelli di Applicazione (Application Layers): Nei sistemi software, i livelli di applicazione sono divisioni logiche o componenti che organizzano e separano le funzionalità del sistema.

- **Livello di Presentazione (Presentation):** incentrato sulla raccolta degli input degli utenti e sulla presentazione dei risultati di un'elaborazione agli utenti del sistema. Ad esempio, un'applicazione bancaria può raccogliere l'input dell'utente attraverso moduli online e visualizzare i saldi o le transazioni in modo user-friendly.
 - Raccogliere input dagli utenti attraverso interfacce utente.
 - Presentare i risultati dell'elaborazione in modo comprensibile per gli utenti.
- **Livello di Elaborazione dell'Applicazione (Application Processing):** si occupa di fornire funzionalità specifiche dell'applicazione. Ad esempio, in un sistema bancario, le funzioni bancarie come apertura di un conto o chiusura di un conto appartengono a questo livello.
 - Implementare le funzionalità specifiche dell'applicazione.
 - Gestire i processi aziendali e le regole di business.
- **Livello di Gestione dei Dati (Data Management):** si occupa di gestire l'accesso ai dati dell'applicazione. Questo può includere l'archiviazione, il recupero e la manipolazione dei dati necessari per il funzionamento dell'applicazione.



- Gestire l'archiviazione e il recupero dei dati.
- Fornire meccanismi per garantire l'integrità e la coerenza dei dati.

Architetture Client/Server a Due Livelli (Two-tier C/S): Nelle architetture client/server a due livelli, si distinguono due modelli principali:

- **Modello Thin-Client:** Tutte le elaborazioni dell'applicazione e la gestione dei dati vengono eseguite sul server. Il client è responsabile solo dell'esecuzione del software di presentazione. Il client raccoglie l'input dell'utente, trasmette le richieste al server, e il server esegue tutte le operazioni di business logic e di gestione dei dati. Il risultato viene quindi restituito al client, che si occupa solo di presentare i dati all'utente.
- **Modello Fat-Client:** Il server è responsabile solo della gestione dei dati. Il software sul client implementa sia la logica dell'applicazione che il software di presentazione. Il client è coinvolto attivamente nell'elaborazione dell'applicazione, eseguendo parte della logica aziendale e gestendo la presentazione dei dati. Il server fornisce i dati necessari, ma il client ha un ruolo significativo nell'elaborazione.

Architetture Client/Server a Tre Livelli (Three-tier C/S): Nelle architetture client/server a tre livelli, ogni livello dell'architettura dell'applicazione viene eseguito su un processore separato. Questa divisione consente un miglioramento delle prestazioni rispetto all'approccio a due livelli, mantenendo una gestione più semplice rispetto a un modello fat-client.

- Ogni livello (presentazione, elaborazione dell'applicazione, gestione dei dati) è eseguito su un processore separato. Il client è responsabile solo della presentazione, il server dell'elaborazione dell'applicazione e della gestione dei dati. Architettura più scalabile: l'aggiunta di server può gestire aumenti di domanda.

Architetture di Oggetti Distribuiti: Le architetture di oggetti distribuiti rompono la distinzione tradizionale tra client e server. Ogni oggetto distribuito può agire sia come client che come server, invocando metodi e rispondendo a invocazioni remote. La comunicazione tra oggetti è gestita da middleware basato sul concetto di "software bus", che rende trasparente la comunicazione remota.

Architetture Basate su Componenti: Definiscono il software come un insieme di componenti software autonomi, riusabili e sostituibili che collaborano all'interno di un framework di componenti. Questo approccio sfrutta l'astrazione, in cui i componenti fungono da blocchi di costruzione per sistemi più complessi, nascondendo i dettagli delle strutture minori.

- **Riutilizzo a Scatola Nera:** Il software è costruito attraverso il riutilizzo di componenti software come entità "black-box". Ogni componente ha dei "plug" con regole predefinite su come può essere collegato ad altri componenti. Invece di dover adattare la struttura di un software per modificare la sua funzionalità, un utente inserisce il comportamento desiderato nei parametri del componente.
- **Adattabilità e Variabilità:** Gli sviluppatori adattano i componenti ai requisiti dell'applicazione senza dover modificare il software esistente. Garantisce variabilità attraverso l'incapsulamento delle strutture software come componenti astratte e adattabilità attraverso la composizione dei componenti con parametri specifici.

Oggetti vs. componenti:

- Gli oggetti incapsulano servizi, mentre i componenti sono astrazioni (che possono essere utilizzate per costruire sistemi orientati agli oggetti).
- Gli oggetti hanno identità, stato e comportamento, e sono sempre entità in esecuzione; i componenti, d'altra parte, sono generalmente entità statiche necessarie al momento della costruzione del sistema (e non necessariamente esistono durante l'esecuzione).
- I componenti possono avere una granularità* più fine o più grossolana rispetto agli oggetti: ad esempio, classi, modelli, mix-in, moduli; i componenti dovrebbero avere un'interfaccia di composizione esplicita, che può essere verificata tramite il controllo del tipo.

** Il termine "granularità" si riferisce al livello di dettaglio o alla dimensione dei singoli componenti all'interno di un sistema o di un'architettura. In generale, una granularità più fine significa che le unità sono più piccole e dettagliate, mentre una granularità più grossolana indica unità più grandi e meno dettagliate. La scelta della granularità dipende spesso dai requisiti specifici del sistema e dai compromessi tra efficienza, flessibilità e facilità di gestione.**

Sviluppo di un Framework (Framework Development): Lo sviluppo di un framework si concentra sulla creazione di una struttura o di un insieme di strumenti che possono essere utilizzati come base per costruire diverse applicazioni.

- **Obiettivi:**
 - Fornire una struttura generica e riutilizzabile.
 - Definire le regole, i pattern e le interfacce comuni che facilitano lo sviluppo delle applicazioni.
 - Offrire funzionalità di base che possono essere estese o personalizzate secondo le esigenze specifiche dell'applicazione.

- **Utilizzo:** Gli sviluppatori di applicazioni utilizzano il framework come piattaforma di sviluppo, sfruttando le sue funzionalità predefinite e personalizzando dove necessario.

Sviluppo di un'Applicazione (Application Development): Lo sviluppo di un'applicazione si concentra sulla creazione di un prodotto software specifico per risolvere un problema o soddisfare un particolare requisito.

- **Obiettivi:**
 - Risolvere un problema specifico o fornire un servizio particolare.
 - Utilizzare il framework o altre librerie di supporto per accelerare lo sviluppo e sfruttare funzionalità predefinite.
 - Adattare il software alle esigenze specifiche dell'utente o del contesto di utilizzo.
- **Utilizzo:** Gli utenti finali interagiscono direttamente con l'applicazione per ottenere i benefici previsti senza dover necessariamente comprendere la complessità del framework utilizzato.

SERVICE ORIENTED ARCHITECTURE (SOA)

La SOA è un'architettura software distribuita che è composta da più servizi autonomi. In questa architettura, i servizi sono distribuiti in modo tale che possano essere eseguiti su nodi diversi forniti da diversi provider di servizi. L'obiettivo principale di SOA è progettare servizi come componenti autonomi e riutilizzabili. Ciò significa che i servizi devono essere autosufficienti e indipendenti.

SOA Protocols:

Per consentire la comunicazione tra i servizi e lo scambio di informazioni, vengono forniti protocolli standard basati su Internet. Ogni servizio ha una descrizione del servizio che consente alle applicazioni di scoprire e comunicare con il servizio. Questa descrizione definisce il nome del servizio, la sua posizione e i requisiti di scambio di dati.

Service Providers and Consumers:

Un service provider è un'entità che supporta servizi utilizzati da più client (o consumatori di servizi). A differenza delle architetture client/server tradizionali, le SOA si basano sul concetto di servizi debolmente accoppiati che possono essere scoperti e collegati dai client (chiamati anche consumatori di servizi o richiedenti di servizi) con l'assistenza di broker di servizi.

In sintesi, la SOA promuove la creazione di sistemi software flessibili, scalabili e riutilizzabili, in cui le funzionalità sono fornite come servizi indipendenti che possono essere combinati per soddisfare le esigenze specifiche di un'applicazione o di un processo aziendale. La comunicazione tra questi servizi avviene attraverso protocolli standard, e i servizi sono progettati per essere debolmente accoppiati, consentendo una maggiore flessibilità e riusabilità nel sistema complessivo.

Principali concetti di progettazione SOA:

- **Componenti autonomi riutilizzabili:** L'obiettivo principale della SOA è progettare servizi come componenti autonomi e riutilizzabili. Questo significa che ogni servizio dovrebbe essere autocontenuto e progettato per essere utilizzato in modo indipendente da altri servizi.
- **Accoppiamento debole:** I servizi devono essere debolmente accoppiati, il che significa che le dipendenze tra i servizi devono essere ridotte al minimo. Questo favorisce la flessibilità e la facilità di aggiornamento o sostituzione di un servizio senza influire sugli altri.
- **Servizi di coordinamento:** Invece di avere un servizio che dipende direttamente da un altro, vengono forniti servizi di coordinamento in situazioni in cui più servizi devono essere acceduti e l'accesso a essi deve essere sequenziato. Ciò può includere modelli come Service Registration, Service Brokering e Service Discovery.
- **Pattern architetturali:** Vengono descritti diversi modelli architetturali per applicazioni orientate ai servizi, tra cui modelli di broker, modelli di transazione (come Two-Phase Commit, Compound e Long-Living Transaction), e modelli di negoziazione.
- **Autonomia:** I servizi devono essere autonomi e indipendenti, gestendo internamente la propria logica e stato.
- **Astrazione:** Nascondere i dettagli interni del servizio, fornendo solo le informazioni essenziali attraverso il contratto.
- **Riutilizzabilità:** Progettare i servizi in modo che possano essere facilmente riutilizzati in diverse applicazioni o contesti.
- **Componibilità:** Consentire la composizione di servizi per creare nuove funzionalità o applicazioni.
- **Statoless:** I servizi devono essere progettati senza dipendenza dallo stato dell'altro, migliorando la scalabilità e la gestione delle risorse.
- **Ricerca automatica (Discoverability):** I servizi devono essere facilmente scopribili, in modo che le applicazioni possano trovare e utilizzare i servizi necessari senza eccessiva complessità.

BROKER PATTERN

- Nel modello di Broker (Broker pattern), il broker funge da intermediario tra client e servizi.
- I servizi si registrano presso il broker, rendendosi noti al broker stesso.
- I client individuano i servizi attraverso il broker.
- Dopo che il broker ha facilitato la connessione tra il client e il servizio, la comunicazione tra il client e il servizio può avvenire direttamente o attraverso il broker.

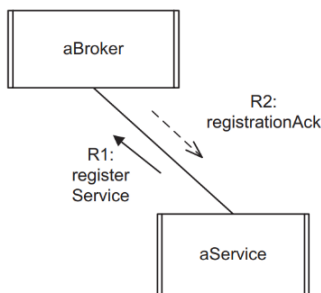
Trasparenza:

- Il broker offre trasparenza sia sulla posizione che sulla piattaforma.
- La trasparenza sulla posizione significa che se il servizio viene spostato in una posizione diversa, i client non ne sono consapevoli ed è necessario notificare solo al broker.
- La trasparenza sulla piattaforma significa che ogni servizio può essere eseguito su una piattaforma hardware/software diversa e non è necessario mantenere informazioni sulle piattaforme su cui eseguono gli altri servizi.

Comunicazione intermediata dal broker:

- Con la comunicazione intermediata dal broker, il client non deve conoscere direttamente la posizione di un determinato servizio.
- Invece, il client interroga il broker per ottenere informazioni sui servizi disponibili.
- Inizialmente, il servizio deve registrarsi presso un broker, come descritto dal "Service Registration pattern" (modello di registrazione dei servizi).

In sintesi, il Broker Pattern in un contesto SOA semplifica la gestione della comunicazione tra client e servizi. Fornisce una maggiore trasparenza sulla posizione e sulla piattaforma, consentendo una maggiore flessibilità nella distribuzione dei servizi e semplificando il processo di scoperta dei servizi da parte dei client.



Service Registration Pattern:

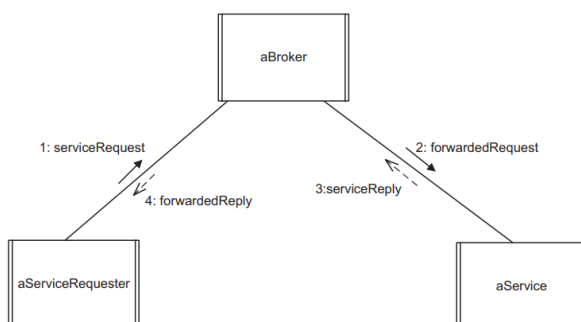
Questo pattern viene utilizzato quando un servizio desidera rendere noto la sua esistenza e mettere a disposizione le informazioni necessarie per la sua utilizzazione. Le informazioni da registrare includono il nome del servizio, una descrizione del servizio e la posizione in cui il servizio è disponibile.

- **Richiesta di registrazione del servizio:** Il servizio invia una richiesta di registrazione al broker. Questa richiesta contiene le informazioni necessarie sul servizio, come il suo nome, una descrizione e la sua posizione.

• **Registrazione presso il broker:** Il broker riceve la richiesta di registrazione e registra le informazioni del servizio nel registro dei servizi (service registry) gestito dal broker. Il registro dei servizi è un repository che contiene informazioni sui servizi disponibili nell'ambiente.

- **Conferma di registrazione:** Dopo aver registrato il servizio con successo, il broker invia una conferma di registrazione (registration acknowledgment) al servizio. Questa conferma informa il servizio che la registrazione è stata completata con successo e che il servizio è ora disponibile per essere scoperto e utilizzato da altri client attraverso il broker.

In questo modo, il Service Registration Pattern fornisce un meccanismo standardizzato attraverso il quale i servizi possono dichiarare la loro disponibilità e i dettagli necessari per l'interazione. Questo semplifica la scoperta dei servizi da parte dei client, in quanto possono consultare il registro dei servizi presso il broker per ottenere informazioni sui servizi disponibili nell'ambiente distribuito.



Il "**Broker Forwarding Pattern**" (Modello di Inoltro del Broker – White Pages) è un pattern architetturale che può essere utilizzato in sistemi distribuiti per migliorare la trasparenza della posizione e ridurre il traffico di messaggi.

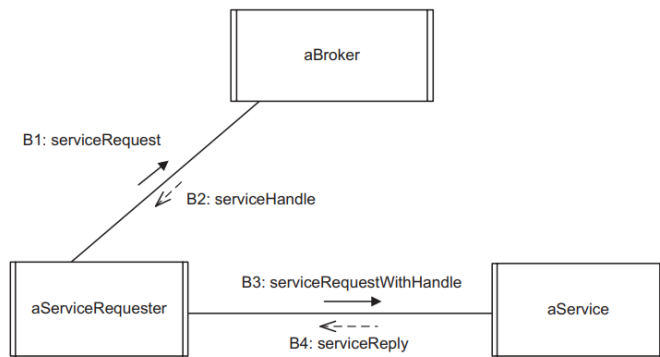
- **Il cliente invia un messaggio identificando il servizio richiesto:** Ad esempio, un cliente potrebbe inviare un messaggio indicando di voler prelevare contanti da una banca specifica.

- **Il broker riceve la richiesta del cliente:** Il broker è un componente software che funge da intermediario tra il cliente e il servizio. Quando riceve la richiesta, determina la posizione del servizio, che è identificato

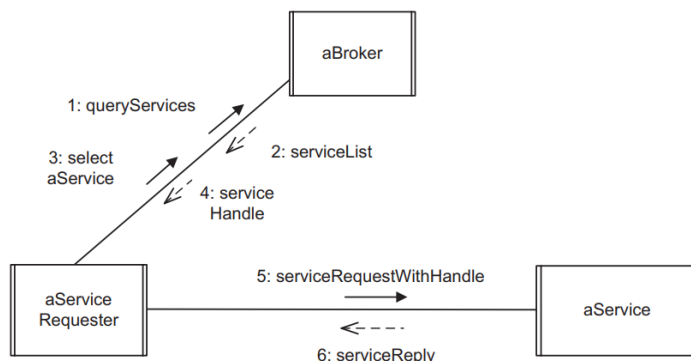
da un nodo specifico, e inoltra il messaggio al servizio sulla posizione specificata.

- **Il messaggio arriva al servizio e il servizio richiesto viene invocato:** Una volta che il messaggio raggiunge il servizio, il servizio richiesto viene attivato per eseguire l'azione desiderata. Ad esempio, potrebbe elaborare la richiesta di prelievo di contanti.

- **Il broker riceve la risposta del servizio e la inoltra al cliente:** Dopo che il servizio ha completato l'operazione richiesta, invia una risposta al broker. Il broker, quindi, inoltra questa risposta al cliente originario.



scambiare più messaggi tra di loro, poiché evita la necessità di inoltrare ogni singolo messaggio attraverso il broker.



Il "**Service Discovery Pattern**" (Modello di Scoperta dei Servizi), anche chiamato "yellow pages" (pagine gialle), è un modello architetturale che consente a un cliente di scoprire dinamicamente i servizi disponibili in un sistema distribuito. Questo è particolarmente utile quando il cliente sa il tipo di servizio che gli serve ma non conosce la sua posizione specifica all'interno della rete distribuita.

- **Il cliente invia una richiesta di query al broker:** Il cliente desidera un servizio di un determinato tipo ma non sa dove si trova nella rete distribuita. Invia quindi una richiesta di query al broker, chiedendo tutti i servizi di un certo tipo.

- **Il broker risponde con un elenco di servizi corrispondenti:** Il broker ha una conoscenza completa dei servizi disponibili nel sistema e risponde alla richiesta del cliente con un elenco di tutti i servizi che corrispondono al tipo richiesto.
- **Il cliente seleziona un servizio specifico:** Dopo aver ricevuto l'elenco di servizi dal broker, il cliente può consultare e scegliere un servizio specifico che sembra adatto alle sue esigenze o preferenze.
- **Il broker restituisce l'identificatore del servizio (service handle):** Una volta che il cliente ha selezionato un servizio, il broker restituisce un identificatore del servizio, noto come "service handle", al cliente. Questo handle può essere utilizzato dal cliente per comunicare direttamente con il servizio senza dover passare attraverso il broker per ogni singola comunicazione.

Questo modello è analogo alle pagine gialle di un elenco telefonico, dove è possibile cercare un tipo di servizio e ottenere un elenco di aziende che offrono quel tipo di servizio, senza dover sapere in anticipo l'ubicazione esatta di ciascuna azienda.

Supporto tecnologico per SOA:

Sebbene le SOA siano concettualmente indipendenti dalla piattaforma, sono attualmente implementate con successo su piattaforme di tecnologia dei servizi Web.

- **Servizi Web:** Un servizio Web è un servizio a cui si accede utilizzando protocolli standard di Internet e basati su XML (eXtensible Markup Language).
- **Protocolli per i Servizi Web:** Applicazioni client e servizi devono avere un protocollo di comunicazione per la comunicazione tra componenti.
- **XML:** Il linguaggio di markup estensibile (XML) è una tecnologia che consente a sistemi diversi di interagire scambiando dati e testo.
- **SOAP (Simple Object Access Protocol):** Un protocollo leggero sviluppato dal World Wide Web Consortium (W3C) che si basa su XML e HTTP per consentire lo scambio di informazioni in un ambiente distribuito. È composto da 3 parti:
 - Un "envelope" che definisce un framework per descrivere cosa c'è in un messaggio e come elaborarlo.
 - Un insieme di regole di codifica per esprimere istanze di tipi di dati definiti dall'applicazione.
 - Una convenzione per rappresentare chiamate di procedura remote e risposte.

Registration Services:

Un servizio di registrazione è fornito per consentire ai servizi di rendere disponibili le proprie funzionalità ai clienti.

- **Service Registry per Web Services:** Nel contesto dei servizi Web, viene fornito un "service registry" (registro dei servizi) per consentire ai servizi di essere pubblicati e individuati attraverso il World Wide Web.

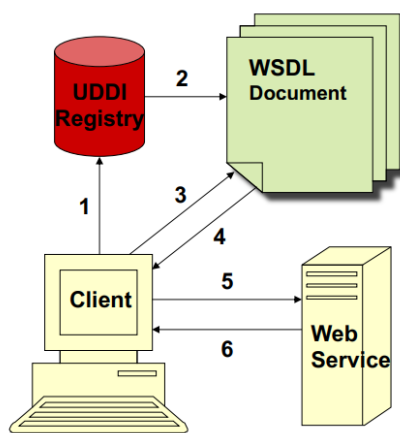
- **Registrazione dei Servizi:** I fornitori di servizi registrano i propri servizi presso un registro dei servizi. Questa registrazione include informazioni sul servizio stesso e una descrizione del servizio, spesso realizzata utilizzando il linguaggio di descrizione dei servizi Web (WSDL).
- **Ricerca dei Servizi:** I clienti che cercano un particolare servizio possono consultare il registro dei servizi. Questo può essere particolarmente utile in scenari in cui i clienti non conoscono a priori la disponibilità o la posizione di un servizio.
- **Utilizzo di WSDL:** Il linguaggio di descrizione dei servizi Web (WSDL) è uno strumento comune per descrivere cosa fa un servizio, dove si trova e come invocarlo. I clienti possono utilizzare queste descrizioni per comprendere come interagire con un servizio specifico.

Brokering and Discovery Services:

In un ambiente distribuito, un broker di oggetti è un intermediario nelle interazioni tra clienti e servizi. Un esempio di tecnologia di intermediazione è un broker di servizi Web.

- **UDDI Framework per Web Services:** Il framework di Universal Description, Discovery, and Integration (UDDI) è utilizzato per l'integrazione dei servizi Web. Si compone di documenti correlati e uno schema XML che definisce un protocollo basato su SOAP per registrare e scoprire i servizi Web.
- **Utilizzo di UDDI:** Un broker di servizi Web può utilizzare il framework UDDI per fornire un meccanismo attraverso il quale i clienti possono trovare dinamicamente i servizi sul Web.

In sostanza, questi servizi di registrazione e scoperta sono fondamentali in un ambiente distribuito per consentire una dinamica individuazione e utilizzo dei servizi. Consentono ai fornitori di servizi di pubblicizzare le loro offerte e ai clienti di trovare e utilizzare dinamicamente i servizi desiderati.



Web Service Protocols and Standards:

1. **UDDI Registry Query:** Il cliente effettua una query al registro UDDI per individuare il servizio desiderato.
2. **Referral to WSDL Document:** Il registro UDDI fornisce al cliente un riferimento al documento WSDL (Web Services Description Language).
3. **Access to WSDL Document:** Il cliente accede al documento WSDL per ottenere informazioni dettagliate su come interagire con il servizio Web.
4. **WSDL Provides Interaction Data:** Il documento WSDL fornisce i dettagli necessari al cliente per interagire con il servizio Web, inclusi i tipi di dati supportati, i metodi disponibili e la struttura delle richieste e delle risposte.
5. **SOAP Message Request:** Il cliente invia una richiesta al servizio Web utilizzando un messaggio SOAP (Simple Object Access Protocol), che è un protocollo basato su XML per la comunicazione tra applicazioni.
6. **SOAP Message Response:** Il servizio Web risponde con un messaggio SOAP che contiene i risultati o le informazioni richieste.

REST (REPRESENTATIONAL STATE TRANSFER)

È un'architettura di sistemi distribuiti che condivide le risorse in rete attraverso l'uso di API (protocollo http)

- **Client-Server:** Esiste una separazione tra il client e il server, con un'interazione pull-based, dove il client richiede e il server risponde.
- **Stateless:** La comunicazione client-server è priva di contesto, il server non memorizza alcun contesto del cliente tra le richieste.
- **Cache:** I client e gli intermediari possono memorizzare in cache le risposte per migliorare le prestazioni.
- **Interfaccia Uniforme:** Tutte le risorse sono accessibili attraverso un'interfaccia generica, ad esempio, utilizzando metodi HTTP standard come GET, POST, PUT e DELETE.
- **Risorse Nominative:** Il sistema è composto da risorse che sono nominate utilizzando un URL (o URI).
- **Rappresentazioni Interconnesse delle Risorse:** Le rappresentazioni delle risorse sono collegate tra loro utilizzando URL, consentendo al client di progredire da uno stato a un altro.

Risorse:

In architettura REST, ogni entità distinguibile è considerata una risorsa. Una risorsa può essere di diversi tipi, come un sito Web, una pagina HTML, un documento XML, un servizio Web, un dispositivo fisico, ecc.

- **Identificazione delle Risorse tramite URL:** Le risorse sono identificate in modo univoco da un URL (Uniform Resource Locator).

SOFTWARE ARCHITECTURAL TRANSACTION PATTERNS

I servizi in un sistema software spesso encapsulano dati o forniscono accesso a dati che i clienti (altre parti del sistema o entità esterne) devono leggere o aggiornare. Molti servizi devono fornire operazioni di aggiornamento coordinate, il che implica che determinate operazioni potrebbero dover essere eseguite insieme per mantenere la coerenza dei dati.

Transazioni:

- Una transazione è una richiesta da parte di un cliente a un servizio.
- Consiste in due o più operazioni che insieme eseguono una singola funzione logica.
- La caratteristica chiave di una transazione è che deve essere completata interamente o per nulla. Questa proprietà è nota come atomicità.

Proprietà ACID delle Transazioni:

- **Atomicità (A):** Le transazioni sono unità indivisibili di lavoro. Sono o completamente completate (validate) o annullate (rollback), il che significa che se una parte della transazione fallisce, l'intera transazione viene annullata per mantenere la coerenza.
- **Coerenza (C):** Dopo che una transazione viene eseguita, il sistema deve trovarsi in uno stato coerente. Questo garantisce che i dati rispettino le regole predefinite e i vincoli di integrità.
- **Isolamento (I):** Il comportamento di una transazione non deve essere influenzato da altre transazioni. Ogni transazione dovrebbe essere eseguita in isolamento dalle altre, impedendo interferenze reciproche.
- **Durabilità (D):** Le modifiche apportate da una transazione sono permanenti dopo il completamento. Queste modifiche devono sopravvivere a guasti di sistema, garantendo che i dati persistano anche in caso di arresto improvviso o perdita di alimentazione.

In sintesi, queste proprietà ACID sono fondamentali per garantire l'affidabilità, l'integrità e la coerenza dei dati in un sistema distribuito in cui più servizi potrebbero interagire contemporaneamente. L'obiettivo è mantenere un alto livello di integrità dei dati e prevenire problemi che potrebbero derivare dall'accesso e dalla modifica concorrente dei dati da parte di diverse parti del sistema.

Protocollo del Commit a Due Fasi:

Il protocollo a due fasi (Two-Phase Commit Protocol) è un meccanismo utilizzato per gestire transazioni atomiche in sistemi distribuiti. Una transazione atomica è un insieme di operazioni che devono essere eseguite in modo atomico, cioè in modo tale che o tutte le operazioni vengano completate con successo o nessuna di esse venga eseguita affatto.

- **CommitCoordinator (Coordinatore di Commit):** Questo è il componente responsabile della coordinazione della transazione. Il suo ruolo principale è garantire che tutti i partecipanti alla transazione siano d'accordo sul commit (impegno) o sull'annullamento della transazione. Il CommitCoordinator sincronizza le fasi di preparazione e di commit tra i partecipanti.
- **Partecipanti:** Ogni nodo coinvolto nella transazione è rappresentato da un servizio partecipante.
 - Nel contesto di un trasferimento bancario, ci sono due partecipanti principali:
 - **firstBankService (Primo servizio bancario):** Questo servizio gestisce l'account dal quale viene trasferita la somma di denaro (Account di partenza).
 - **secondBankService (Secondo servizio bancario):** Questo servizio gestisce l'account a cui viene trasferita la somma di denaro (Account di destinazione).
- **Fasi del protocollo a due fasi:**
 - **Fase di preparazione:** Durante questa fase, i partecipanti preparano la transazione e comunicano al CommitCoordinator che sono pronti per il commit. Il CommitCoordinator raccoglie i consensi da tutti i partecipanti.
 - **Fase di commit:** Se tutti i partecipanti sono d'accordo, il CommitCoordinator emette un segnale di commit a tutti i partecipanti, indicando loro di eseguire definitivamente la transazione. In caso contrario, il CommitCoordinator emette un segnale di rollback (annullamento) per annullare la transazione.

L'obiettivo del Two-Phase Commit Protocol è garantire la coerenza delle transazioni distribuite. Tuttavia, è importante notare che questo protocollo potrebbe diventare un punto critico in termini di prestazioni, poiché richiede la sincronizzazione tra tutti i partecipanti prima di procedere con il commit o il rollback. Inoltre, può essere vulnerabile a situazioni di fallimento e richiede strategie di gestione delle eccezioni.

Compound Transaction Pattern:

Il modello di transazione composta (Compound Transaction Pattern) è un approccio che offre una maggiore flessibilità rispetto alle transazioni piatte (flat transactions) "tutto-o-niente". In una transazione piatta, o tutte le operazioni vengono eseguite con successo, o nessuna di esse viene eseguita affatto. D'altra parte, una transazione composta consente un rollback parziale, il che significa che è possibile annullare solo una parte della transazione senza dover annullare l'intera transazione.

- **Caratteristiche della transazione piatta:** La transazione bancaria di trasferimento menzionata precedentemente è un esempio di transazione piatta. Se qualcosa va storto durante qualsiasi fase della transazione, è necessario annullare l'intera operazione.
- **Caratteristiche della transazione composta:** La transazione composta è più flessibile e consente rollback parziali. Questo modello è utile quando la richiesta di transazione del cliente può essere suddivisa in transazioni atomiche più piccole, in cui ogni transazione atomica può essere eseguita e annullata separatamente.
- **Esempio:** Immaginiamo un agente di viaggi che gestisce una prenotazione composta, costituita da una prenotazione aerea, una prenotazione alberghiera e una prenotazione di un'auto a noleggio. Trattando questa prenotazione come una transazione composta, ogni parte (prenotazione aerea, alberghiera e auto a noleggio) è gestita come una transazione piatta separata. Se ad esempio c'è un problema con la prenotazione dell'auto a noleggio, è possibile annullare solo quella parte della transazione senza dover annullare anche la prenotazione aerea e alberghiera.

In sintesi, il Compound Transaction Pattern offre maggiore flessibilità nella gestione delle transazioni, consentendo rollback parziali e trattando una transazione complessa come una serie di transazioni atomiche più piccole.

Long-Living Transaction Pattern:

Il modello di transazione a lunga durata (Long-Living Transaction Pattern) è un approccio progettato per gestire transazioni che coinvolgono interazioni umane e possono richiedere un tempo prolungato, possibilmente indefinito, per essere completate. Queste transazioni sono particolarmente complesse a causa dell'imprevedibilità del comportamento umano e richiedono intervalli di tempo tra le diverse fasi in cui una decisione umana è necessaria.

- **Definizione di transazione a lunga durata:** Una transazione a lunga durata è un tipo di transazione che coinvolge attivamente una persona nel processo decisionale. A differenza di transazioni più standard che possono essere eseguite in modo automatizzato e relativamente rapido, le transazioni a lunga durata possono richiedere tempi prolungati a causa della variabilità e imprevedibilità delle azioni umane coinvolte.
- **Suddivisione della transazione:** Il modello di transazione a lunga durata affronta questa complessità suddividendo la transazione in due o più transazioni separate, solitamente due. Queste transazioni sono organizzate in modo che intervalli di tempo significativi separino le diverse fasi, consentendo il coinvolgimento umano e la presa di decisioni tra le diverse fasi.
- **Esempio:** Immaginiamo una transazione in cui un cliente sta richiedendo un prestito presso una banca. La prima transazione potrebbe coinvolgere la richiesta del prestito e la raccolta di informazioni iniziali. Successivamente, la transazione potrebbe essere sospesa, consentendo al sistema di attendere una risposta umana (ad esempio, l'approvazione o meno del prestito da parte di un analista del credito). Una volta che la decisione umana è stata presa, la seconda transazione può essere eseguita, comportando l'erogazione del prestito o la notifica di rifiuto.

In sintesi, il modello di transazione a lunga durata è progettato per affrontare la complessità delle transazioni che richiedono decisioni umane e possono estendersi per periodi di tempo significativi. La suddivisione della transazione consente di gestire in modo efficiente le pause tra le fasi e di adattarsi a tempi imprevedibili.

NEGOTIATION PATTERN

Il modello di negoziazione (Negotiation Pattern) è un approccio utilizzato in alcune architetture orientate ai servizi (SOA) per coordinare le interazioni tra servizi. Questo modello coinvolge negoziazioni tra agenti software, in cui un agente del cliente propone un'azione a un agente del servizio, che a sua volta cerca di soddisfare la proposta del cliente.

- **Agenti coinvolti:**
 - **Cliente agente:** Rappresenta l'utente e agisce a suo nome. Propone un'azione o richiesta al servizio agente.
 - **Servizio agente:** Rappresenta il servizio e agisce a suo nome. Cerca di soddisfare la proposta del cliente e può interagire con altri servizi per farlo.

Possibili azioni del cliente agente:

- **Proporre un servizio:** Il cliente agente propone un servizio al servizio agente, indicando che è disposto a negoziare.
- **Richiedere un servizio:** Il cliente agente chiede specificamente un servizio senza possibilità di negoziazione.
- **Rifiutare un'offerta di servizio:** Il cliente agente può rifiutare un'offerta fatta dal servizio agente se non è soddisfatto.

Possibili azioni del servizio agente:

- **Offrire un servizio:** In risposta a una proposta del cliente, il servizio agente offre una controproposta che cerca di soddisfare i requisiti del cliente.

- **Rifiutare una richiesta/proposta del cliente:** Il servizio agente può rifiutare la richiesta o proposta del cliente se non può soddisfarla o se le condizioni non sono accettabili.
- **Accettare una richiesta/proposta del cliente:** Se il servizio agente può soddisfare la richiesta o proposta del cliente, può accettarla e procedere con l'azione richiesta.

In sintesi, il modello di negoziazione è un approccio in cui un agente del cliente e un agente del servizio collaborano per giungere a un accordo riguardo a un'azione o una richiesta di servizio. Questo modello fornisce flessibilità consentendo la negoziazione di proposte, controproposte e l'esecuzione di azioni coerenti con le decisioni prese durante la negoziazione.

SERVICE INTERFACE DESIGN IN SOA

La progettazione dell'interfaccia del servizio in un'architettura orientata ai servizi (SOA) è un passo cruciale nel processo di creazione di nuovi servizi.

- **Iniziale progettazione dei servizi:** Inizialmente, i nuovi servizi sono progettati utilizzando criteri di strutturazione di classi. Questo significa che i servizi sono concepiti inizialmente considerando la struttura delle classi, che può includere attributi e comportamenti associati.
- **Modellazione dell'interazione dinamica:** Durante la modellazione dell'interazione dinamica, viene analizzato come avvengono le interazioni tra gli oggetti del cliente (client objects) e gli oggetti del servizio (service objects). Si identificano i flussi di dati e le comunicazioni tra il client e il servizio. Questo passo è cruciale per comprendere come il cliente e il servizio si scambiano informazioni durante l'esecuzione.
- **Progettazione delle operazioni del servizio:** L'approccio utilizzato per progettare le operazioni del servizio è simile a quello utilizzato nella progettazione delle interfacce di classe. L'attenzione si sposta dalla struttura interna del servizio all'interfaccia che il servizio offre per interagire con il mondo esterno, in particolare con i client.
- **Analisi dei messaggi in arrivo:** La progettazione delle operazioni del servizio si basa sui messaggi che arrivano al servizio. Questi messaggi rappresentano le richieste provenienti dai clienti o le informazioni che il servizio deve elaborare. Durante l'analisi dei messaggi, si determina il nome dell'operazione del servizio, nonché i parametri di input e output associati.

In sintesi, la progettazione dell'interfaccia del servizio in SOA segue una metodologia che inizia con la strutturazione delle classi, si sposta poi verso la comprensione delle interazioni dinamiche tra client e servizio e si conclude con la progettazione delle operazioni del servizio basata sui messaggi in arrivo. Questo approccio mira a garantire che l'interfaccia del servizio sia ben definita e in grado di soddisfare le esigenze degli utenti e degli altri servizi nell'ambiente SOA.

SERVICE COORDINATION IN SOA

La coordinazione dei servizi è un aspetto fondamentale nelle applicazioni basate su architetture orientate ai servizi (SOA) che coinvolgono più servizi. La coordinazione è necessaria per garantire che i servizi lavorino insieme in modo efficace e coerente. Ci sono due approcci principali per la coordinazione dei servizi in SOA:

- **Orchestrazione:** L'orchestrazione coinvolge la coordinazione di più servizi attraverso una logica di flusso di lavoro centralmente controllata. In un processo di orchestrazione, un componente centrale, noto come orchestratore, definisce e controlla il flusso delle attività e delle interazioni tra i servizi partecipanti. L'orchestrazione è particolarmente utile quando si desidera creare nuove applicazioni componendo e coordinando servizi esistenti in un flusso di lavoro specifico. Questo approccio favorisce la riutilizzabilità dei servizi, in quanto consente di incorporare servizi esistenti in nuove applicazioni senza dover modificare i servizi stessi.
- **Coreografia:** La coreografia fornisce una coordinazione distribuita tra i servizi senza un controllo centralizzato. In un modello di coreografia, ciascun servizio è consapevole delle interazioni che deve eseguire in risposta a eventi specifici e collabora autonomamente con gli altri servizi coinvolti. La coreografia è spesso utilizzata quando la coordinazione coinvolge diversi servizi appartenenti a organizzazioni aziendali separate. In questo modo, la coreografia facilita la collaborazione tra servizi forniti da diverse organizzazioni. A differenza dell'orchestrazione, la coreografia è caratterizzata da un controllo distribuito, senza un orchestratore centrale.

Terminologia: Poiché i termini orchestrazione e coreografia vengono spesso utilizzati in modo intercambiabile, viene utilizzato il termine più generale di "coordinamento" per descrivere il controllo e la sequenza di diversi servizi come necessario per un'applicazione SOA.

Pattern di transazione per la coordinazione del servizio: I pattern di transazione possono essere utilizzati anche per coordinare i servizi, garantendo la coerenza e la correttezza delle transazioni distribuite.

LEGGE DI DEMETER

La Legge di Demeter (Law of Demeter) è un principio di progettazione del software che promuove la riduzione dell'accoppiamento tra le classi. Secondo questa legge, un metodo di una classe dovrebbe interagire solo con oggetti strettamente correlati e non dovrebbe accedere a oggetti che appartengono ad altre classi. In altre parole, un oggetto dovrebbe comunicare solo con i suoi vicini immediati e non "parlare agli estranei". Ciò significa che un oggetto dovrebbe limitare le sue dipendenze agli oggetti con cui ha una relazione diretta, come il proprio oggetto stesso, gli oggetti passati come argomenti al metodo, gli oggetti a cui fa riferimento tramite i suoi attributi, gli oggetti creati dal metodo o gli oggetti a cui fa riferimento tramite una variabile globale.

La Legge di Demeter mira a migliorare la manutenibilità e la flessibilità del codice, riducendo la dipendenza tra le classi e favorendo una struttura più modulare e comprensibile.

CLASSI STRUTTURATE

UML (Unified Modeling Language) è un linguaggio di modellazione visuale ampiamente utilizzato nel campo dell'ingegneria del software per descrivere, progettare e documentare i sistemi software. Una "structured class" in UML è una classe che ha una struttura interna complessa, composta da ruoli o parti che realizzano il suo comportamento.

Structured Class e Struttura Interna:

Una classe strutturata è una classe che non è semplicemente un insieme di attributi e metodi, ma ha una struttura interna più complessa. Questa struttura interna è formata da ruoli o parti che contribuiscono a definire il comportamento complessivo della classe. Quindi, al contrario di una classe "semplice", una structured class contiene componenti interne che contribuiscono in modo specifico al funzionamento complessivo della classe.

Parti Strutturate:

Le parti che costituiscono la struttura interna di una classe strutturata possono essere anch'esse classi strutturate. Questo consente una strutturazione gerarchica, dove le parti possono essere ulteriormente scomposte in sotto-parti, permettendo una chiara espressione di modelli multilivello.

Connettore e Associazione:

- In UML, un connettore viene utilizzato per rappresentare un'associazione in un contesto specifico.
- L'associazione rappresenta una relazione tra le parti o ruoli all'interno della classe strutturata.
- I connettori sono utilizzati per visualizzare i percorsi di comunicazione tra le diverse parti della classe strutturata.

In sintesi, una classe strutturata in UML è un modo di modellare classi più complesse, con una struttura interna che coinvolge ruoli, parti e, se necessario, una gerarchia di classi strutturate. L'utilizzo di connettori consente di rappresentare le relazioni e le comunicazioni tra le diverse parti della classe.

Usi delle Classi Strutturate:

- **Blocchi Fondamentali di un'applicazione:**
 - Le classi strutturate possono essere utilizzate come mattoni fondamentali per la costruzione di un'applicazione.
 - Offrono una rappresentazione grafica degli elementi di progettazione.
 - Possono nascondere i dettagli di implementazione, permettendo un'astrazione potente.
- **Strumento di Astrazione Potente:**
 - Le classi strutturate servono come uno strumento di astrazione potente in quanto lo stesso costrutto può essere applicato a diversi livelli semantici.
 - Questo facilita la progettazione e la comprensione di sistemi complessi.
- **Comunicazione Chiara e Comprensione dell'Architettura di Sistema:**
 - Consentono una comunicazione chiara e una comprensione dell'architettura di sistema.
 - L'incapsulamento rigoroso del comportamento aiuta a definire in modo chiaro le interazioni.
- **Interazioni Limitate a Comunicazioni Basate su Messaggi:**
 - Le interazioni sono limitate a comunicazioni basate su messaggi, che vengono passati attraverso interfacce esterne (porte).
 - Ciò promuove un approccio a messaggi per la comunicazione tra parti del sistema.

Unità di Design Autonoma:

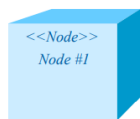
- Una classe strutturata è considerata un'unità di design autonoma.
- L'incapsulamento rigoroso assicura che l'implementazione sia indipendente dall'ambiente circostante.
- Le porte possono svolgere un ruolo di mediazione bidirezionale.

Progettazione e Test Unitario Indipendenti: Le classi strutturate possono essere progettate e testate in modo indipendente, poiché l'incapsulamento garantisce che l'ambiente veda solo la porta della classe strutturata.

In sintesi, l'uso di classi strutturate fornisce un modo organizzato ed efficiente per progettare, comunicare e implementare sistemi complessi, con un'enfasi particolare sulla chiarezza, sull'incapsulamento e sulla progettazione indipendente delle unità.

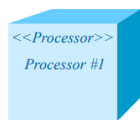
DIAGRAMMA DI DISTRIBUZIONE – DEPLOYMENT DIAGRAM

La **configurazione della piattaforma** descrive la soluzione hardware/software che definisce come la funzionalità di un sistema può essere distribuita su nodi fisici. Questo spiega la relazione tra gli elementi del modello e la loro implementazione, nonché il loro deployment. La configurazione della piattaforma viene ottenuta definendo la configurazione attraverso l'uso di un diagramma di deployment e allocando gli elementi di sistema (artefatti) ai nodi di questo diagramma.

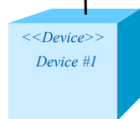


Deployment Model Modeling Elements (Elementi di Modellazione del Modello di Deployment):

- **Node (Nodo):** Rappresenta una risorsa computazionale in esecuzione a livello fisico, come un nodo di elaborazione o un dispositivo. I nodi possono essere processori che eseguono il software di sistema o dispositivi controllati da un processore.
- **Connection (Connessione):** Rappresenta un meccanismo di comunicazione, inclusi il mezzo fisico e il protocollo software.



Connection



Cos'è un Nodo? Rappresenta una risorsa computazionale in esecuzione e generalmente ha almeno la memoria e spesso la capacità di elaborazione. Può essere un dispositivo fisico o un ambiente di esecuzione specifico.

Cos'è un Connector? Un connector rappresenta un meccanismo di comunicazione descritto da un mezzo fisico e da un protocollo software.

Cos'è il Deployment? Il deployment è l'assegnazione o la mappatura degli artefatti* software ai nodi fisici durante l'esecuzione. Gli artefatti sono le entità che vengono deployate sui nodi fisici, come processi assegnati a computer o file, eseguibili, tabelle del database, pagine web, ecc.

Cos'è la Manifestazione? La manifestazione è l'implementazione fisica di un elemento di modello come un artefatto. È la relazione tra l'elemento di modello e l'artefatto che lo implementa. Gli elementi di modello sono tipicamente implementati come un insieme di artefatti, come file sorgenti, file eseguibili, file di documentazione, ecc.

Cos'è una Deployment Specification? Una Deployment Specification è una specifica dettagliata dei parametri del deployment di un artefatto su un nodo. Può definire valori che parametrizzano l'esecuzione dell'artefatto.

**** Artefatti** = entità che vengono distribuite sui nodi fisici**

DESIGN PATTERNS

La progettazione del software orientato agli oggetti è un processo complesso che coinvolge la creazione di programmi informatici organizzati in modo che le entità fondamentali siano gli "oggetti", che raggruppano dati e funzionalità correlate. Ogni oggetto è un'istanza di una "classe", che definisce la struttura e il comportamento dell'oggetto.

Una delle maggiori difficoltà che il progettista deve affrontare è l'individuazione di un insieme di oggetti. Gli oggetti devono essere identificati correttamente, devono essere riusabili e le relazioni tra di essi devono essere ben definite. Questo processo richiede una comprensione approfondita del dominio del problema e delle relazioni tra le entità coinvolte.

Caratteristiche:

- Gli oggetti dovrebbero rappresentare soluzioni a problemi comuni che si presentano durante lo sviluppo del software. Favoriscono il riuso del codice, impedendo al progettista di dover "reinventare la ruota" ogni volta che affronta un nuovo problema. Consentono di imparare dagli altri, evitando errori comuni e promuovendo la condivisione di conoscenze.
- **Linguaggio comune:** Gli oggetti consentono di definire un linguaggio comune che semplifica la comunicazione tra coloro che lavorano al progetto. Questo riduce la possibilità di fraintendimenti e promuove una comprensione più chiara tra i membri del team.
- **Strutture valide:** Indirizzano verso la scrittura di codice che utilizza strutture valide, migliorando la qualità complessiva del software.
- **Buona progettazione:** In genere, portano a una buona progettazione del software, semplificando la manutenzione in tutte le sue forme (adattativa, perfetta, preventiva e correttiva).
- **Limitazioni:** Nonostante i vantaggi, la progettazione orientata agli oggetti non risolve tutti i problemi.

CLASSIFICAZIONE

Classificazione in base allo scopo (purpose):

- **Creazionali:** Gestiscono il processo di creazione di oggetti.
- **Strutturali:** Definiscono la struttura del sistema in termini di composizione di classi e oggetti. Si basano su concetti di ereditarietà e polimorfismo.
- **Comportamentali:** Modellano il comportamento del sistema, definendo le responsabilità delle sue componenti e le modalità di interazione.

Classificazione in base al raggio di azione (scope):

- **Classi:** Riguarda le relazioni fra classi e sottoclassi, spesso basate su concetti di ereditarietà. Queste relazioni sono statiche e definite a tempo di compilazione.
- **Oggetti:** Riguarda le relazioni tra oggetti, che possono cambiare durante l'esecuzione del programma. Sono più dinamiche rispetto alle relazioni tra classi.

		Scopo		
		Creazionale	Strutturale	Comportamentale
Raggio d'azione	Classi	Factory Method	Adapter (class)	Interpreter Template Method
	Oggetti	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Facade Flyweight Proxy	Chain of responsibility Iterator Mediator Memento Observer State Strategy Visitor

DESCRIZIONE

La descrizione dei design pattern fornisce un approfondimento dettagliato su come sono strutturati e come possono essere utilizzati nei processi di progettazione del software.

- **Nome e Classificazione:** Il nome del design pattern illustra l'essenza del pattern, fornendo un linguaggio comune tra i progettisti. La classificazione identifica il pattern in termini di scopo e raggio di azione, aiutando a comprendere quando e come applicarlo.
- **Motivazione:** La motivazione fornisce uno scenario astratto che descrive il problema al quale il design pattern mira a rispondere. Può anche includere precondizioni necessarie per l'applicabilità del pattern.
- **Applicabilità:** Questa sezione descrive le situazioni specifiche in cui il design pattern può essere applicato con successo. Aiuta a capire in quali contesti il pattern può essere più efficace.
- **Struttura:** Rappresenta graficamente la configurazione di elementi che compongono il design pattern. Questo include relazioni, responsabilità e collaborazioni tra classi e oggetti. È uno schema di soluzione astratta, non una soluzione specifica per un progetto particolare.
- **Partecipanti:** Indica le classi e gli oggetti che fanno parte del design pattern, specificando le loro responsabilità. Aiuta a comprendere come le diverse entità interagiscono all'interno del pattern.

- **Conseguenze:** Descrive i risultati che si ottengono applicando il design pattern. Questi risultati possono includere vantaggi e svantaggi associati all'uso del pattern.
- **Implementazioni:** Fornisce tecniche e suggerimenti pratici per implementare il design pattern. Questo può includere considerazioni specifiche del linguaggio di programmazione o altre raccomandazioni.
- **Codice di esempio:** Presenta frammenti di codice che illustrano come implementare il design pattern in un determinato linguaggio di programmazione. Questi esempi pratici aiutano i progettisti a comprendere l'implementazione concreta del pattern.
- **Usi conosciuti:** Fornisce esempi di applicazione del design pattern in sistemi reali. Questi esempi pratici dimostrano come il pattern può essere utilizzato in contesti del mondo reale.
- **Pattern correlati:** Indica altri design pattern correlati, offrendo un collegamento a soluzioni simili o complementari che possono essere considerate durante il processo di progettazione.

In sintesi, la descrizione dei design pattern fornisce una guida completa per comprendere, applicare e implementare questi schemi di soluzione nel contesto dello sviluppo del software.

FRAMEWORK

Un framework è un insieme di strumenti e convenzioni che forniscono una struttura per lo sviluppo di software. A differenza di una libreria, che è un insieme di funzioni che possono essere chiamate dal tuo codice, un framework è più simile a un'architettura predefinita che fornisce linee guida su come organizzare e sviluppare il tuo software.

- **Design Riutilizzabile:** Un framework offre un design riutilizzabile per un sistema o una parte di esso. Questo significa che è possibile utilizzare il framework come base per progetti simili senza dover reinventare la ruota ogni volta.
- **Classi Astratte:** Nel contesto dei framework, le classi astratte sono classi che contengono almeno un metodo astratto, ovvero un metodo che deve essere implementato dalle classi derivate. Le classi astratte forniscono uno scheletro o un template per le implementazioni specifiche.
- **Personalizzazione da Parte dello Sviluppatore:** Gli sviluppatori utilizzano il framework come uno scheletro per la propria applicazione. Personalizzano il framework implementando le classi astratte e le interfacce secondo le esigenze specifiche del loro progetto.
- **Struttura Statica del Sistema:** Il framework definisce la struttura statica del sistema, cioè l'organizzazione e la relazione tra le diverse parti del software.

Vantaggi:

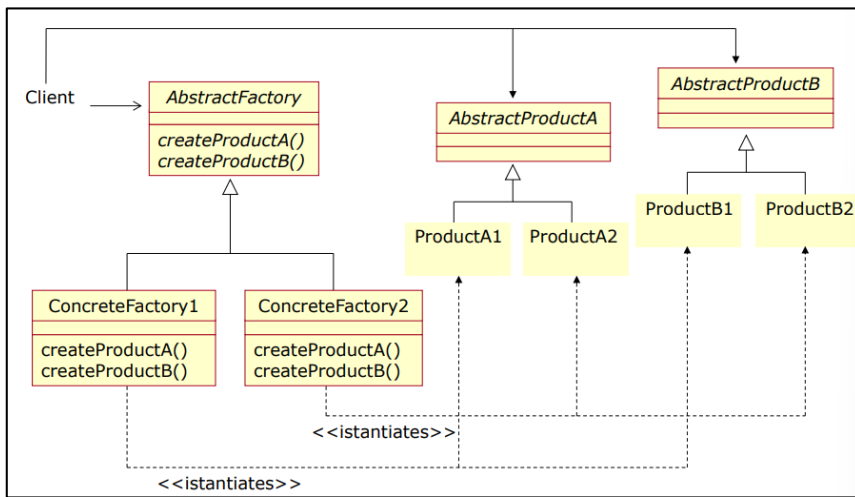
- **Riuso del Design:** Gli sviluppatori possono riutilizzare il design del framework come punto di partenza per sviluppare nuove applicazioni.
- **Riuso del Codice:** Il codice già implementato nel framework può essere riutilizzato, contribuendo a una maggiore efficienza nello sviluppo.
- **Progettazione Orientata agli Oggetti:** I framework sono spesso basati su principi di progettazione orientata agli oggetti, che promuovono la modularità e la manutenibilità del codice.

Classe Astratta:

Una classe astratta è una classe che contiene almeno un metodo astratto, cioè un metodo che è dichiarato, ma non è implementato nella classe astratta stessa. Le classi astratte forniscono uno scheletro o un modello per le classi derivate che devono implementare i metodi astratti.

Pattern:

I pattern sono spesso utilizzati nella progettazione di framework. I pattern sono soluzioni generali per problemi ricorrenti che si verificano durante lo sviluppo del software. Essi possono essere considerati come "mattoni" che vengono utilizzati nella costruzione di un framework, fornendo soluzioni comuni e testate per determinati tipi di problemi.



ABSTRACT FACTORY

L'Abstract Factory è un pattern creazionale che fornisce un'interfaccia per la creazione di famiglie di oggetti correlati senza specificarne le classi concrete. Questo è particolarmente utile quando si desidera sviluppare un sistema indipendente dalle modalità di creazione dei prodotti con cui opera.

Motivazione:

Un esempio comune di utilizzo dell'Abstract Factory è nella realizzazione di strumenti per lo sviluppo di interfacce utente (UI) che devono supportare diversi tipi di "look & feel". Per garantire la portabilità di un'applicazione tra

diverse modalità di interfaccia utente, è importante che gli oggetti non siano direttamente "cablati" nel codice, ma che l'applicazione utilizzi un'interfaccia comune fornita dalla factory.

Classificazione:

L'Abstract Factory rientra nella categoria creazionale basato su oggetti, poiché si concentra sulla creazione di oggetti, in particolare di famiglie di oggetti correlati.

Applicabilità:

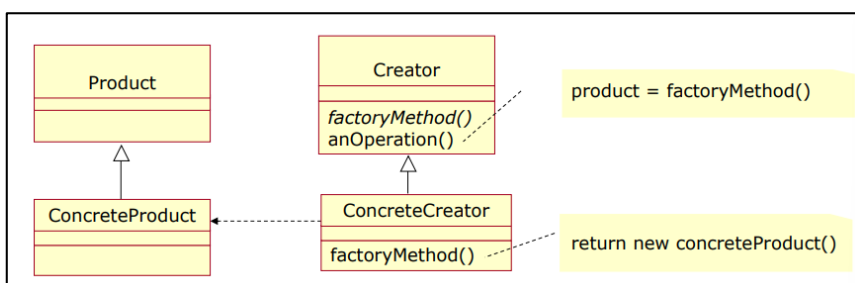
- A sistemi che devono essere indipendenti dalle modalità di creazione dei prodotti con cui operano.
- A sistemi che devono poter essere configurati per utilizzare famiglie di prodotti diverse.
- Il client (l'applicazione) non deve essere legato a una specifica famiglia di oggetti.

Partecipanti:

- **AbstractFactory e ConcreteFactory:**
 - L'AbstractFactory dichiara un'interfaccia per la creazione di prodotti astratti.
 - La ConcreteFactory implementa l'interfaccia dell'AbstractFactory per creare prodotti concreti.
- **AbstractProduct e ConcreteProduct:**
 - L'AbstractProduct dichiara un'interfaccia comune per un tipo di prodotto.
 - La ConcreteProduct implementa l'interfaccia dell'AbstractProduct e fornisce l'implementazione concreta di un prodotto.
- **Applicazione Client:** Utilizza le interfacce definite da AbstractFactory e AbstractProduct per interagire con i prodotti senza dover conoscere le classi concrete.

Conseguenze:

- Le classi concrete sono isolate e sotto controllo.
- La famiglia di prodotti può essere cambiata rapidamente perché l'intera factory compare in un unico punto del codice.
- Aggiungere nuove famiglie di prodotti richiede la ricompilazione poiché l'insieme di prodotti gestiti è legato all'interfaccia della factory.



FACTORY METHOD

Il Factory Method è un pattern creazionale che ha come scopo principale quello di definire un'interfaccia per creare un oggetto in una classe, lasciando alle sottoclassi il compito di modificare il tipo di oggetto che verrà istanziato. Questo consente di decidere dinamicamente, a tempo di esecuzione, quale specifico oggetto deve essere creato.

Motivazione:

Nel contesto dei framework, le classi astratte definiscono spesso le relazioni tra gli elementi del dominio, ma è compito delle sottoclassi fornire l'implementazione concreta degli oggetti. Il Factory Method è utile quando una classe non può sapere in anticipo quali classi di oggetti deve creare o quando si desidera che le sottoclassi scelgano gli oggetti da creare.

Classificazione:

Il Factory Method è un pattern creazionale basato su classi, poiché coinvolge la definizione di classi per creare oggetti.

Applicabilità:

- Una classe non è in grado di sapere in anticipo le classi di oggetti che deve creare.
- Una classe vuole che le sue sottoclassi scelgano gli oggetti da creare.
- Le classi delegano la responsabilità di creazione.

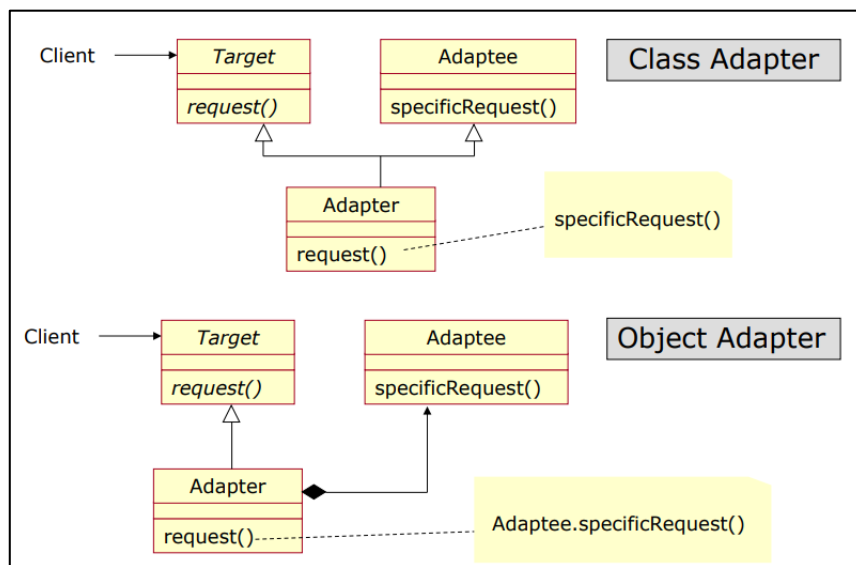
Partecipanti:

- **Product e ConcreteProduct:**
 - `Product` è l'interfaccia comune per gli oggetti creati dal Factory Method.
 - `ConcreteProduct` è un'implementazione concreta di `Product`.
- **Creator e ConcreteCreator:**
 - `Creator` dichiara il Factory Method, che restituisce un oggetto di tipo `Product`.
 - `ConcreteCreator` implementa il Factory Method e crea un oggetto di tipo `ConcreteProduct`.

Conseguenze:

- Elimina la necessità di riferirsi a classi dipendenti dall'applicazione all'interno del codice.
- Fornisce una maggiore flessibilità alle sottoclassi per cambiare il tipo di oggetto creato senza modificare il codice client.

In breve, il Factory Method è utile quando si desidera che le sottoclassi forniscano l'implementazione concreta degli oggetti da creare, offrendo una maggiore flessibilità nel cambiare il tipo di oggetto istanziato senza modificare il codice cliente.



ADAPTER

L'Adapter è un pattern strutturale utilizzato quando si ha una classe con un'interfaccia incompatibile con quella richiesta da un client, e si desidera adattare l'interfaccia della classe esistente in modo che possa essere utilizzata dal client senza modificare il codice sorgente della classe stessa. In altre parole, fornisce un modo per far collaborare classi con interfacce diverse che altrimenti non potrebbero lavorare insieme.

Motivazione:

Immagina di avere un editor che consente di disegnare e comporre elementi grafici, dove l'astrazione chiave è rappresentata da un singolo oggetto grafico. Ora, se vuoi integrare un nuovo componente che, per qualche motivo, ha

un'interfaccia incompatibile con l'editor, puoi utilizzare l'Adapter per creare un ponte tra l'interfaccia del nuovo componente (Adaptee) e quella richiesta dall'editor (Target).

Applicabilità:

Si usa quando si vuole riutilizzare una classe esistente, ma la sua interfaccia è incompatibile con quella desiderata.

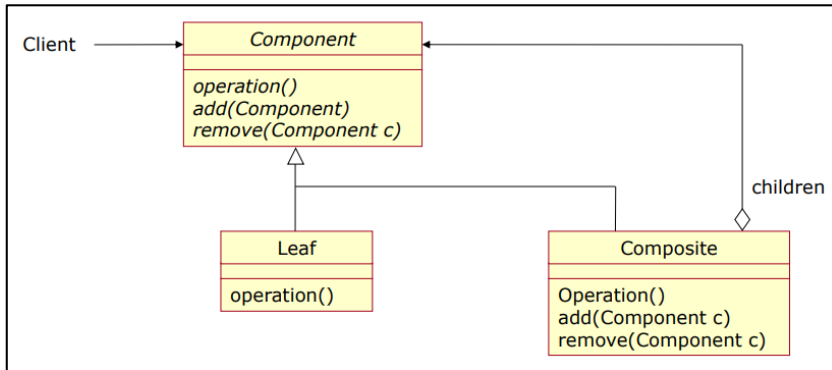
Partecipanti:

- **Client:** È il componente che richiede un'interfaccia specifica.
- **Target:** È l'interfaccia desiderata dal client. L'Adapter implementa questa interfaccia.
- **Adapter ed Adaptee:**
 - **Adapter:** È la classe che adatta l'interfaccia dell'Adaptee in quella richiesta dal Target.
 - **Adaptee:** È la classe esistente con un'interfaccia incompatibile che deve essere adattata.

Conseguenze:

È necessario prendere in considerazione l'effort necessario all'adattamento. Creare un Adapter può richiedere lavoro aggiuntivo a seconda della complessità dell'Adaptee e del livello di adattamento richiesto.

In sintesi, l'Adapter è un pattern utile quando si ha una classe esistente con un'interfaccia incompatibile e si desidera riutilizzarla in un contesto in cui si richiede un'interfaccia diversa. L'Adapter funge da intermediario, consentendo al client di interagire con l'Adaptee attraverso un'interfaccia compatibile.



COMPOSITE

Il Composite è un pattern strutturale utilizzato per creare strutture gerarchiche di oggetti, in modo che oggetti singoli (Leaf) e oggetti composti (Composite) possano essere trattati in modo uniforme. Questo è particolarmente utile in situazioni in cui si desidera manipolare una gerarchia di oggetti in modo uniforme, ad esempio nelle applicazioni grafiche in cui si devono gestire sia le forme geometriche di base che gli oggetti più complessi costruiti da queste forme.

Motivazione:

Nei contesti come gli editor grafici, è comune avere oggetti singoli come linee o cerchi, ma anche oggetti composti come gruppi di figure. Il Composite consente di trattare entrambi i tipi di oggetti in modo uniforme, semplificando il codice del client.

Classificazione:

Il Composite è un pattern strutturale basato su oggetti, poiché coinvolge la composizione di oggetti per formare strutture più complesse.

Applicabilità:

Si usa quando si vogliono rappresentare gerarchie di oggetti in modo che oggetti semplici e oggetti composti siano trattati in modo uniforme.

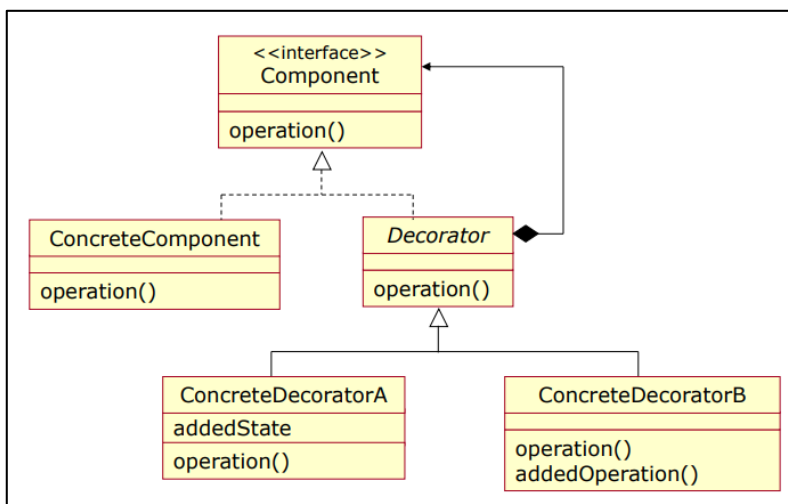
Partecipanti:

- **Component e Composite:**
 - 'Component' è l'interfaccia comune per tutti gli oggetti, sia Leaf che Composite.
 - 'Composite' è l'oggetto composto, che può contenere altri oggetti Component.
- **Leaf:** Rappresenta gli oggetti "foglia" o oggetti atomici che non hanno sottocomponenti.
- **Client:** Utilizza l'interfaccia comune 'Component' per interagire con oggetti sia Leaf che Composite.

Conseguenze:

- I client sono semplificati perché gli oggetti singoli e quelli composti sono trattati allo stesso modo.
- L'aggiunta di nuovi oggetti 'Leaf' o 'Composite' è semplice, e questi possono sfruttare il codice dell'applicazione client già esistente.
- Può rendere il sistema troppo generico. Non è possibile imporre restrizioni su quali tipi di oggetti un oggetto composto può contenere.

In sintesi, il Composite è utile quando si desidera trattare in modo uniforme sia gli oggetti singoli che quelli composti. La sua struttura gerarchica consente una maggiore flessibilità nella creazione e manipolazione di strutture complesse di oggetti.



DECORATOR

Il pattern Decorator è un pattern strutturale il cui scopo principale è aggiungere dinamicamente funzionalità (responsabilità) ad un oggetto senza modificare la sua interfaccia. Questo è particolarmente utile quando si vuole estendere il comportamento di un oggetto in modo flessibile e dinamico, senza ricorrere all'eccessivo uso di sottoclassi.

Motivazione:

Uno scenario classico di applicazione per il pattern Decorator si trova nell'implementazione di interfacce utente, dove si possono aggiungere dinamicamente funzionalità come un testo scorrevole o un particolare

bordo a un oggetto senza modificare la sua struttura di base. L'alternativa statica, come il subclassing, può essere limitata poiché è a livello di classe e non di oggetto.

Classificazione:

Il Decorator è un pattern strutturale basato su oggetti, poiché coinvolge la composizione di oggetti per estendere le loro funzionalità.

Applicabilità:

- Si applica quando è necessario aggiungere responsabilità agli oggetti in modo trasparente e dinamico.
- Si applica quando il subclassing non è adatto o potrebbe generare una gerarchia di classi troppo complessa.

Partecipanti:

- **Component e ConcreteComponent:**
 - `Component` definisce l'interfaccia comune per gli oggetti che possono essere decorati.
 - `ConcreteComponent` è l'oggetto di base che può essere decorato.
- **Decorator e ConcreteDecorator(s):**
 - `Decorator` è l'interfaccia comune per tutti i decorator.
 - `ConcreteDecorator` è l'implementazione concreta del decorator. Aggiunge responsabilità all'oggetto di base.

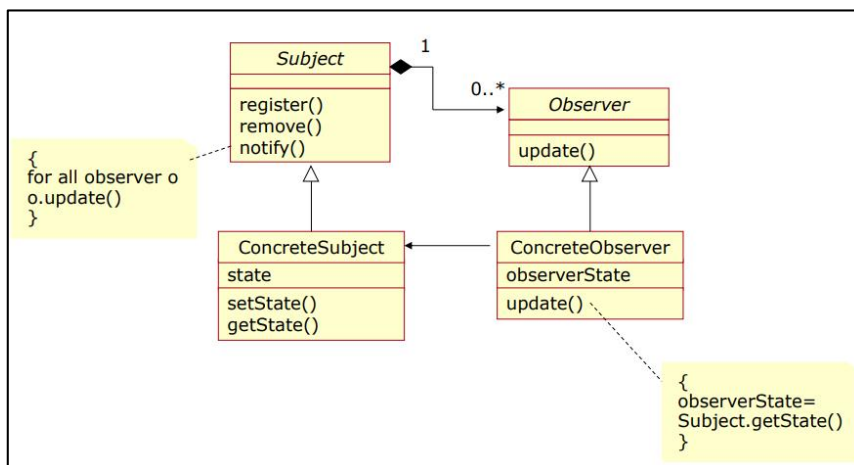
Conseguenze:

- Maggiore flessibilità rispetto all'approccio statico di subclassing.
- Evita la definizione di strutture gerarchiche complesse, che potrebbero diventare difficili da gestire.

Alcune note:

- Nella programmazione in Java, il pattern Decorator è ampiamente utilizzato nella definizione degli Stream di I/O, come `BufferedInputStream` o `DataInputStream`.
- Sebbene il pattern Decorator sia simile al pattern Composite, la finalità è diversa. Il Decorator serve ad aggiungere responsabilità in modo dinamico, mentre il Composite si concentra sulla composizione di oggetti per formare strutture più complesse.
- È anche simile al pattern Adapter, ma mentre l'Adapter si limita a un adattamento limitato di un'interfaccia, il Decorator estende le funzionalità dell'oggetto in modo più flessibile.

In breve, il pattern Decorator è utile quando si desidera estendere le funzionalità di un oggetto in modo dinamico e flessibile, senza dover ricorrere a una gerarchia di classi troppo complessa.



OBSERVER

Il pattern Observer è un pattern comportamentale che ha come scopo principale definire una dipendenza uno a molti tra oggetti, mantenendo basso il grado di accoppiamento. In altre parole, il pattern Observer consente a un oggetto, chiamato Subject, di notificare automaticamente gli oggetti interessati, chiamati Observer, quando il suo stato cambia, in modo che tutti gli oggetti dipendenti possano aggiornarsi automaticamente.

Motivazione:

Un tipico scenario in cui si applica il pattern Observer è nelle applicazioni con interfaccia grafica (GUI), realizzate secondo il paradigma Model-View-Control. Quando il Model cambia, gli oggetti che implementano la View devono aggiornarsi per riflettere correttamente lo stato attuale del Model.

Classificazione:

Il pattern Observer è un pattern comportamentale basato su oggetti, poiché coinvolge la definizione di come gli oggetti interagiscono tra loro durante il cambiamento di stato.

L'approccio corretto:

Il pattern Observer prevede che gli osservatori si registrino presso l'oggetto osservato. In questo modo, è l'oggetto osservato che notifica ogni cambiamento di stato agli osservatori. Quando un osservatore rileva la notifica, può interrogare l'oggetto osservato o svolgere altre operazioni indipendenti dal valore specifico dello stato.

Applicabilità:

- Si applica quando un'azione può essere scomposta in due ambiti, ciascuno dei quali è incapsulato in oggetti separati per mantenere basso il livello di accoppiamento.
- È utile per gestire le modifiche di oggetti conseguenti alla variazione dello stato di un oggetto.

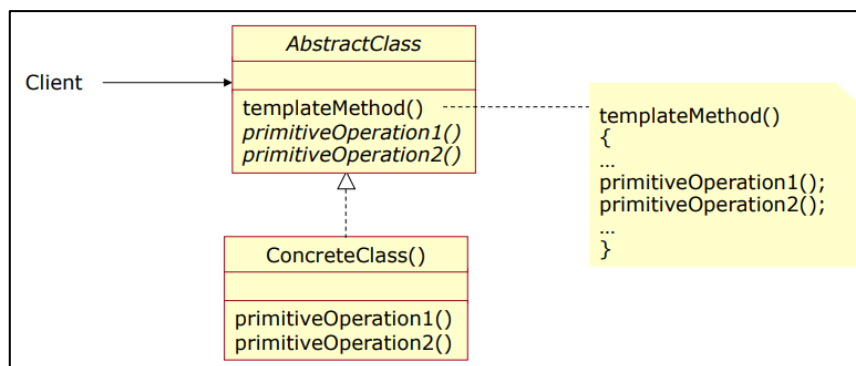
Partecipanti:

- **Subject e ConcreteSubject:**
 - `Subject` definisce l'interfaccia comune per l'oggetto osservato.
 - `ConcreteSubject` è l'oggetto osservato concreto che implementa l'interfaccia di `Subject`.
- **Observer e ConcreteObserver:**
 - `Observer` definisce l'interfaccia comune per gli osservatori.
 - `ConcreteObserver` è l'oggetto osservatore concreto che implementa l'interfaccia di `Observer`.

Conseguenze:

- L'accoppiamento tra `Subject` ed `Observer` è astratto; il `Subject` conosce solo la lista degli osservatori.
- La notifica è una comunicazione di tipo broadcast; il `Subject` non si occupa di quanti sono gli osservatori registrati.
- Attenzione perché una modifica al `Subject` scatena una serie di modifiche su tutti gli osservatori e su tutti gli oggetti da questi dipendenti.

In sintesi, il pattern Observer è utile quando si vuole stabilire una dipendenza uno-a-molti tra oggetti in modo che quando uno cambia, tutti gli altri interessati vengano notificati e aggiornati automaticamente. Questo riduce l'accoppiamento e aumenta la flessibilità nella gestione degli stati e delle dipendenze nel sistema.



TEMPLATE METHOD

Il pattern Template Method è un pattern comportamentale che ha come scopo principale definire la struttura di un algoritmo all'interno di un metodo di una classe base (AbstractClass), delegando alcuni passi specifici alle sottoclassi concrete (ConcreteClass). Questo pattern è particolarmente utile quando si desidera fornire un'implementazione generica di un algoritmo e permettere alle sottoclassi di personalizzare alcune parti dell'algoritmo senza modificarne la

struttura di base.

Motivazione:

Considera un framework per costruire applicazioni in grado di gestire documenti diversi. Il Template Method definisce un algoritmo in base ad operazioni astratte che saranno definite nelle sottoclassi specifiche. In questo modo, è possibile fornire una struttura di base per l'algoritmo e lasciare alle sottoclassi la definizione di passi variabili.

Classificazione:

Il Template Method è un pattern comportamentale basato su classi, in quanto coinvolge la definizione di comportamenti comuni nella classe astratta e la delega di comportamenti specifici alle sottoclassi concrete.

Applicabilità:

- È utilizzato per implementare la parte invariante di un algoritmo, lasciando alle sottoclassi la definizione degli step variabili.
- È utile quando ci sono comportamenti comuni che possono essere inseriti nel template.

Partecipanti:

- **AbstractClass e ConcreteClass:**
 - `AbstractClass` è la classe astratta che definisce il Template Method, che contiene la struttura generale dell'algoritmo.

- `ConcreteClass` è la classe concreta che implementa le operazioni astratte definite nella classe astratta.
- **Client:** Utilizza l'`AbstractClass` per eseguire l'algoritmo.

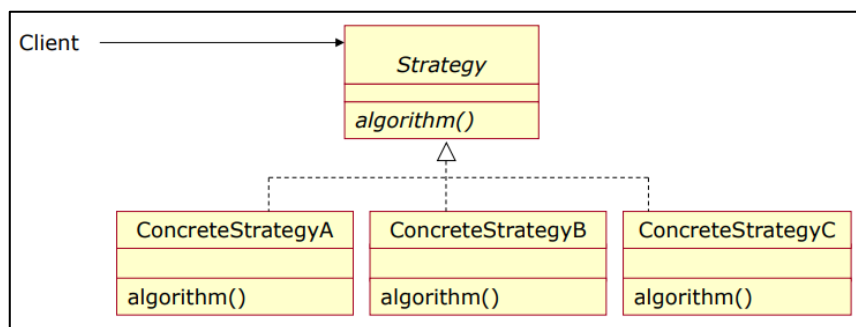
Conseguenze:

- I metodi template permettono il riuso del codice, poiché la struttura generale dell'algoritmo è definita nella classe astratta.
- Creano una struttura di controllo invertito, dove è la classe padre che chiama le operazioni ridefinite nelle sottoclassi e non viceversa.
- Per controllare l'estendibilità delle sottoclassi, i metodi richiamati dal template sono chiamati metodi gancio (hook).
- I metodi hook possono essere implementati, offrendo un comportamento standard che la sottoclasse può volendo ridefinire.

Alcune Note:

- Il Template Method è simile al Factory Method nel senso che entrambi coinvolgono l'invocazione di metodi astratti tramite un'interfaccia e la rimandata implementazione di metodi a classi concrete non note.
- Tuttavia, indirizzano problemi diversi: il Template Method generalizza un algoritmo, mentre il Factory Method crea e restituisce un'istanza di classe concreta per sganciare il cliente dalla scelta del tipo specifico.

In sintesi, il Template Method è utile quando si vuole fornire una struttura generica per un algoritmo e consentire alle sottoclassi di personalizzarne alcune parti senza modificare la struttura complessiva dell'algoritmo. Questo promuove il riuso del codice e la flessibilità nell'estensione del comportamento.



STRATEGY

Il pattern Strategy è un pattern comportamentale che ha come scopo principale definire ed incapsulare una famiglia di algoritmi in modo da renderli intercambiabili indipendentemente dal client che li usa. Questo pattern permette di definire un'interfaccia comune per un insieme di algoritmi, incapsulare ciascun algoritmo in una classe separata, e permettere ai client di selezionare

dinamicamente l'algoritmo da utilizzare.

Motivazione:

Considera una situazione in cui hai una famiglia di algoritmi, ad esempio algoritmi di ordinamento come QuickSort, BubbleSort e MergeSort. Invece di implementare tutti questi algoritmi all'interno di una singola classe, il pattern Strategy suggerisce di creare una classe separata per ciascun algoritmo, tutte implementando la stessa interfaccia (Strategy). In questo modo, puoi facilmente cambiare l'algoritmo utilizzato in un'applicazione senza dover modificare il client.

Classificazione:

Il pattern Strategy è un pattern comportamentale basato su oggetti, poiché coinvolge la definizione di comportamenti (algoritmi) all'interno di oggetti.

Applicabilità:

- Molte classi correlate differiscono solo per il comportamento. Il pattern fornisce un modo per avere un'interfaccia comune.
- Sono necessarie più varianti di uno stesso algoritmo, a seconda dei tipi di dato in ingresso o delle condizioni operative.

Partecipanti:

- **Strategy e ConcreteStrategy:**
 - `Strategy` è l'interfaccia comune per tutti gli algoritmi.
 - `ConcreteStrategy` sono le classi concrete che implementano le varianti specifiche degli algoritmi.
- **Client:** Utilizza un oggetto `Strategy` per invocare un algoritmo specifico.

Conseguenze:

- Il pattern separa l'implementazione degli algoritmi dal contesto dell'applicazione, consentendo una maggiore flessibilità.

- Le diverse strategie eliminano i blocchi condizionali che sarebbero necessari inserendo tutti i diversi comportamenti in una unica classe.
- Lo svantaggio principale è che i client devono conoscere le diverse strategie, ma questo è spesso accettabile per ottenere la flessibilità desiderata.

Note Aggiuntive:

- Il pattern Strategy elimina la necessità di utilizzare un subclassing e permette di comporre il comportamento degli oggetti.
- Se l'uso del subclassing della classe client per aggiungere un algoritmo non sarebbe una buona scelta, il pattern Strategy offre un'alternativa più flessibile.

In conclusione, il pattern Strategy è utile quando hai una famiglia di algoritmi e vuoi rendere il loro utilizzo intercambiabile senza influire sul client. Incapsulare gli algoritmi in classi separate permette una maggiore manutenibilità e facilità di estensione del sistema.

METRICHE DI STRUTTURA

MISURE DI PROGETTAZIONE PRELIMINARE

- **Misure Intermodulari:** Si prendono in considerazione le dipendenze tra i moduli secondo l'architettura di sistema sviluppata nella fase di progettazione. Queste misure considerano come i vari moduli interagiscono tra loro.
- **Misure Intramodulari:** Si riferiscono alle caratteristiche dei singoli moduli. Vengono utilizzate durante la progettazione dettagliata e l'implementazione di ciascun modulo.

Definizione di Modulo: Un modulo è una sequenza contigua di istruzioni del programma, delimitata da elementi di confine.

Impatto delle Decisioni di Architettura:

- Le decisioni prese durante la fase di progettazione preliminare influenzano molte qualità importanti del software risultante, come facilità di implementazione, affidabilità e manutenibilità.
- Le misure di progettazione preliminare possono fornire un feedback cruciale sulle caratteristiche del sistema in fase di sviluppo.

Relazioni tra Progettazione Preliminare e Codice:

Esiste una relazione uno a uno tra i moduli indicati nella progettazione e i moduli effettivamente presenti nel codice.

- **Corrispondenza Connessione Intermodulare:** Le connessioni tra moduli indicate nella progettazione devono corrispondere alle referenze intermodulari nel codice.
- **Corrispondenza Interfaccia Dati Intermodulare:** Le interfacce dati intermodulari definite nella progettazione devono riflettersi nei dati condivisi tra i moduli nel codice.

In breve, la progettazione preliminare svolge un ruolo cruciale nel determinare l'architettura del sistema e influenza le qualità chiave del software risultante. La correlazione tra progettazione preliminare e codice assicura che l'implementazione rispecchi fedelmente le decisioni prese durante la progettazione del sistema.

ARCHITETTURA DEI MODULI – STRUCTURE CHART

L'architettura dei moduli di un sistema software può essere rappresentata mediante un grafo, denotato da $S = \{N, R\}$.

- N rappresenta l'insieme dei nodi, dove ogni nodo n corrisponde a un modulo.
- R rappresenta l'insieme delle relazioni, dove ogni arco r indica una relazione tra due sottosistemi (ad esempio, chiamata di procedura, flusso di dati, ecc.).

Modularità:

La modularità è l'estensione in cui il software è composto da componenti discrete in modo che una modifica a una componente abbia un impatto minimo sulle altre componenti. Un'elevata modularità è desiderabile perché i programmi con una bassa modularità sono ritenuti più soggetti agli errori, meno manutenibili, meno riutilizzabili, ecc.

Attributi della Modularità:

- **Coesione:** L'estensione in cui un singolo modulo svolge un compito ben definito.
- **Accoppiamento:** Il grado di interdipendenza tra i moduli.
- **Morfologia:** La forma della struttura complessiva del sistema.
- **Flusso di Informazioni:** Le interconnessioni che un modulo ha con altri moduli nel sistema, ovvero il fan-in e il fan-out del modulo.

MORFOLOGIA

La morfologia si riferisce alla forma generale dell'architettura di un sistema software. In altre parole, è una caratteristica che descrive la struttura e la disposizione complessiva dei componenti del sistema.

- **Dimensione (Size):** Indica il numero totale di nodi e archi nel grafo che rappresenta l'architettura del sistema.
- **Profondità (Depth):** Rappresenta il percorso più lungo dal nodo radice a un nodo foglia. In altre parole, è la lunghezza massima di una catena di nodi dal vertice principale al punto più distante nel sistema.
- **Ampiezza (Width):** Indica il numero massimo di nodi in qualsiasi livello del grafo. Questo offre un'idea della larghezza massima del sistema a ogni livello di profondità.
- **Rapporto tra Archi e Nodi (Edge-to-Node Ratio):** Misura la densità di connettività del sistema. Indica quanti archi ci sono rispetto al numero totale di nodi. Un rapporto più alto potrebbe indicare una maggiore complessità delle connessioni nel sistema.

TREE IMPURITY

La "Tree Impurity" è una misura utilizzata per valutare quanto un grafo G sia diverso da un albero. In altre parole, si cerca di misurare quanto il grafo differisce da una struttura ad albero, che è spesso associata a una buona organizzazione e chiarezza nella progettazione. La Tree Impurity può essere definita come la differenza tra il numero massimo di archi possibili in un grafo e il numero effettivo di archi nel grafo, diviso per il numero massimo di archi possibili in un albero:

$$m(G) = \frac{\text{numero massimo di archi possibili in un grafo} - \text{numero di archi nel grafo}}{\text{numero massimo di archi possibili in un albero}}$$

- **$m(G) = 0$** indica che il grafo G è un albero, cioè ha la struttura più "albero-like" possibile.
- **$m(G) = 1$** indica che il grafo G è completamente diverso da un albero, ovvero ha il massimo numero possibile di differenze rispetto a una struttura ad albero.

In generale, un valore più piccolo di $m(G)$ è desiderabile, poiché indica una maggiore similarità del grafo a una struttura ad albero, che è spesso associata a progettazioni più chiare e organizzate.

INTERNAL REUSE

La "Internal Reuse" è una misura che indica l'estensione del riutilizzo dei moduli all'interno dello stesso prodotto software. È importante notare che questa misura si concentra sul riutilizzo interno al prodotto, in contrapposizione al "riutilizzo esterno" che potrebbe coinvolgere il riutilizzo di moduli tra prodotti diversi. La formula specifica proposta per la "Internal Reuse" è la seguente:

$$r(G) = e - n + 1$$

dove:

- **$r(G)$** rappresenta la misura di riutilizzo interno.
- **e** è il numero di archi nel grafo (rappresentante i moduli).
- **n** è il numero di nodi nel grafo (rappresentante i moduli).
- **+1** è un termine aggiunto per compensare alcune delle limitazioni della misura.

La misura $r(G)$ viene interpretata come quanto il sistema riutilizza i moduli internamente. Un valore più basso di $r(G)$ indica un maggiore riutilizzo interno.

- Se **$r(G) = 0$** , allora non c'è alcun riutilizzo interno, ogni modulo è utilizzato solo una volta.
- Se **$r(G) < 0$** , indica un'alta probabilità di moduli che vengono utilizzati più volte, implicando un riutilizzo significativo.

Critiche:

- **Mancanza di Considerazione per Chiamate Ripetute:** La misura non tiene conto delle chiamate ripetute ai moduli. Se un modulo è chiamato più volte, la misura potrebbe non riflettere accuratamente il grado di riutilizzo.
- **Non Considera la Dimensione dei Moduli Riutilizzati:** La misura non tiene conto della dimensione o della complessità dei moduli riutilizzati. Un modulo piccolo riutilizzato molte volte potrebbe avere lo stesso impatto di un modulo grande riutilizzato poche volte.

In sintesi, la "Internal Reuse" è una misura che cerca di quantificare il grado di riutilizzo dei moduli all'interno di uno stesso prodotto. Tuttavia, ha alcune limitazioni e critiche, come la mancanza di considerazione per le chiamate ripetute e la mancanza di considerazione per la dimensione dei moduli riutilizzati.

INFORMATION FLOW – FLUSSO DI INFORMAZIONI

Il concetto di flusso di informazioni in ingegneria del software si basa sull'idea che la complessità di un modulo dipenda da due fattori principali: la complessità del codice del modulo e la complessità delle interfacce del modulo, cioè le sue connessioni con l'ambiente circostante.

- **Inter-modular Attribute (Attributo Inter-modulare):** Si riferisce al livello totale di flusso di informazioni attraverso un sistema, dove i moduli sono considerati come componenti atomiche.
- **Intra-modular Attribute (Attributo Intra-modulare):** Si riferisce al livello totale di flusso di informazioni tra un modulo individuale e il resto del sistema.

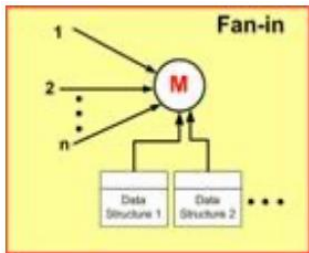
Misure del Flusso di Informazioni:

- **Local Flow of Information (Flusso Locale di Informazioni):**
 - **Diretto:** Avviene quando un modulo chiama un altro modulo.
 - **Indiretto:** Si verifica attraverso i valori restituiti da un modulo all'altro.
- **Global Flow of Information (Flusso Globale di Informazioni):** Coinvolge l'informazione passata tra i moduli tramite una struttura dati globale condivisa.

Utilità delle Misure del Flusso di Informazioni:

- **Identificazione delle Parti Critiche del Sistema:** Le misure del flusso di informazioni possono essere utilizzate per individuare le parti critiche di un sistema software, cioè le parti che sono fortemente coinvolte nella trasmissione di informazioni.
- **Individuazione dei Punti di Stress nel Sistema:** Le misure del flusso di informazioni possono evidenziare i punti di stress nel sistema, dove la quantità di informazioni che attraversa determinati moduli è significativamente elevata.
- **Comprensione dei Problemi di Progettazione Potenziali:** Forniscono una visione sui possibili problemi di progettazione, poiché la complessità derivante dai flussi di informazioni può indicare aree in cui la struttura del sistema potrebbe essere migliorata.

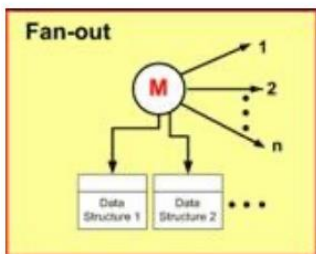
In sintesi, le misure del flusso di informazioni forniscono un modo per valutare quanto le informazioni si muovono attraverso il sistema e possono essere utilizzate per identificare aree critiche, punti di stress e potenziali problemi di progettazione all'interno di un sistema software.



Fan-in e Fan-out:

I concetti di "Fan-in" e "Fan-out" sono misure utilizzate per quantificare i flussi di informazioni in un sistema software, concentrandosi su un modulo specifico.

- Il **"Fan-in"** di un modulo M è il numero totale di flussi locali (diretti e indiretti) che terminano in M, più il numero di flussi globali (strutture dati) da cui M recupera informazioni.
- Il **"Fan-out"** di un modulo M è il numero totale di flussi locali (diretti e indiretti) che iniziano in M, più il numero di flussi globali (strutture dati) che vengono aggiornati da M.



Interpretazione delle Misure Fan-in e Fan-out:

- **Alto Fan-out:** Indica che il modulo influenza o controlla molti altri moduli.
- **Alto Fan-in:** Indica che il modulo è influenzato o controllato da molti altri moduli.
- **Modulo Centrale:** Un modulo con alto fan-in e fan-out è spesso al centro del sistema, influenzando molti moduli e venendo influenzato da molti altri.
- **Modulo Periferico:** Un modulo con basso fan-in e fan-out è generalmente ai margini del sistema, con meno connessioni dirette con altri moduli.
- **Complessità e Potenziali Problemi di Progettazione:** Moduli con elevati valori di fan-in e fan-out possono essere complessi e più inclini a errori di progettazione. Questo perché possono svolgere più di una funzione, e se la struttura di progettazione deve essere modificata, gli aspetti da cambiare non sono concentrati in un unico punto, ma sono distribuiti tra vari moduli interconnessi.

out possono essere complessi e più inclini a errori di progettazione. Questo perché possono svolgere più di una funzione, e se la struttura di progettazione deve essere modificata, gli aspetti da cambiare non sono concentrati in un unico punto, ma sono distribuiti tra vari moduli interconnessi.

Misura del Flusso di Informazioni (Henry & Kafura):

L'Information Flow (IF) per un modulo M_i secondo la proposta di Henry-Kafura è definito come:

$$IF(M_i) = [fan - in(M_i) * fan - out(M_i)]^2$$

L'IF per un sistema con n moduli è la somma dell'IF per ciascun modulo nel sistema.

La misura originale di Henry-Kafura include sia i flussi di controllo che i flussi di informazioni nel conteggio di fan-in e fan-out.

Una variante proposta da Shepperd include solo i flussi di informazioni.

In sintesi, fan-in e fan-out sono misure che riflettono quanto un modulo è coinvolto nelle informazioni che fluiscono attraverso il sistema. Valori elevati possono indicare complessità e potenziali problemi di progettazione, mentre la misura dell'Information Flow cerca di quantificare questi flussi in modo più consolidato.

STRUTTURA DI MISURAZIONE

La "Struttura di Misurazione" in ambito software si riferisce al modo in cui viene organizzato e gestito il flusso di controllo, il flusso di dati e la struttura dei dati all'interno di un programma durante la progettazione dettagliata e l'implementazione. Si suddivide in tre componenti principali:

- **Struttura del Flusso di Controllo:** Rappresenta la sequenza di esecuzione delle istruzioni del programma. In altre parole, indica l'ordine in cui vengono eseguite le diverse parti del codice.
- **Flusso di Dati:** Traccia dei dati mentre vengono creati o gestiti dal programma. Aiuta a comprendere come i dati si muovono attraverso il programma e come vengono manipolati.
- **Struttura Dati:** Rappresenta l'organizzazione dei dati indipendentemente dal programma. Si concentra sulla disposizione e la gestione dei dati nel contesto complessivo.

Rappresentazione della Struttura di un Programma: Viene rappresentata utilizzando un "grafo di flusso di controllo" o semplicemente "grafo di flusso". Questo grafo mostra le relazioni e il flusso tra diverse istruzioni del programma.

Definizione della Complessità Strutturale: La "complessità" in termini di struttura viene definita utilizzando il concetto di "complessità ciclomatica". Questa misura offre un'indicazione della complessità del codice sorgente misurando il numero di cammini linearmente indipendenti attraverso il grafo di controllo.

Struttura di Controllo di Base (BCS): Le BCS sono meccanismi essenziali di flusso di controllo utilizzati per costruire la struttura logica del programma. I tipi di BCS includono: **Sequenza, Selezione, Iterazione.**

Strutture di Controllo Avanzate (ACS): Oltre alle BCS, ci sono anche le ACS, che includono **Chiamata di Procedura/Funzione/Agente, Ricorsione (auto-chiamata), Interruzione, Concorrenza.** Queste ACS aggiungono complessità alla struttura del programma e possono influenzare la sua esecuzione e il flusso di controllo.

FLOWGRAPH – GRAFO DI FLUSSO

Il grafo di flusso (Flowgraph) è un modo di modellare la struttura del flusso di controllo di un programma attraverso un grafo diretto (di-graph). Questo grafo è rappresentato come: $FG = \{N, E\}$ dove N è l'insieme di nodi, ognuno dei quali corrisponde a un'istruzione del programma ed E è l'insieme di archi, che rappresentano il flusso di controllo da un'istruzione del programma a un'altra.

Nodi Speciali nel Flowgraph:

- **Nodi di Procedura:** Nodi con un grado uscente di 1. Rappresentano il punto in cui il controllo passa a una procedura, funzione o sotto-programma.
- **Nodi Predicato:** Nodi con un grado uscente diverso da 1. Indicano la presenza di un'istruzione di selezione (if) e possono condurre a percorsi diversi a seconda delle condizioni.
- **Nodo di Inizio:** Nodi con un grado entrante di 0. Rappresentano il punto di inizio del programma o di una porzione del programma.
- **Nodi Terminali (End):** Nodi con un grado uscente di 0. Rappresentano il punto di fine del programma o di una porzione del programma.

Grado Entrante e Grado Uscente:

- **Grado Uscente:** Indica il numero di archi che escono da un nodo. I nodi di procedura avranno un grado uscente di 1, mentre i nodi terminali avranno un grado uscente di 0.
- **Grado Entrante:** Indica il numero di archi che entrano in un nodo. I nodi di inizio avranno un grado entrante di 0.

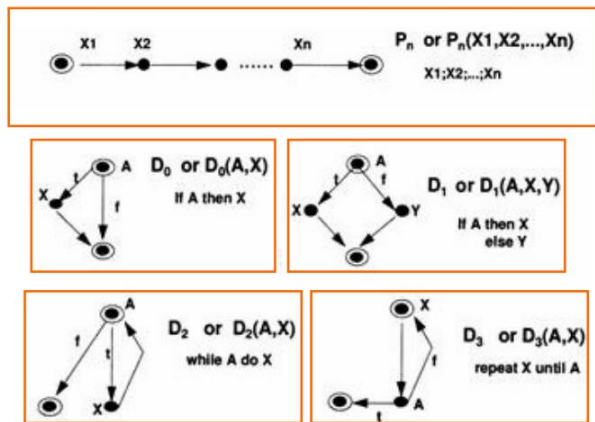
Basic CS	Sequence	
	Selection	
	Iteration	
Advanced CS	Procedure/function call	
	Recursion	
	Interrupt	
	Concurrency	

Costrutti del Flowgraph:

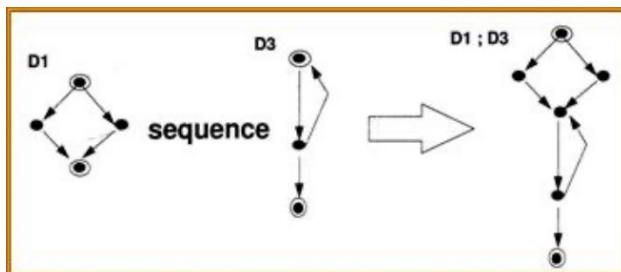
- **Sequence (Sequenza):** Rappresenta un elenco sequenziale di istruzioni senza strutture di controllo aggiuntive.
- **Selection (Selezione):** Indicata dai nodi predicato, rappresenta l'istruzione di selezione (if ... then ... else) e il flusso di controllo diverge a seconda delle condizioni.
- **Iteration (Iterazione):** Rappresenta le istruzioni di iterazione come do ... while o ripeti ... fino a quando.
- **Procedure/Function Call (Chiamata di Procedura/Funzione):** Rappresenta il passaggio del controllo a una procedura, funzione o sotto-programma.
- **Recursion (Ricorsione):** Rappresenta l'auto-chiamata di una funzione all'interno di se stessa.

- **Interrupt (Interruzione):** Rappresenta l'interruzione del flusso di controllo causato da eventi esterni o interrupt.
- **Concurrency (Concorrenza):** Rappresenta l'esecuzione simultanea di più istruzioni o porzioni di codice.

Common Flowgraph Program Models



In sintesi, il Flowgraph è uno strumento visuale che aiuta a rappresentare la struttura del flusso di controllo di un programma attraverso nodi e archi, consentendo una comprensione più chiara della logica e della sequenza delle istruzioni nel codice.



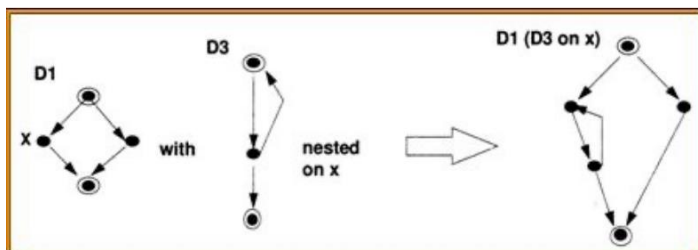
Sequencing (Sequenziazione):

Il concetto di sequenziazione è indicato dalla notazione $(F1; F2)$, che rappresenta la sequenza di due flowgraph $F1$ e $F2$. La sequenziazione è un modo di combinare due flowgraph in modo che il risultato sia un nuovo flowgraph che rappresenta l'esecuzione sequenziale di $F1$ seguito da $F2$.

- Supponiamo di avere due flowgraph, $F1$ e $F2$.
- La sequenza $(F1; F2)$ viene formata unendo il nodo terminale di $F1$ con il nodo iniziale di $F2$.

- In questo modo, il flusso di controllo passa da $F1$ a $F2$ senza alcuna decisione o iterazione tra i due.

In termini pratici, questo rappresenta l'esecuzione di una sequenza di istruzioni nel programma, dove le istruzioni di $F1$ vengono eseguite prima di passare a $F2$.



Nesting (Annidamento):

Il concetto di annidamento è indicato dalla notazione $F1(F2)$, che rappresenta l'annidamento di $F2$ su $F1$ in un punto specifico x . L'annidamento crea un nuovo flowgraph formando $F1$ e sostituendo l'arco uscente da x con l'intero flowgraph $F2$.

- Supponiamo di avere due flowgraph, $F1$ e $F2$.
- L'annidamento $F1(F2)$ avviene sostituendo l'arco uscente da un nodo specifico x in $F1$ con l'intero flowgraph $F2$.

- In questo modo, il flusso di controllo passa attraverso $F1$, raggiunge il punto di annidamento x , esegue $F2$ e quindi prosegue.

L'annidamento è utilizzato per incorporare un flowgraph all'interno di un altro, consentendo una maggiore complessità e struttura nei programmi. Può essere utilizzato per rappresentare sottoprogrammi, funzioni o procedure che vengono chiamati da un punto specifico nel programma principale.

Prime Flowgraphs:

I "Prime Flowgraphs" sono flowgraph che non possono essere decomposti in modo non banale attraverso sequenziazione e annidamento. In altre parole, sono flowgraph elementari che rappresentano unità atomiche e non possono essere ulteriormente suddivisi in moduli più piccoli senza perdere la loro struttura e funzionalità. Ad esempio:

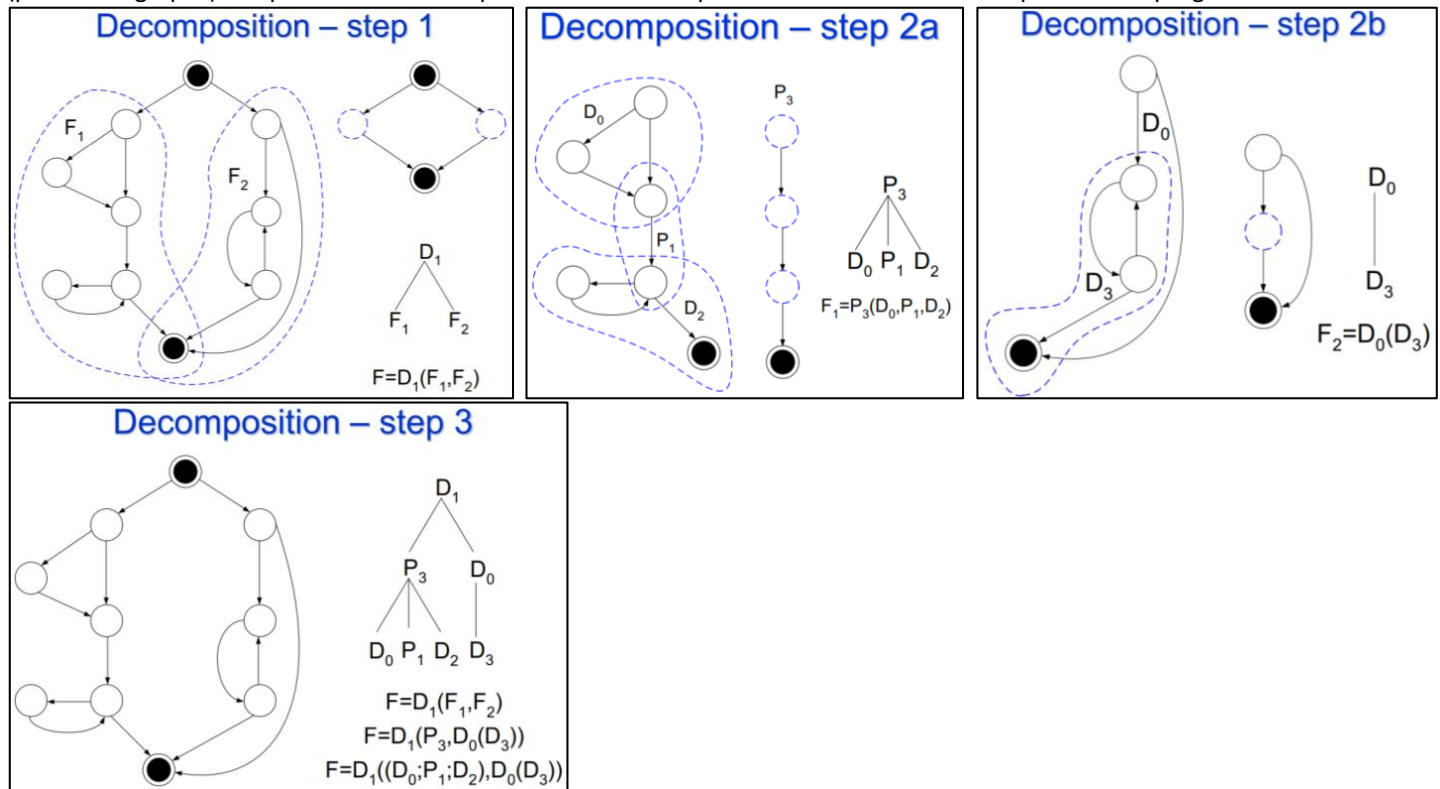
- **P1:** Un singolo nodo o un'istruzione senza alcuna struttura di controllo.
- **D0:** Un flowgraph che rappresenta un'istruzione di selezione condizionale (if) senza alcuna alternativa (else).
- **D1:** Un flowgraph che rappresenta un'istruzione di selezione condizionale con un'alternativa (else).
- **D2:** Un flowgraph che rappresenta un'istruzione di selezione condizionale con due alternative.
- **D3:** Un flowgraph che rappresenta un'istruzione di selezione condizionale con tre alternative.

Le "D-Structures" sono tipiche della programmazione strutturata, e il "D" proviene da Edsger Dijkstra, che è stato un pioniere nella programmazione strutturata. Queste strutture sono fondamentali per mantenere la chiarezza e la comprensibilità del codice.

Prime Decomposition:

Il "Prime Decomposition Theorem" (Teorema di Decomposizione Prime) è formulato da Fenton e Whitty. Esso afferma che ogni flowgraph può essere univocamente decomposto in una gerarchia di prime. Questa gerarchia è nota come "decomposition tree" (albero di decomposizione). L'albero di decomposizione mostra come il flowgraph complessivo è formato dalla combinazione di diverse prime flowgraphs attraverso sequenziazione e annidamento.

In altre parole, il teorema afferma che è possibile scomporre ogni flowgraph complesso in un insieme di flowgraph più semplici (prime flowgraphs) che possono essere sequenziati o annidati per ottenere la struttura complessiva del programma.



MISURAZIONE GERARCHICA

La "Hierarchical Measurement" è un modo di definire misure per i flowgraph utilizzando l'albero di decomposizione. Questo approccio si basa sull'idea che possiamo misurare un attributo del flowgraph in modo gerarchico seguendo questi passaggi:

- Definizione della misura per i prime flowgraphs.
- Descrizione di come l'operazione di sequenziazione influisce sull'attributo.
- Descrizione di come l'operazione di annidamento influisce sull'attributo.

Depth of Nesting (Profondità di Annidamento):

Questa misura indica la profondità degli annidamenti nel codice. Maggiore è la profondità di annidamento, maggiore è la complessità. La profondità di annidamento è calcolata gerarchicamente attraverso l'albero di decomposizione, considerando la profondità dei primes, la sequenziazione e l'annidamento.

La profondità di annidamento $n(F)$ per un flowgraph F può essere misurata in termini di:

- Per i **primes**: $n(P_1) = 0$, $n(P_2) = n(P_3) = \dots = n(P_k) = 1$, $n(D_0) = n(D_1) = n(D_2) = n(D_3) = 1$.
- Per la **sequenziazione**: $n(F_1; F_2; \dots; F_k) = \max\{n(F_1), n(F_2), \dots, n(F_k)\}$.
- Per l'**annidamento**: $n(F(F_1, F_2, \dots, F_k)) = 1 + \max\{n(F_1), n(F_2), \dots, n(F_k)\}$.

D-Structuredness (Strutturazione D):

Questa misura valuta se un programma segue i principi della programmazione strutturata. Un programma è considerato strutturato se può essere composto utilizzando un numero limitato di costrutti ammissibili, come sequenza, selezione e iterazione. La misura di strutturazione D è calcolata in base ai principi della programmazione strutturata.

La strutturazione D è una misura che verifica se un programma è strutturato secondo le definizioni di programmazione strutturata. La strutturazione D $d(F)$ per un flowgraph F può essere misurata in termini di:

- Per i **primes**: $d(P_1) = 1$, $d(D_0) = d(D_1) = d(D_2) = d(D_3) = 1$, 0 altrimenti.
- Per la **sequenziazione**: $d(F_1; F_2; \dots; F_k) = \min\{d(F_1), d(F_2), \dots, d(F_k)\}$.
- Per l'**annidamento**: $d(F(F_1, F_2, \dots, F_k)) = \min\{d(F), d(F_1), d(F_2), \dots, d(F_k)\}$.

COMPLESSITA' CICLOMATICA

La "Cyclomatic Complexity" è una misura della complessità di un programma basata sul grafo di flusso del programma. Può essere calcolata in due modi: basato sul grafo di flusso e basato sul codice. La formula generale per calcolare la complessità ciclomatica $v(F)$ di un programma con il flowgraph F è $v(F) = e - n + 2$ dove e è il numero di archi (rappresentanti rami e cicli) nel grafo di flusso ed n è il numero di nodi (rappresentanti blocchi di codice sequenziale) nel grafo di flusso.

La complessità ciclomatica misura il numero di percorsi linearmente indipendenti attraverso il flowgraph. I percorsi sono considerati linearmente indipendenti se nessun percorso nel set è una combinazione lineare di un altro percorso.

Alternativamente, la complessità ciclomatica $v(F)$ può essere misurata come $v(F) = 1 + d$ dove d è il numero di nodi predicato (cioè, nodi con un grado uscente maggiore di 1) nel flowgraph. Questi nodi rappresentano i punti decisionali nel programma.

Misura basata sul Grafo di Flusso: Questo approccio calcola la complessità ciclomatica contando il numero di rami e cicli nel grafo di flusso. L'idea è che ogni branch o ciclo aggiunge complessità al programma, e la formula $e - n + 2$ tiene conto di questi elementi.

Misura basata sul Codice: Questo approccio conta direttamente il numero di nodi predicato nel grafo di flusso. I nodi predicato rappresentano decisioni nel codice, e la formula $1 + d$ riflette il fatto che la complessità è data dalla somma di un termine costante e il numero di decisioni.

Interpretazione della Complessità Ciclomatica: Maggiore è la complessità ciclomatica, maggiore è il numero di percorsi attraverso il programma, e quindi, maggiore è la complessità del codice. Un valore elevato potrebbe indicare una maggiore difficoltà nella comprensione e nella manutenzione del codice.

La complessità ciclomatica può anche essere vista come una misura gerarchica, poiché può essere calcolata in modo aggregato considerando la complessità dei singoli blocchi (primes) e combinando tali misure attraverso sequenziazione e annidamento.

Complessità dei Primes: La complessità di un prime è determinata dal numero di decisioni nel blocco di codice rappresentato da quel prime. Aggiungendo uno, si tiene conto dell'effetto complessivo delle decisioni su quel blocco. La formula è $v(F) = 1 + d$

Complessità della Sequenziazione: La complessità di una sequenza, indicata come $v(F_1; F_2; \dots; F_k)$, è pari alla somma delle complessità dei singoli componenti $v(F_i)$ meno il numero totale di componenti (k) più uno.

La formula è $v(F_1; F_2; \dots; F_k) = \sum_{i=1}^k v(F_i) - k + 1$. Questo significa che la complessità di una sequenza è influenzata dalla complessità di ciascun componente e dal numero totale di componenti nella sequenza.

Complessità dell'Annidamento su un Prime dato: La complessità dell'annidamento è influenzata dalla complessità del prime F e dalla complessità dei singoli componenti nell'annidamento. La complessità dell'annidamento di componenti su un prime F è pari alla complessità di F più la somma delle complessità dei singoli componenti meno il numero totale di componenti.

La formula è $v(F(F_1, F_2, \dots, F_k)) = v(F) + \sum_{i=1}^k v(F_i) - k$. Questo significa che la complessità dell'annidamento è influenzata dalla complessità del prime $v(F)$ e dalla complessità dei singoli componenti nell'annidamento.

Essential Complexity di McCabe:

L'Essential Complexity di McCabe, indicata come $ev(F)$, è una misura che tiene conto della complessità essenziale di un programma, escludendo alcune strutture di flusso che potrebbero non contribuire significativamente alla complessità del codice. La formula per calcolare l'Essential Complexity è data da $ev(F) = v(F) - m$ dove $v(F)$ è la complessità ciclomatica totale del flowgraph F e m è il numero di sottografi D_0 , D_1 , D_2 e D_3 presenti in F .

In altre parole, l'Essential Complexity è la complessità ciclomatica meno il numero di sottografi specifici (D_0 , D_1 , D_2 , D_3) che vengono considerati meno rilevanti o "essenziali" nella valutazione della complessità.

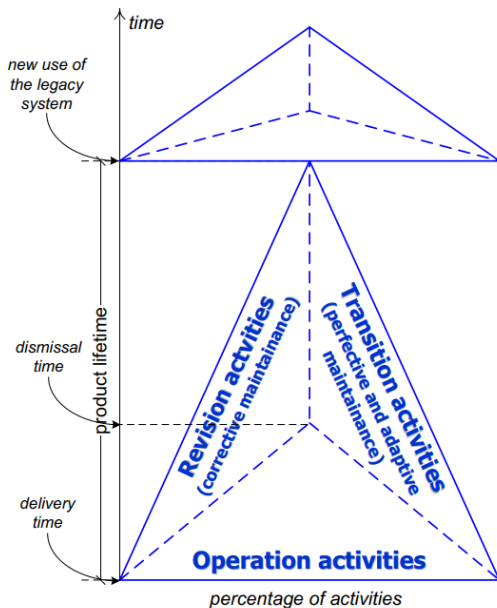
Critiche alla Complessità Ciclomatica:

- **Vantaggi:**
 - **Misurazione Obiettiva della Complessità:** La complessità ciclomatica offre una misura oggettiva della complessità del programma.
- **Svantaggi:**
 - **Utilizzabile solo a livello di componente:** La complessità ciclomatica è più adatta per valutare la complessità a livello di componenti e non a livello di sistema completo.
 - **Differenze nell'Impegno di Programmazione:** Due programmi con lo stesso valore di complessità ciclomatica possono richiedere sforzi di programmazione differenti.
 - **Richiede Visibilità Completa del Design o del Codice:** Per calcolare la complessità ciclomatica, è necessaria una visibilità completa del design o del codice sorgente.

QUALITA' DEL SOFTWARE

La qualità del software è un concetto complesso che riflette il grado in cui il software possiede una combinazione desiderata di attributi. Questa definizione è fornita dallo standard IEEE 1061-1998, che stabilisce una metodologia per le metriche di qualità del software. La qualità del software può essere considerata da diversi punti di vista:

- **Trascendente:** Questa prospettiva vede la qualità del software come un'eccellenza intrinseca, ovvero la qualità è una caratteristica intrinseca al software stesso.
- **Utente:** Dal punto di vista dell'utente, la qualità del software è definita dalla sua adattabilità all'uso, dalla capacità di soddisfare le esigenze dell'utente, dalla piacevolezza nell'uso e dalla soddisfazione dell'utente.
- **Prodotto:** Questa prospettiva considera la qualità del software in termini di attributi desiderati, come affidabilità, correttezza, aderenza ai requisiti e conformità agli standard.
- **Organizzazione:** Dal punto di vista dell'organizzazione, la qualità del software può essere misurata in termini di costi, profitti, efficienza, efficacia e valore aggiunto all'organizzazione, inclusa la capacità di produrre un prodotto commercializzabile.



Il Triangolo della Qualità (Modello di Qualità McCall):

Il Triangolo della Qualità, basato sul modello di qualità di McCall, rappresenta un approccio per valutare la qualità del software. Questo modello include tre lati del triangolo:

- **Tempo di Vita del Prodotto:** Indica la durata del prodotto software nel tempo, compreso il periodo di sviluppo, manutenzione e eventualmente il suo successivo rimpiazzo o dismissione.
- **Percentuale delle Attività:** Rappresenta la distribuzione percentuale delle attività durante il ciclo di vita del software. Queste attività includono la consegna del software, le attività di transizione (ad esempio, formazione degli utenti), e le attività di revisione (ad esempio, manutenzione correttiva).
- **Nuovo Uso del Sistema Legacy:** Indica l'uso continuato o l'integrazione del sistema legacy con nuove attività o nuove funzionalità.

Questo modello riflette l'idea che la qualità del software è un concetto complesso che va oltre la fase di sviluppo iniziale e considera il ciclo di vita completo del software, inclusa la manutenzione e l'evoluzione nel tempo.

INDICI E ATTRIBUTI DI QUALITA' DEL SOFTWARE

Gli indici di qualità del software sono misure utilizzate per valutare diversi aspetti della qualità di un prodotto software. Questi indici possono essere suddivisi in diverse categorie in base alle attività coinvolte. Le principali categorie sono le attività operative, di revisione e di transizione.

Attività Operative: Questi indici misurano la qualità del prodotto in termini di esecuzione effettiva delle sue funzioni e della sua aderenza alle specifiche.

- **i1 Correttezza:** Indica fino a che punto un prodotto soddisfa le specifiche e gli obiettivi degli utenti. Risponde alla domanda: "Fa ciò che voglio?".
- **i2 Affidabilità:** Misura fino a che punto ci si può aspettare che un prodotto esegua la sua funzione prevista con la precisione richiesta. Risponde alla domanda: "Lo fa in modo accurato tutto il tempo?".
- **i3 Efficienza:** Indica la quantità di risorse di calcolo e di codice richiesta da un prodotto per eseguire una funzione. Risponde alla domanda: "Si eseguirà sul mio hardware nel modo più efficiente possibile?".
- **i4 Integrità:** Valuta fino a che punto l'accesso al software o ai dati da parte di persone non autorizzate può essere controllato. Risponde alla domanda: "È sicuro?".
- **i5 Usabilità:** Misura lo sforzo richiesto per imparare, utilizzare, preparare input e interpretare l'output di un prodotto. Risponde alla domanda: "Posso usarlo?".

Attività di Revisione: Questi indici riflettono la facilità con cui il prodotto può essere corretto, testato e modificato dopo essere diventato operativo.

- **i6 Manutenibilità:** Misura lo sforzo richiesto per individuare e correggere un errore in un programma operativo. Risponde alla domanda: "Posso correggerlo?".
- **i7 Testabilità:** Indica lo sforzo richiesto per testare un prodotto per garantire che svolga la sua funzione prevista. Risponde alla domanda: "Posso testarlo?".

- **i8 Flessibilità:** Misura lo sforzo richiesto per modificare un prodotto operativo. Risponde alla domanda: "Posso cambiarlo?".

Attività di Transizione: Questi indici considerano la facilità con cui il prodotto può essere trasferito, riutilizzato in altri contesti, interfacciato con altri sistemi e adattato a nuovi requisiti.

- **i9 Portabilità:** Indica lo sforzo richiesto per trasferire un prodotto da un ambiente hardware e/o software a un altro. Risponde alla domanda: "Sarà possibile utilizzarlo su un'altra macchina?".
- **i10 Riutilizzabilità:** Valuta fino a che punto un prodotto (o parti di esso) può essere riutilizzato in altre applicazioni. Risponde alla domanda: "Potrò riutilizzare parte del software?".
- **i11 Interoperabilità:** Misura lo sforzo richiesto per collegare un prodotto con un altro. Risponde alla domanda: "Potrò interfacciarlo con un altro sistema?".
- **i12 Evolvibilità:** Indica lo sforzo richiesto per aggiornare il prodotto per soddisfare nuovi requisiti. Risponde alla domanda: "È facile aggiornarlo quando cambiano i requisiti?".

Gli Attributi di Qualità:

- **a1 Complessità:** Misura il livello di comprensibilità e verificabilità degli elementi del software e delle loro interazioni.
- **a2 Precisione:** Indica la precisione delle computazioni e degli output del software.
- **a3 Completezza:** Valuta la piena implementazione delle funzionalità richieste dal software.
- **a4 Coerenza:** Misura l'uso di tecniche di progettazione uniformi e notazioni coerenti nell'implementazione.
- **a5 Tolleranza agli Errori:** Rappresenta la continuità di operazione garantita anche in condizioni avverse.
- **a6 Tracciabilità:** Valuta il grado in cui può essere stabilita una relazione tra due o più prodotti del processo di sviluppo.
- **a7 Espandibilità:** Indica se lo spazio di archiviazione o le funzioni possono essere espansi.
- **a8 Generalità:** Valuta l'ampiezza delle possibili applicazioni del software.
- **a9 Modularità:** Misura la fornitura di moduli altamente indipendenti all'interno del software.
- **a10 Auto-documentazione:** Indica la presenza di documentazione incorporata nel codice stesso.

Relazioni tra Indici di Qualità e Attributi:

Ogni indice di qualità i_k è una combinazione di attributi di qualità a_j , contribuendo così a definire la qualità complessiva del software.

(+/- denotes positive/negative impact)

i_1 Correctness	i_2 Reliability
+ Completeness	+ Error Tolerance
+ Consistency	+ Consistency
+ Traceability	+ Accuracy
	- Complexity
i_7 Flexibility	i_{12} Evolvability
+ Traceability	+ Consistency
+ Consistency	- Complexity
- Complexity	+ Modularity
+ Modularity	+ Expandability
+ Generality	+ Generality
+ Auto-documentation	+ Auto-documentation

METODO DELLA CHECKLIST

Il Metodo della Checklist è un approccio utilizzato per valutare il livello di qualità di ciascun attributo in un software. Questo metodo coinvolge la compilazione di una checklist, che consiste in una serie di domande a cui si attribuiscono valutazioni. Il risultato della checklist viene quindi elaborato mediante algoritmi appropriati per ottenere un punteggio normalizzato, indicando così il livello di qualità di ciascun attributo.

Fasi Principali:

- **Valutazione degli Attributi con Checklist:** La qualità di ciascun attributo è valutata attraverso l'uso di tecniche basate su checklist. Ogni checklist è composta da domande, e per ciascuna domanda sono previste quattro risposte, ciascuna con un valore associato.
- **Elaborazione del Risultato della Checklist:** Il risultato della checklist viene elaborato utilizzando algoritmi appropriati. Questo processo di elaborazione fornisce un punteggio normalizzato per ogni attributo, riflettendo il livello di qualità.
- **Agruppare i Punteggi degli Attributi:** Una volta valutati i punteggi di ciascun attributo, i valori rilevanti vengono raggruppati per ciascun attributo. Questi valori vengono quindi sommati o sintetizzati in modo appropriato.
- **Normalizzazione dei Punteggi:** I punteggi normalizzati vengono ottenuti per riflettere il livello di qualità per ciascun indice. La normalizzazione può coinvolgere la trasformazione dei punteggi in una scala specifica o l'applicazione di ponderazioni per attribuire maggiore importanza a determinati attributi.

- **Considerare la Polarità degli Attributi:** Gli attributi devono essere considerati con la loro polarità, ovvero se hanno un impatto positivo o negativo sulla qualità complessiva. Ad esempio, un attributo come la complessità potrebbe avere un impatto negativo, mentre la completezza potrebbe avere un impatto positivo.

Valutazione della Checklist:

- **Composizione della Checklist:** Una checklist è composta da diverse domande, ciascuna con quattro possibili risposte valutate con un punteggio. Le domande non considerate applicabili o che non possono essere valutate sono escluse dal calcolo.
- **Team di Valutazione della Checklist:** La valutazione della checklist coinvolge un team composto da almeno quattro persone, ognuna con competenze diverse. Questa diversità di punti di vista aiuta a garantire una valutazione più precisa. La composizione consigliata del team comprende uno specialista di assicurazione della qualità, un leader di progetto, un ingegnere di sistema e un analista del software.

Calcolo del Punteggio degli Attributi: Per ogni attributo, eccetto "Modularità", il punteggio viene calcolato prendendo in considerazione le risposte alle domande associate all'attributo. Questo punteggio normalizzato riflette la valutazione dell'attributo basata sulle risposte fornite nel questionario.

Valutazione del Livello di Qualità dell'Indice: Il livello di qualità dell'indice è calcolato sommando i punteggi normalizzati degli attributi che influenzano positivamente e negativamente l'indice. Il risultato è quindi diviso per il numero totale di attributi associati all'indice. Questo fornisce un indicatore normalizzato del livello di qualità dell'indice.

SOFTWARE QUALITY ASSURANCE – SQA

Il Software Quality Assurance è un approccio pianificato e sistematico per garantire che sia il processo software che il prodotto software siano conformi agli standard, processi e procedure stabiliti. L'obiettivo principale di SQA è migliorare la qualità del software monitorando sia il software stesso che il processo di sviluppo per assicurare la piena conformità agli standard e alle procedure prestabilite.

Ruolo del SQA:

- **Fornire l'assicurazione alla gestione:** Il SQA ha il compito di garantire alla direzione che il processo software ufficialmente stabilito viene effettivamente implementato. Questa assicurazione è fondamentale per garantire la qualità del software e la conformità agli standard e alle procedure definite.
- **Garantire l'adozione di una metodologia di sviluppo appropriata:** Il SQA verifica che sia in atto una metodologia di sviluppo adeguata. Ciò significa che il processo di sviluppo segue una struttura organizzata e controllata che è in linea con gli standard e le best practice dell'industria.
- **Utilizzare standard e procedure nei progetti:** Il SQA si assicura che i progetti seguano gli standard e le procedure definiti. Questo contribuisce a garantire coerenza e coesione nel lavoro svolto, riducendo la possibilità di errori e garantendo una maggiore prevedibilità nei risultati.
- **Condurre revisioni e audit:** Il SQA è responsabile di organizzare e condurre revisioni e audit. Questi possono includere revisioni di codice, revisioni di documentazione e audit di processo. L'obiettivo è identificare eventuali problemi o non conformità e correggerli tempestivamente.
- **Produrre documentazione di supporto per la manutenzione e l'aggiornamento:** Il SQA assicura che siano prodotti documenti adeguati per supportare le attività di manutenzione e miglioramento del software nel tempo.
- **Gestire la configurazione del software per controllare i cambiamenti:** Il SQA verifica che sia in atto un sistema di gestione della configurazione del software. Questo implica il controllo rigoroso dei cambiamenti al software per garantire che ogni modifica sia documentata, autorizzata e tracciata.
- **Eseguire e superare i test:** Il SQA si assicura che siano eseguiti test appropriati e che il software superi tali test. Questo è essenziale per garantire che il software funzioni correttamente e soddisfi i requisiti definiti.
- **Identificare, documentare e portare all'attenzione della direzione le deficienze e le deviazioni:** Nel caso in cui vengano identificate deficienze o deviazioni rispetto agli standard e alle procedure, il SQA è responsabile di documentarle e portarle all'attenzione della direzione. Questo consente di prendere provvedimenti correttivi tempestivi.

Obiettivi del SQA:

- **Riduzione dei rischi:** L'obiettivo principale del SQA è ridurre i rischi associati allo sviluppo del software. Ciò implica l'identificazione e la gestione proattiva di potenziali problemi che potrebbero verificarsi durante il processo di sviluppo.
- **Monitoraggio appropriato del software e del processo di sviluppo:** Il SQA si impegna a monitorare attentamente sia il software che il processo di sviluppo. Questo monitoraggio è finalizzato a garantire che il software e il processo siano in linea con gli standard e le procedure definiti.

- **Assicurare la piena conformità agli standard e alle procedure:** Il SQA si assicura che ci sia una conformità completa agli standard e alle procedure stabiliti per il software e il processo di sviluppo. Questo contribuisce a garantire la coerenza, la qualità e la trasparenza nel processo di sviluppo.
- **Portare all'attenzione della direzione eventuali carenze nel prodotto, processo o standard:** Il SQA è responsabile di identificare e segnalare alla direzione eventuali carenze nel prodotto software, nel processo di sviluppo o negli standard. Questo è fondamentale per consentire alla gestione di prendere provvedimenti correttivi tempestivi e migliorare continuamente il processo.
- **Non responsabile della produzione di prodotti di qualità:** Va notato che il SQA non è direttamente responsabile della produzione di prodotti di qualità. La sua responsabilità principale è quella di condurre audit approfonditi (ossia esaminare il prodotto in dettaglio, confrontandolo con gli standard e le procedure stabiliti) sulle azioni di qualità e di segnalare alla gestione eventuali deviazioni.
- **Audit delle azioni di qualità e segnalazione di deviazioni:** Il SQA si occupa di eseguire audit sulle azioni di qualità, verificando che siano conformi agli standard e alle procedure. Qualora vengano identificate deviazioni, il SQA è responsabile di segnalarle alla direzione, consentendo così l'adozione di misure correttive.

Standard:

Gli standard rappresentano i criteri a cui i prodotti software devono essere confrontati. In altre parole, gli standard definiscono ciò che dovrebbe essere fatto per garantire la qualità e la coerenza dei prodotti software. Nell'ambito del SQA, ci sono diversi tipi di standard, e alcuni dei requisiti minimi includono:

- **Standard di Documentazione:** Questi standard specificano la forma e il contenuto della documentazione utilizzata per la pianificazione, il controllo e la produzione del prodotto software. L'obiettivo è fornire coerenza nell'approccio alla documentazione durante tutto il progetto.
- **Standard di Progettazione:** Gli standard di progettazione definiscono la forma e il contenuto del prodotto di progettazione. Questi standard stabiliscono regole per tradurre i requisiti software nella progettazione del software e rappresentarli nella documentazione di progettazione. L'obiettivo è garantire una progettazione coerente e di alta qualità.
- **Standard di Codifica:** Gli standard di codifica specificano il linguaggio in cui il codice deve essere scritto e definiscono eventuali restrizioni sull'uso delle caratteristiche del linguaggio. Questi standard definiscono la struttura legale del linguaggio, le convenzioni di stile, le regole per le strutture dati e la documentazione interna del codice. L'obiettivo è garantire un codice coerente, leggibile e manutenibile.

Procedure:

Le procedure rappresentano i criteri a cui i processi di sviluppo e controllo devono essere confrontati. In altre parole, le procedure definiscono come il lavoro deve essere effettivamente svolto, chi deve svolgerlo, quando e cosa fare con i risultati ottenuti. Alcuni esempi di procedure includono:

- **Procedure di Sviluppo:** Queste procedure specificano come dovrebbe essere effettuato il processo di sviluppo del software. Ciò può includere fasi di pianificazione, progettazione, implementazione, test e manutenzione. Le procedure di sviluppo delineano chi è responsabile di ciascuna fase, quando svolgerla e quali risultati dovrebbero essere prodotti.
- **Procedure di Controllo:** Queste procedure stabiliscono come dovrebbero essere controllati e gestiti i cambiamenti nel software. Ciò include il controllo delle versioni del codice, la gestione delle modifiche, e le procedure di revisione e approvazione per garantire che ogni modifica sia gestita in modo controllato e documentato.

SQA Plan:

Il piano SQA è un documento che specifica gli obiettivi del Software Quality Assurance, le attività da svolgere e gli standard e le procedure in base ai quali il lavoro di sviluppo deve essere valutato. Questo piano fornisce una guida dettagliata su come il processo di sviluppo del software sarà gestito e controllato per garantire la qualità del prodotto finale.

TESTING

Verification (Verifica): La verifica è il processo di assicurarsi che il software sia costruito correttamente, conformemente alle specifiche. In altre parole, risponde alla domanda "Stiamo costruendo il prodotto in modo corretto?". Durante la verifica, vengono eseguite attività come revisioni del codice, ispezioni dei documenti e analisi statica per garantire che il software sia conforme alle specifiche.

Validation (Validazione): La validazione è il processo di assicurarsi che il software soddisfi effettivamente le esigenze dell'utente e che stia costruendo il prodotto giusto. Risponde alla domanda "Stiamo costruendo il prodotto giusto?". La validazione è spesso eseguita attraverso attività dinamiche come il testing del software, in cui il sistema viene eseguito con dati di test per osservare il suo comportamento operativo.

Ispezioni del Software:

Le ispezioni del software sono un processo di verifica che si concentra sull'analisi della rappresentazione statica del sistema per scoprire problemi. Questo approccio utilizza tecniche statiche, come la revisione del codice e l'ispezione della documentazione, per individuare errori, discrepanze o problemi potenziali nella fase di sviluppo.

Testing del Software:

Il testing del software è un processo di validazione che coinvolge l'esecuzione del sistema con dati di test per osservare il suo comportamento operativo. Questo approccio dinamico mira a scoprire errori nel software attraverso l'esecuzione e l'osservazione del suo comportamento in vari scenari. Il testing del software può includere test di unità, test di integrazione, test di sistema e test di accettazione.

In breve, la verifica si concentra su "costruire il prodotto correttamente", garantendo che il software sia conforme alle specifiche, mentre la validazione si concentra su "costruire il prodotto giusto", assicurandosi che il software soddisfi le esigenze dell'utente. Le ispezioni del software sono attività di verifica che analizzano staticamente il sistema, mentre il testing del software è un'attività dinamica che esercita e osserva il comportamento del software in esecuzione. Entrambi i processi sono essenziali per garantire la qualità del software durante lo sviluppo.

Tipi di Testing:

- **Validation Testing:** È un tipo di testing inteso a dimostrare che un sistema soddisfa i requisiti dell'utente. Durante il validation testing, si cercano di eseguire test di accettazione che dimostrino che il sistema funziona correttamente secondo i casi di test definiti. Il successo di un test di validazione richiede che il sistema si comporti correttamente secondo i criteri di accettazione prestabiliti.
- **Defect Testing:** È un tipo di testing progettato per scoprire difetti o difetti nel sistema. Mentre il validation testing mira a dimostrare che il sistema funziona correttamente, il defect testing mira a individuare difetti, il che significa che un test di difetto avrà successo se rileva la presenza di difetti nel sistema. In altre parole, il defect testing si concentra sulla ricerca di anomalie o problemi nel comportamento del software.
- **Statistical Testing:** È un tipo di testing progettato per riflettere la frequenza delle operazioni dell'utente. Questo tipo di testing è spesso utilizzato per stime di affidabilità, cercando di modellare e testare il software in modo da riflettere le condizioni reali di utilizzo.

Fasi del Defect Testing:

- **Component Testing (Testing del Componente):** È il testing di singoli componenti di software, come moduli o unità. Solitamente, la responsabilità di questo tipo di testing è affidata allo sviluppatore del componente. I test vengono derivati dall'esperienza del programmatore e mirano a garantire che ogni componente funzioni correttamente in isolamento.
- **Integration Testing (Testing di Integrazione):** È il testing di gruppi di componenti integrati per creare un sistema o sottosistema. La responsabilità di questo tipo di testing è spesso affidata a un team di testing indipendente. I test sono basati su una specifica di sistema e mirano a garantire che i componenti interagiscano correttamente quando vengono combinati.
- **User Testing (Testing dell'Utente):** Non fa parte del processo di defect testing, ma è menzionato come parte del processo di validation testing. Questo tipo di testing, noto anche come validation o acceptance testing, coinvolge gli utenti finali nel testare il sistema per garantire che soddisfi i requisiti utente e funzioni come previsto in un ambiente di produzione.

POLITICHE DI TESTING

Il testing esaustivo, che copre tutte le possibili combinazioni di input, percorsi e condizioni, è teoricamente ideale ma impraticabile nella realtà a causa del numero spesso vastissimo di casi di test possibili per un software complesso.

Poiché il testing esaustivo è impossibile, si fa affidamento su un sottoinsieme rappresentativo di casi di test. La selezione di questo sottoinsieme dovrebbe seguire politiche obiettive formulate dal team di verifica e validazione, garantendo un approccio imparziale al testing. Le politiche di testing dovrebbero essere definite dal team di verifica e validazione, non dal team di sviluppo. Questo assicura che il processo di testing sia indipendente e miri a garantire l'obiettività e l'imparzialità nei confronti del software. È importante testare situazioni tipiche in modo da identificare e correggere i difetti che sono più probabili di verificarsi durante l'utilizzo normale del software.

Test Cases (Casi di Test): I casi di test sono input progettati per testare il sistema e le relative uscite previste se il sistema opera secondo le specifiche. Di solito, i casi di test sono generati manualmente perché è difficile derivare automaticamente l'output di test da specifiche informali.

Test Data (Dati di Test): I dati di test sono input progettati specificamente per testare il sistema. Possono essere generati automaticamente e sono essenziali per eseguire i casi di test. I dati di test devono coprire una varietà di scenari per garantire una copertura adeguata durante il testing.

Black-Box Testing:

Il black-box testing è un approccio al testing in cui il programma viene considerato come una "scatola nera". In questo contesto, il tester non conosce i dettagli interni dell'implementazione del software; si concentra solo sugli input e sugli output del sistema.

- **Derivazione dei Casi di Test:** I casi di test per il black-box testing vengono derivati dalla specifica del sistema. Il tester presenta input al componente o al sistema, esamina l'output corrispondente e verifica se gli output sono conformi a quelli specificati nella specifica del sistema.
- **Scoperta dei Problemi:** Se gli output non corrispondono a quelli specificati, il test ha rilevato un problema nel software. Il black-box testing è anche chiamato functional testing poiché il tester è interessato solo alla funzionalità del software e non alla sua implementazione interna.

Equivalence Partitioning:

L'equivalence partitioning è una tecnica di progettazione dei test in cui i dati di input e i risultati di output sono suddivisi in classi di equivalenza. Ogni classe di equivalenza è un insieme di input o output che dovrebbero essere trattati in modo equivalente dal software.

- **Partizionamento in Classi di Equivalenza:** Nel partizionamento per equivalenza, si suddividono gli input e gli output del sistema in "partizioni di equivalenza". Ogni partizione rappresenta un insieme di input o output che dovrebbero produrre risultati simili o equivalenti.
- **Selezione dei Casi di Test:** I casi di test dovrebbero essere scelti da ciascuna partizione per garantire una copertura adeguata. Ad esempio, se un input deve essere un numero compreso tra 10.000 e 99.999, le partizioni di equivalenza potrebbero essere < 10.000, 10.000-99.999, > 99.999. I casi di test saranno quindi selezionati dai confini di queste partizioni e vicino al punto medio con input validi.

Testing Strutturale:

Il testing strutturale, talvolta indicato come white-box testing, consiste nella creazione di casi di test basati sulla struttura interna del programma. I casi di test derivano dal codice sorgente del programma, e la conoscenza della logica interna del programma viene utilizzata per identificare scenari di test aggiuntivi. L'obiettivo principale del testing strutturale è eseguire tutte le istruzioni del programma, garantendo che ciascuna riga di codice venga eseguita almeno una volta. Non si occupa necessariamente di testare tutte le possibili combinazioni di percorsi attraverso il programma, ma piuttosto di assicurarsi che ciascuna istruzione sia coperta.

Testing dei Percorsi:

Il testing dei percorsi mira a garantire che ciascun percorso attraverso il programma, e di conseguenza ciascuna istruzione, venga eseguito almeno una volta. Un grafo di flusso del programma, che rappresenta nodi per le decisioni del programma e archi per il flusso di controllo, è spesso utilizzato come punto di partenza per il testing dei percorsi. Questa tecnica è più fattibile nelle fasi di testing delle unità o dei moduli, poiché il numero di percorsi attraverso un programma tende a crescere con la sua dimensione.

Percorsi Indipendenti:

I percorsi indipendenti si riferiscono a sequenze uniche di istruzioni e rami in un programma. Nel testing, è essenziale progettare casi di test che coprano questi percorsi indipendenti per garantire una copertura completa del codice.

Testing di Integrazione:

Il testing di integrazione è un livello di testing del software in cui sistemi o sottosistemi completi, composti da componenti integrate, vengono testati. L'obiettivo è verificare che le componenti integrate funzionino insieme come previsto per raggiungere la funzionalità del sistema. Il testing di integrazione è tipicamente un testing black-box, il che significa che i casi di test sono derivati dalla specifica del sistema piuttosto che dai dettagli interni del codice.

Approcci al Testing di Integrazione:

- **Testing Top-Down:** Questo approccio inizia con l'alto livello del sistema e integra le componenti dall'alto verso il basso. Le singole componenti vengono sostituite da stub (versioni simulate semplici) quando appropriato. Questo metodo consente di testare i percorsi di controllo principali in anticipo nel processo.
- **Testing Bottom-Up:** Al contrario, il testing bottom-up integra le singole componenti a livelli, progredendo fino a quando il sistema completo è costruito. Le componenti stub vengono sostituite da componenti effettive man mano che progredisce il testing. Questo approccio consente il testing dei dettagli a basso livello in anticipo nel processo.
- **Combination of Strategies:** In pratica, molti sforzi di testing di integrazione coinvolgono una combinazione di strategie top-down e bottom-up. Questo approccio ibrido mira a sfruttare i vantaggi di entrambi i metodi e può includere tecniche aggiuntive come big-bang integration (integrazione di tutte le componenti in una sola volta) o incremental integration (integrazione e testing in modo incrementale).

TESTING DELLE INTERFACCE

Il testing delle interfacce avviene quando moduli o sottosistemi vengono integrati per formare sistemi più ampi. Gli obiettivi del testing delle interfacce sono rilevare errori derivanti da errori di interfaccia o da supposizioni non valide sulle interfacce. Questo tipo di testing è particolarmente cruciale nello sviluppo orientato agli oggetti, dove gli oggetti sono definiti dalle loro interfacce.

Tipi di Interfacce:

- **Interfacce Parametriche:** Coinvolge il passaggio di dati da una procedura a un'altra attraverso parametri.
- **Interfacce con Memoria Condivisa:** In questo tipo, un blocco di memoria è condiviso tra procedure o componenti.
- **Interfacce Procedurali:** Il sottosistema racchiude un insieme di procedure da chiamare da altri sottosistemi.
- **Interfacce di Passaggio Messaggi:** I sottosistemi richiedono servizi ad altri sottosistemi attraverso il passaggio di messaggi.

Errori delle Interfacce:

- **Abuso dell'Interfaccia:** Si verifica quando un componente chiamante commette un errore nell'uso dell'interfaccia di un altro componente, come fornire parametri nell'ordine sbagliato o del tipo sbagliato.
- **Mancato Comprendimento dell'Interfaccia:** Si verifica quando un componente chiamante incorpora supposizioni incorrette sul comportamento del componente chiamato.
- **Errori Temporal:** Sorgono quando i componenti chiamati e chiamanti operano a velocità diverse, portando all'accesso a informazioni obsolete.

STRESS TESTING

Lo stress testing implica di spingere il sistema oltre il suo carico massimo di progettazione. Questo viene fatto per scoprire difetti che potrebbero non essere evidenti in condizioni operative normali.

Lo stress testing non riguarda solo il sovraccarico del sistema, ma anche il test del suo comportamento in caso di fallimento. I sistemi non dovrebbero fallire in modo catastrofico, e il fallimento non dovrebbe comportare una perdita inaccettabile di servizio o dati.

Lo stress testing è particolarmente rilevante per i sistemi distribuiti. I sistemi distribuiti possono subire una grave degradazione quando le condizioni di rete diventano sovraccariche. Lo stress testing aiuta a identificare quanto bene il sistema può gestire tali scenari e se degrada in modo adeguato.

TESTING ORIENTATO AGLI OGGETTI

Il testing orientato agli oggetti è un approccio complesso che richiede una strategia dettagliata per garantire l'integrità e la correttezza del sistema. Esso inizia con il testing degli oggetti a livello di metodi, assicurandosi che ciascun oggetto si comporti come previsto e che i suoi metodi e attributi funzionino correttamente. Successivamente, si estende al testing delle classi di oggetti, concentrandosi sulla copertura completa dei test per una classe, compresi i test delle operazioni, la configurazione e l'interrogazione degli attributi, e l'esecuzione dell'oggetto in tutti gli stati possibili. Questo processo continua con il testing dei cluster di oggetti cooperanti, che rappresentano il modo in cui gli oggetti collaborano per raggiungere la funzionalità del sistema. Infine, si arriva al testing del sistema orientato agli oggetti completo, che verifica il comportamento globale del sistema.

Le sfide nel testing delle classi di oggetti includono la gestione dell'ereditarietà, poiché le informazioni da testare non sono localizzate in una singola classe, ma possono essere ereditate da classi superiori. L'ereditarietà rende fondamentale considerare l'intera gerarchia delle classi durante il processo di testing.

Nel contesto dell'integrazione degli oggetti, i livelli di integrazione sono meno distinti rispetto ai sistemi tradizionali. Il testing coinvolge cluster di oggetti cooperanti anziché singole funzioni. L'approccio al testing è adattato alla natura della programmazione orientata agli oggetti.

Gli approcci al testing di cluster comprendono il testing basato su casi d'uso o scenario, il thread testing e il testing delle interazioni degli oggetti. Il testing basato su casi d'uso si concentra sulle interazioni dell'utente con il sistema, derivando scenari di test dai casi d'uso. Il thread testing valuta la risposta del sistema agli eventi mentre progrediscono attraverso il sistema. Il testing delle interazioni degli oggetti testa le sequenze di interazioni degli oggetti, sottolineando le collaborazioni tra gli oggetti all'interno del sistema.

Il testing basato su scenario implica l'identificazione di scenari dai casi d'uso, completando la rappresentazione con diagrammi di interazione come i diagrammi di sequenza. Ad esempio, in un sistema di stazione meteorologica, uno scenario potrebbe coinvolgere la generazione di un bollettino meteorologico.

I banchi di lavoro per il testing svolgono un ruolo chiave nel mitigare le sfide e i costi associati al testing, fornendo strumenti e funzionalità per ridurre il tempo e i costi totali del processo di testing. Sono progettati come sistemi aperti, adattabili alle specifiche esigenze di testing di un'organizzazione. Questa flessibilità è essenziale dato che i requisiti di testing possono variare ampiamente tra diverse organizzazioni. In sintesi, il testing orientato agli oggetti richiede un approccio dettagliato e personalizzato, con il supporto di strumenti specifici per ridurre le complessità e i costi associati a questo processo.